

train_diffusion

May 17, 2026

1 Embryo Diffusion - Next-Frame Prediction

This notebook trains a context-conditioned diffusion model to predict frame $t+1$ from the last k grayscale frames.

Conditioning policy in this notebook - Conditioning is strictly temporal: context frames only (k previous timesteps). - There is no focal-plane ID conditioning in the model. - Training still uses sequences from all focal planes by building independent (`embryo`, `plane`) streams.

What is saved each epoch - `diffusion_samples/epoch_XX/mixed` - `diffusion_samples/epoch_XX/transitions` - `diffusion_samples/epoch_XX/rollouts`

1.1 Notebook Map

1. **Configuration & paths** — all hyperparameters and directory roots.
2. **Imports + reproducibility** — third-party libraries, seeds, device selection.
3. **Indexing** (cached CSV) — scan silver dataset, build frame→phase table.
4. **Train/val/test splits** by embryo — stratified, cached to CSV.
5. **Diffusion schedules** — DDPM training schedule + DDIM sampling schedule.
6. **Model** — DiffusionCond UNet wrapper with context conditioning and optional time map.
7. **Image preprocessing & dataset** — center-crop/resize, PairDataset (context→target windows), augmentation, context perturbation.
8. **DataLoaders** — transition-aware weighted sampler for training.
9. **Sampling helpers** — model init, optimizer, DDIM sampling (`ddim_sample_from_model`, `sample_from_model`).
10. **Utilities & metrics**
 - 10.1 Sampling & guidance helpers (model wrappers, DDIM step)
 - 10.2 Perceptual metrics — SSIM & LPIPS
 - 10.3 Phase classifier architecture — `EfficientNetV2MGray` (rollout guidance only)
 - 10.4 Classifier helpers & rollout policy — lazy load/unload, phase prediction, `choose_rollout_next_phase`
11. **Training infrastructure**
 - Training state & LR scheduler
 - Logging, EMA & checkpoint helpers
 - Training step & `one_epoch`
 - Validation metrics & fixed row collections
 - Rollout runner & grid saver (`_run_rollout`, `_save_rollout_grid`)
 - One-step sample grid helper (`_save_sample_grid`)
12. **Training loop** — per-epoch train/val pass, checkpoint saving, epoch sample generation.

13. **Training curves** — load `metric_history.json` and plot loss/rollout/perceptual curves.
14. **Inference helpers** — `predict_next_frame`, `rollout_next_frames`, `visualize_samples`.

1.2 1. Configuration

This cell defines data roots, model/training hyperparameters, and qualitative-sampling settings.

Important knobs - FOCAL_PLANES: planes included in dataset indexing and training streams. - CONTEXT_K: number of context frames used to predict one next frame. - USE_EMA, EMA_DECAY: optional exponential moving average of weights for validation, samples, and best checkpoints. - PERTURB_CONTEXT, CONTEXT_PERTURB_*: mild, recency-weighted corruption of context frames during training. The newest frame is perturbed most often; older frames have progressively smaller probability. - PHASE_LABEL_JITTER, PHASE_LABEL_JITTER_*: training-only adjacent-label jitter for context phase labels, used to make rollout-time classifier mistakes less brittle. - PHASE_POLICY_MODE, PHASE_POLICY_ADVANCE_THRESHOLD: rollout-time phase update policy used during validation and qualitative rollouts when classifier conditioning is enabled. - COMPUTE_VAL_ROLLOUT_METRICS, VAL_ROLLOUT_STEPS, and VAL_ROLLOUT_SAMPLES: fixed rollout-validation settings. - VAL_ROLLOUT_LOSS_WEIGHT: how much the rollout metric contributes to the weighted validation objective used for LR scheduling and the main best checkpoint.

```
[ ]: # ==== CONFIG (edit as needed) ====
# This cell centralizes all knobs for paths, training, and sampling.
from pathlib import Path
import os

# DEBUG MODE: quick test run with reduced epochs.
DEBUG_MODE = False
VERBOSE = True

# Project root resolution: if opened from repo root, auto-jump into /diffusion.
PROJECT_ROOT = Path.cwd().resolve()
if PROJECT_ROOT.name != 'diffusion' and (PROJECT_ROOT / 'diffusion').exists():
    PROJECT_ROOT = PROJECT_ROOT / 'diffusion'

# Data roots
DATA_ROOT = PROJECT_ROOT.parent / 'data'
SILVER_ROOT = DATA_ROOT / 'embryo_dataset_silver'
FOCAL_PLANES = ['F0'] # ['F-45', 'F-30', 'F-15', 'F0', 'F15', 'F30', 'F45']
TARGET_PLANE = 'F0' # default plane for ad hoc visualizations

# Context window for next-frame prediction
CONTEXT_K = 3
MODEL_CONTEXT_K = CONTEXT_K
MODEL_OUT_CHANNELS = 1 # always predict one grayscale next frame

# Time conditioning
USE_TIME_MAP = True
MAX_FRAME_INDEX = 800
```

```

TIME_EMB_DIM = 8

# Context encoder / attention width
CONTEXT_EMB_DIM = 128 # lowered for faster attention
CTX_IMG_CHANNELS = 8
CTX_IMG_HIDDEN = 32

# Keep the same 5-stage UNet depth, but trim the two deepest stages for a
# lighter backbone with a smaller expected quality hit than reducing depth.
MODEL_BLOCK_OUT_CHANNELS = (32, 64, 128, 256, 256)
MODEL_LAYERS_PER_BLOCK = 1 # 1 residual block per stage (halves ops vs default,
    ↳2); keeps full spatial hierarchy
MODEL_ARCH_TAG = 'u' + '-'.join(str(ch) for ch in MODEL_BLOCK_OUT_CHANNELS) +
    ↳f'_l{MODEL_LAYERS_PER_BLOCK}'

# Outputs
OUT_MODELS_DIR = PROJECT_ROOT / 'embryo_diffusion_models'
OUT_SAMPLES_DIR = PROJECT_ROOT / 'diffusion_samples'
CACHE_DIR = PROJECT_ROOT / 'cache'
for d in (OUT_MODELS_DIR, OUT_SAMPLES_DIR, CACHE_DIR):
    d.mkdir(parents=True, exist_ok=True)

# Classifier-driven rollout phase policy (lazy-loaded only for qualitative,
    ↳rollouts)
USE_CLASSIFIER_PHASE_POLICY = True
PHASE_POLICY_MODE = 'monotonic_adjacent_confidence' # one of: 'argmax',
    ↳'monotonic_adjacent', 'monotonic_adjacent_confidence'
PHASE_POLICY_ADVANCE_THRESHOLD = 0.55
assert PHASE_POLICY_MODE in {'argmax', 'monotonic_adjacent',
    ↳'monotonic_adjacent_confidence'}
CLASSIFIER_MODEL_DIR = PROJECT_ROOT.parent / 'classifier' /
    ↳'embryo_phase_models'
CLASSIFIER_CKPT_PATH = CLASSIFIER_MODEL_DIR / 'phase_classifier_f0_f1=643.pth'
if not CLASSIFIER_CKPT_PATH.exists():
    ckpts = sorted(CLASSIFIER_MODEL_DIR.glob('phase_classifier_*.pth'),
    ↳key=lambda p: p.stat().st_mtime, reverse=True)
    CLASSIFIER_CKPT_PATH = ckpts[0] if ckpts else CLASSIFIER_CKPT_PATH

# Transition-aware additive term on transition windows
TRANSITION_LOSS_WEIGHT = 0.5 # Lowered from 1.0 to 0.5
TRAIN_TARGET_TRANSITION_FRAC = 0.20

# Training hyperparameters
IMG_SIZE = 256
BATCH_SIZE = 32
LR = 1e-4

```

```

USE_LR_SCHEDULER = True
LR_MIN = 1e-7
LR_WARMUP_EPOCHS = 10
LR_WARMUP_START_FACTOR = 0.10
EPOCHS = 1000 if not DEBUG_MODE else 6
GRAD_CLIP = 1.0
L1_XO_WEIGHT = 0.02 # Was 0.1 in early experiments, but lowered to 0.02
NUM_WORKERS = 8

# Exponential moving average of model weights.
# EMA weights are used for validation, sample generation, and best-model_
↪ checkpoints.
USE_EMA = True
EMA_DECAY = 0.9995
EMA_UPDATE_AFTER_STEP = 100
EMA_USE_FOR_EVAL = True

# DataLoader stability/perf toggles (for memory-leak diagnostics)
TRAIN_DROP_LAST = True # keep fixed train batch shape
TRAIN_PERSISTENT_WORKERS = True # keep workers alive between epochs
DATALOADER_PREFETCH_FACTOR = 2

# Runtime diagnostics
ENABLE_TORCH_COMPILE = False
LOG_MEMORY_STATS = False
LOG_MEMORY_EVERY_N_BATCHES = 1000
VAL_SPLIT = 0.10
TEST_SPLIT = 0.10
SPLIT_SEED = 42

# Checkpointing/logging
RESUME_TRAINING = False
RUN_STEM = f'nextframe_ctxk{CONTEXT_K}_{IMG_SIZE}px_{MODEL_ARCH_TAG}'
LAST_CKPT_PATH = OUT_MODELS_DIR / f'{RUN_STEM}_last.pt'
BEST_CKPT_PATH = OUT_MODELS_DIR / f'{RUN_STEM}_best.pt'
LOG_PATH = OUT_MODELS_DIR / f'{RUN_STEM}_log.txt'
METRIC_HISTORY_PATH = OUT_MODELS_DIR / f'{RUN_STEM}_metric_history.json'

# Diffusion process
TIMESTEPS = 1000
BETA_START, BETA_END = 1e-4, 2e-2

# DDIM sampling
USE_DDIM_SAMPLING = True
DDIM_STEPS = 250
DDIM_ETA = 0.0

```

```

# Speed
USE_GRAD_CHECKPOINTING = True
USE_TF32 = True

# Sampling visualization
SAMPLE_PLANES = list(FOCAL_PLANES)
SAMPLES_PER_PLANE = 1
ENABLE_ROLLOUT_GRIDS = True
ROLLOUT_GRID_STEPS = 3

# Data augmentation (applied identically to context and target)
AUGMENT = True
AUGMENT_HFLIP = 0.5
AUGMENT_VFLIP = 0.1
AUGMENT_MAX_ROT = 5.0
AUGMENT_BRIGHTNESS = 0.1
AUGMENT_CONTRAST = 0.1

# Context robustness: mildly degrade context frames during training.
# Perturbation is applied per-batch in the main process (one_epoch) so that
# adaptive scaling of these globals takes effect immediately each epoch.
# Recency weighting keeps the strongest perturbation on the newest frame.
PERTURB_CONTEXT = True
CONTEXT_PERTURB_PROB = 0.50      # newest-frame probability (raised from 0.10)
CONTEXT_PERTURB_DECAY = 0.50    # second-newest gets p*decay, third gets
    ↪ p*decay^2, ...
CONTEXT_PERTURB_NOISE_STD = 0.07 # raised from 0.03 to match real rollout
    ↪ error magnitude
CONTEXT_PERTURB_MIN_SCALE = 0.80 # raised from 0.70 (less extreme blur, more
    ↪ representative)

# Adaptive perturbation: automatically intensify noise/prob when the per-step
    ↪ rollout
# gap grows. The ratio is (rollout_loss / rollout_steps) / val_loss -
    ↪ normalising by
# steps makes the comparison scale-independent. A ratio of 1 means rollout
    ↪ matches
# one-step val; values of 2-4 are expected (error compounds); > 4 suggests the
    ↪ model
# is increasingly fragile to noisy context and benefits from harder
    ↪ perturbation.
PERTURB_ADAPT_ENABLED = True
PERTURB_ADAPT_ROLLOUT_RATIO_THRESHOLD = 4.0 # per-step rollout / val; fires
    ↪ only when genuinely bad
PERTURB_ADAPT_NOISE_STD_MAX = 0.10          # conservative cap (was 0.15 -
    ↪ too high)

```

```

PERTURB_ADAPT_PROB_MAX = 0.75 # cap on CONTEXT_PERTURB_PROB
PERTURB_ADAPT_STEP_SIZE = 0.002 # smaller nudge per epoch (was 0.
    ↪ 0.005)

# Scheduled sampling: prevent exposure bias by training on model-generated
    ↪ context.
# With probability SS_MAX_PROB (linearly ramped from 0 over SS_WARMUP_EPOCHS),
    ↪ the
# newest context frame is replaced by the model's own prediction of that frame,
    ↪ generated
# from the prior window [t-k, t-k+1, ..., t-1]. This trains the model to
    ↪ predict well
# when its own (imperfect) output is fed back as context - directly bridging the
# train/rollout distribution gap.
# Fast DDIM (SS_DDIM_STEPS) keeps overhead low (~5-8% extra compute at
    ↪ max_prob=0.25).
SCHEDULED_SAMPLING_ENABLED = True
SCHEDULED_SAMPLING_MAX_PROB = 0.15 # target probability after warmup
SCHEDULED_SAMPLING_WARMUP_EPOCHS = 10 # ramp 0 → max_prob over this many
    ↪ training epochs
SCHEDULED_SAMPLING_DDIM_STEPS = 30 # fast DDIM steps for SS frame generation

# Validation metrics
COMPUTE_VAL_METRICS = False
VAL_METRIC_SAMPLES = 128
VAL_METRIC_MAX_BATCHES = 16
METRIC_SEED = 123

# Rollout-aware validation.
# The rollout metric is logged separately and contributes a small weighted term
    ↪ to the
# validation-selection objective. With only 100 samples the estimate has high
    ↪ variance;
# 125 samples halves the SEM so genuine one-step val improvements register as
    ↪ new bests
# more reliably and the no_improve counter resets before early stopping fires.
COMPUTE_VAL_ROLLOUT_METRICS = True
VAL_ROLLOUT_STEPS = 3
VAL_ROLLOUT_SAMPLES = 125 # raised from 100: halves rollout estimate variance
VAL_ROLLOUT_LOSS_WEIGHT = 0.15
BEST_ROLLOUT_CKPT_PATH = OUT_MODELS_DIR /
    ↪ f'{RUN_STEM}_best_rollout_h{VAL_ROLLOUT_STEPS}.pt'
BEST_ONE_STEP_CKPT_PATH = OUT_MODELS_DIR / f'{RUN_STEM}_best_one_step.pt'

# Min-SNR weighting
USE_MIN_SNR = True

```

```

MIN_SNR_GAMMA = 5.0
assert MIN_SNR_GAMMA > 0

# Prediction parameterization
# USE_RESIDUAL_PREDICTION: denoise delta = (x_tgt - x_last) / 2 rather than the
    ↳ absolute next frame.
# This concentrates the denoising task on the small inter-frame change, which
    ↳ is much easier for
# the model to learn and closes the gap vs the copy-last-frame baseline.
# USE_V_PREDICTION: predict velocity  $v = \text{sqrt\_alpha} * \text{eps} - \text{sqrt\_one\_minus} * x_0$ 
    ↳ instead of eps.
# V-prediction gives better SNR coverage at both ends of the noise schedule
    ↳ (especially with
# the cosine schedule below) and is the default in modern diffusion pipelines.
# NOTE: this flag MUST be consistent with prediction_type in the
    ↳ DDPM Scheduler / DDIM Scheduler
# cells below. Mismatching them causes sampling to produce wrong outputs (the
    ↳ DDIM step
# misinterprets the model's output), while the training loss still looks fine
    ↳ because p_losses
# never calls ddim_scheduler.step().
USE_RESIDUAL_PREDICTION = True
USE_V_PREDICTION = False

print('Project root:', PROJECT_ROOT)
print('Silver root:', SILVER_ROOT)
print('Focal planes:', FOCAL_PLANES)
print('Default display plane:', TARGET_PLANE)
print(f'Context window k={CONTEXT_K}')
print(f'Model output channels={MODEL_OUT_CHANNELS}')
print(f'UNet block_out_channels: {MODEL_BLOCK_OUT_CHANNELS} layers_per_block:
    ↳ {MODEL_LAYERS_PER_BLOCK}')
print(f'Run stem: {RUN_STEM}')
print(f'DDIM: steps={DDIM_STEPS}, eta={DDIM_ETA}, enabled={USE_DDIM_SAMPLING}')
print(f'Grad checkpointing: {USE_GRAD_CHECKPOINTING}')
print(f'Classifier rollout policy: {USE_CLASSIFIER_PHASE_POLICY}')
print(f'Phase policy mode: {PHASE_POLICY_MODE} |
    ↳ threshold={PHASE_POLICY_ADVANCE_THRESHOLD}')
print(f'Classifier checkpoint: {CLASSIFIER_CKPT_PATH}')
print(f'EMA: enabled={USE_EMA}, decay={EMA_DECAY},
    ↳ update_after_step={EMA_UPDATE_AFTER_STEP}, eval={EMA_USE_FOR_EVAL}')
print(f'LR scheduler: enabled={USE_LR_SCHEDULER},
    ↳ warmup_epochs={LR_WARMUP_EPOCHS}, min_lr={LR_MIN}')
print(f'Min-SNR: enabled={USE_MIN_SNR}, gamma={MIN_SNR_GAMMA}')
print(f'Transition loss weight: {TRANSITION_LOSS_WEIGHT}')

```

```

print(f'Train target transition fraction: {100.0 * TRAIN_TARGET_TRANSITION_FRAC:
↳.1f}%')
print(f'Context perturbation: enabled={PERTURB_CONTEXT},␣
↳newest_p={CONTEXT_PERTURB_PROB}, decay={CONTEXT_PERTURB_DECAY}')
print(f'Adaptive perturbation: enabled={PERTURB_ADAPT_ENABLED}, per-step␣
↳ratio_threshold={PERTURB_ADAPT_ROLLOUT_RATIO_THRESHOLD},␣
↳noise_std_max={PERTURB_ADAPT_NOISE_STD_MAX},␣
↳prob_max={PERTURB_ADAPT_PROB_MAX}, step={PERTURB_ADAPT_STEP_SIZE}')
print(f'Scheduled sampling: enabled={SCHEDULED_SAMPLING_ENABLED},␣
↳max_prob={SCHEDULED_SAMPLING_MAX_PROB},␣
↳warmup_epochs={SCHEDULED_SAMPLING_WARMUP_EPOCHS},␣
↳ddim_steps={SCHEDULED_SAMPLING_DDIM_STEPS}')
print(f'Residual prediction: {USE_RESIDUAL_PREDICTION} | V-prediction:␣
↳{USE_V_PREDICTION}')
print(f'L1-x0 weight: {L1_X0_WEIGHT}')
print(f'Rollout val: enabled={COMPUTE_VAL_ROLLOUT_METRICS},␣
↳H={VAL_ROLLOUT_STEPS}, samples={VAL_ROLLOUT_SAMPLES},␣
↳weight={VAL_ROLLOUT_LOSS_WEIGHT}')
if DEBUG_MODE:
    print('DEBUG_MODE is ON - using reduced settings')
for plane in FOCAL_PLANES:
    plane_dir = SILVER_ROOT / plane
    assert plane_dir.exists(), f"Silver {plane} folder not found: {plane_dir}"

```

Project root: /home/khasion/Projects/thesis/diffusion
 Silver root: /home/khasion/Projects/thesis/data/embryo_dataset_silver
 Focal planes: ['F0']
 Default display plane: F0
 Context window k=3
 Model output channels=1
 UNet block_out_channels: (32, 64, 128, 256, 256) layers_per_block: 1
 Run stem: nextframe_ctxk3_256px_u32-64-128-256-256_lpb1
 DDIM: steps=150, eta=0.0, enabled=True
 Grad checkpointing: True
 Classifier rollout policy: True
 Phase policy mode: monotonic_adjacent_confidence | threshold=0.55
 Classifier checkpoint: /home/khasion/Projects/thesis/classifier/embryo_phase_mod
 els/phase_classifier_f0_f1=648.pth
 EMA: enabled=True, decay=0.9995, update_after_step=100, eval=True
 LR scheduler: enabled=True, warmup_epochs=10, min_lr=1e-07
 Min-SNR: enabled=True, gamma=5.0
 Transition loss weight: 0.5
 Train target transition fraction: 20.0%
 Context perturbation: enabled=True, newest_p=0.5, decay=0.5
 Adaptive perturbation: enabled=True, per-step ratio_threshold=4.0,
 noise_std_max=0.1, prob_max=0.75, step=0.002
 Scheduled sampling: enabled=True, max_prob=0.15, warmup_epochs=10, ddim_steps=20

Residual prediction: True | V-prediction: False
L1-x0 weight: 0.02
Rollout val: enabled=True, H=3, samples=125, weight=0.15

1.3 2. Imports and small utilities

The notebook relies on **PyTorch** for training, **diffusers** for the UNet + schedulers, and **LPIPS** for perceptual validation metrics.

```
[ ]: # Imports & utils
import copy
import json, math, random, glob, re, os, hashlib
from contextlib import contextmanager
import numpy as np
import pandas as pd
from PIL import Image
from tqdm import tqdm
from tqdm.auto import tqdm as tqdm_auto
import matplotlib.pyplot as plt

try:
    import psutil
except Exception:
    psutil = None

# PyTorch core
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler
from torchvision.transforms import functional as TF
from torchvision.models import efficientnet_v2_m, efficientnet_v2_s

# Perceptual metric
from lpips import LPIPS

# Diffusers library for UNet and schedulers
from diffusers import UNet2DConditionModel, DDPMScheduler, DDIMScheduler

# AMP (automatic mixed precision) flag
# Set to False to avoid CheckpointError
USE_AMP = torch.cuda.is_available() # avoid CheckpointError (was True)
print(f'AMP enabled: {USE_AMP}')

# TQDM helper: force ETA/remaining to show consistently
# tqdm.auto picks the notebook-aware widget renderer in Jupyter and falls back
# to the standard terminal renderer otherwise.
```

```
TQDM_BAR_FORMAT = "{l_bar}{bar} {n_fmt}/{total_fmt} [{elapsed}<{remaining},  
↳{rate_fmt}]"

def tqdm_bar(iterable, **kwargs):
    """Convenience wrapper with consistent progress-bar formatting."""
    return tqdm_auto(iterable, leave=True, bar_format=TQDM_BAR_FORMAT, **kwargs)
```

AMP enabled: True

1.4 3. Reproducibility and label setup

We seed all relevant RNGs for reproducibility, pick the compute device, and define the ordered phase labels used throughout the notebook.

```
[ ]: # Initialize seed and device (must be before using PHASE_LABELS in later cells)
def seed_everything(seed=42):
    """Make results reproducible across Python/NumPy/PyTorch."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

seed_everything(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

if USE_TF32 and torch.cuda.is_available():
    torch.backends.cuda.matmul.allow_tf32 = True
    torch.backends.cudnn.allow_tf32 = True
    torch.set_float32_matmul_precision('high')
print(f"TF32 enabled: {USE_TF32 and torch.cuda.is_available()}")

# Ordered list of morphokinetic phase labels (target classes)
# pHB is merged into pEB to match the classifier notebook aliasing.
PHASE_LABELS = [
    'pPB2', 'pPNa', 'pPNf', 'p2', 'p3', 'p4', 'p5', 'p6', 'p7', 'p8', 'p9+', 'pM', 'pSB', 'pB', 'pEB']
PHASE_ALIAS = {
    'pHB': 'pEB',
}

def normalize_phase_label(label: str) -> str:
    p = str(label).strip()
    p = re.sub(r"\s+", "", p)
    if p.startswith('t'):
        p = p.replace('t', 'p', 1)
    return PHASE_ALIAS.get(p, p)

NUM_PHASES = len(PHASE_LABELS)
```

```

PHASE_TO_ID = {p: i for i, p in enumerate(PHASE_LABELS)}

# Used only for path parsing/validation in inference helpers.
FOCAL_PLANE_SET = set(FOCAL_PLANES)

```

Using device: cuda
TF32 enabled: True

1.5 4. Build a frame index (cached)

We scan the *silver* dataset and build a table of frame paths + phase labels. This is cached to CSV so you do it once.

```

[ ]: # Build frame index from all focal planes: phase CSV lives next to frames
# Output columns: embryo_id, plane, frame_num (1-based), path, phase_id
INDEX_CACHE = CACHE_DIR / 'silver_all_planes_frame_index_phase_merged.csv'

```

```

def build_index_from_silver():
    rows = []
    # For each focal plane
    for plane in FOCAL_PLANES:
        plane_root = SILVER_ROOT / plane
        embryo_ids = sorted([d.name for d in plane_root.iterdir() if d.
↪is_dir()])
        # For each embryo id included in focal plane
        for eid in tqdm(embryo_ids, desc=f'Indexing {plane}'):
            folder = plane_root / eid
            ann_path = folder / f"{eid}_phases.csv"
            if not ann_path.exists():
                continue
            # Use .jpeg files (silver dataset uses .jpeg)
            img_files = sorted(glob.glob(str(folder / '*.jpeg')))
            if not img_files:
                continue
            # Extract frame numbers from filenames (00037.jpeg → 37)
            frame_numbers = {}
            # For each frame image file
            for f in img_files:
                basename = os.path.basename(f)
                m = re.search(r'(\d+)\.jpeg$', basename)
                if m:
                    frame_num = int(m.group(1))
                    frame_numbers[frame_num] = f

            # Parse CSV (phase, start, end) allowing labels like "t9 +"
            def _parse_phase_csv(path: Path):
                spans = []
                with open(path, 'r', encoding='utf-8') as fh:

```

```

        for line in fh:
            line = line.strip()
            if not line:
                continue
            # normalize delimiters to spaces
            line = re.sub(r"[;,]", " ", line)
            line = re.sub(r"\s+", " ", line).strip()
            m = re.match(r"^(.*?)(\d+)\s+(\d+)$", line)
            if not m:
                continue
            phase_raw = m.group(1).strip()
            start = int(m.group(2))
            end = int(m.group(3))
            spans.append((phase_raw, start, end))
    return spans

def phase_to_id(ph: str):
    ph_norm = normalize_phase_label(ph)
    if ph_norm not in PHASE_TO_ID:
        raise ValueError(f"Unknown phase label after normalization: {ph_norm}")
    return PHASE_TO_ID[ph_norm]

label_spans = []
for phase_raw, start, end in _parse_phase_csv(ann_path):
    try:
        pid = phase_to_id(phase_raw)
        label_spans.append((int(start), int(end), pid))
    except Exception:
        pass
# assign phase per frame using actual frame numbers from filenames
for frame_num, path in frame_numbers.items():
    pid = None
    for s, e, p in label_spans:
        if s <= frame_num <= e: # frame_num is 1-based, matches CSV
            pid = p
            break
    if pid is None:
        continue
    rows.append({'embryo_id': eid, 'plane': plane, 'frame_num': frame_num, 'path': path, 'phase_id': pid})
return pd.DataFrame(rows)

```

1.5.1 4.1 Index caching

We cache the full frame index to speed up restarts. Delete the cache file if you change the dataset layout.

```
[ ]: if INDEX_CACHE.exists():
    index_df = pd.read_csv(INDEX_CACHE)
else:
    index_df = build_index_from_silver()
    index_df.to_csv(INDEX_CACHE, index=False)

print('Indexed frames (full):', len(index_df))
print('Planes:', sorted(index_df['plane'].unique().tolist()))
index_df.head()
```

```
Indexed frames (full): 289675
Planes: ['F0']
```

```
[ ]: embryo_id plane  frame_num  \
0    AA83-7    F0          5
1    AA83-7    F0          6
2    AA83-7    F0          7
3    AA83-7    F0          8
4    AA83-7    F0          9
```



```
path  phase_id
0  /home/khasion/Projects/thesis/data/embryo_data...  0
1  /home/khasion/Projects/thesis/data/embryo_data...  0
2  /home/khasion/Projects/thesis/data/embryo_data...  0
3  /home/khasion/Projects/thesis/data/embryo_data...  0
4  /home/khasion/Projects/thesis/data/embryo_data...  0
```

1.6 5. Train/val/test splits (by embryo)

We split **by embryo ID**, not by frame, to avoid leaking information across sets. Splits are cached on disk.

```
[ ]: # Build embryo splits (no embryo appears in more than one split)
SPLIT_CACHE_STEM = (
    f"splits_embryo_phase_ctxk{CONTEXT_K}_"
    f"seed{SPLIT_SEED}_v{VAL_SPLIT}_t{TEST_SPLIT}"
    + ("_debug" if DEBUG_MODE else "")
)
TRAIN_SPLIT_CACHE = CACHE_DIR / f"{SPLIT_CACHE_STEM}_train.csv"
VAL_SPLIT_CACHE = CACHE_DIR / f"{SPLIT_CACHE_STEM}_val.csv"
TEST_SPLIT_CACHE = CACHE_DIR / f"{SPLIT_CACHE_STEM}_test.csv"

def stratified_split(ids: list[str], labels: dict[str, int], val_frac: float,
    test_frac: float, seed: int):
    """Stratified split by dominant phase label per embryo.
    Args:
        ids: List of embryo IDs to split.
        labels: Mapping from embryo ID to dominant phase label ID.
```

```

    val_frac: Fraction of embryos to assign to validation set.
    test_frac: Fraction of embryos to assign to test set.
    seed: Random seed for reproducibility.
Returns:
    train_ids, val_ids, test_ids: Sets of embryo IDs for each split.
"""
rng = np.random.RandomState(seed)
train_ids, val_ids, test_ids = [], [], []
by_label: dict[int, list[str]] = {}
for eid in ids:
    by_label.setdefault(labels[eid], []).append(eid) # group by label,
↳Example: by_label == {0: ['e1', 'e3'], 1: ['e2']}
for label, group in by_label.items():
    group = list(group)
    rng.shuffle(group)
    n = len(group)
    n_test = int(round(n * test_frac))
    n_val = int(round(n * val_frac))
    n_test = min(n_test, n)
    n_val = min(n_val, n - n_test)
    test_ids.extend(group[:n_test])
    val_ids.extend(group[n_test:n_test + n_val])
    train_ids.extend(group[n_test + n_val:])
return set(train_ids), set(val_ids), set(test_ids)

# In DEBUG mode, sample only a few embryos
index_for_pairs = index_df.copy()
if DEBUG_MODE:
    unique_embryos = index_for_pairs["embryo_id"].unique()
    if len(unique_embryos) > 10:
        debug_embryos = np.random.choice(unique_embryos, 10, replace=False)
        index_for_pairs = index_for_pairs[index_for_pairs["embryo_id"].
↳isin(debug_embryos)]
        print(f"DEBUG_MODE: Using only {len(debug_embryos)} embryos for
↳testing")
    else:
        debug_embryos = unique_embryos

print("Indexed frames available:", len(index_for_pairs))
print("Planes in index:", sorted(index_for_pairs["plane"].unique().tolist()))

def _debug_split(ids: list[str]):
    """Small deterministic split for quick debug runs."""
    ids = list(ids)
    rng = np.random.RandomState(SPLIT_SEED)
    rng.shuffle(ids)
    if len(ids) >= 3:

```

```

        return set(ids[2:]), {ids[1]}, {ids[0]}
    if len(ids) == 2:
        return {ids[1]}, {ids[0]}, set()
    if len(ids) == 1:
        return {ids[0]}, set(), set()
    return set(), set(), set()

if TRAIN_SPLIT_CACHE.exists() and VAL_SPLIT_CACHE.exists() and TEST_SPLIT_CACHE.
    exists():
        train_df = pd.read_csv(TRAIN_SPLIT_CACHE)
        val_df = pd.read_csv(VAL_SPLIT_CACHE)
        test_df = pd.read_csv(TEST_SPLIT_CACHE)
    else:
        # Dominant phase per embryo for stratification
        embryo_phase = (
            index_for_pairs.groupby("embryo_id")["phase_id"]
            .agg(lambda s: s.value_counts().idxmax())
            .to_dict()
        )
        embryo_ids = list(embryo_phase.keys())
        if DEBUG_MODE:
            train_ids, val_ids, test_ids = _debug_split(embryo_ids)
        else:
            train_ids, val_ids, test_ids = stratified_split(embryo_ids,
                embryo_phase, VAL_SPLIT, TEST_SPLIT, SPLIT_SEED)
            train_df = pd.DataFrame({"embryo_id": list(train_ids)})
            val_df = pd.DataFrame({"embryo_id": list(val_ids)})
            test_df = pd.DataFrame({"embryo_id": list(test_ids)})
            train_df.to_csv(TRAIN_SPLIT_CACHE, index=False)
            val_df.to_csv(VAL_SPLIT_CACHE, index=False)
            test_df.to_csv(TEST_SPLIT_CACHE, index=False)

train_ids = set(train_df["embryo_id"].tolist())
val_ids = set(val_df["embryo_id"].tolist())
test_ids = set(test_df["embryo_id"].tolist())

# DEBUG safeguard: ensure we have a non-empty val set for metrics/samples
if DEBUG_MODE:
    all_ids = sorted(set(index_for_pairs["embryo_id"].unique().tolist()))
    if len(val_ids) == 0 and len(all_ids) >= 2:
        train_ids, val_ids, test_ids = _debug_split(all_ids)
        train_df = pd.DataFrame({"embryo_id": list(train_ids)})
        val_df = pd.DataFrame({"embryo_id": list(val_ids)})
        test_df = pd.DataFrame({"embryo_id": list(test_ids)})
        train_df.to_csv(TRAIN_SPLIT_CACHE, index=False)
        val_df.to_csv(VAL_SPLIT_CACHE, index=False)
        test_df.to_csv(TEST_SPLIT_CACHE, index=False)

```

```

print(f"Splits cached: {TRAIN_SPLIT_CACHE.name}, {VAL_SPLIT_CACHE.name},  

↳{TEST_SPLIT_CACHE.name}")
print(f"Train embryos: {len(train_ids)} | Val embryos: {len(val_ids)} | Test  

↳embryos: {len(test_ids)}")

if VERBOSE:
    def _split_stats(name: str, ids: set[str]):
        df_split = index_for_pairs[index_for_pairs["embryo_id"].isin(ids)]
        frames = len(df_split)
        sequences = df_split.groupby(["embryo_id", "plane"]).ngroups
        planes = df_split["plane"].nunique()
        embryos = df_split["embryo_id"].nunique()
        print(f"[{name}] embryos={embryos} | planes={planes} |  

↳sequences={sequences} | frames={frames}")
        _split_stats("train", train_ids)
        _split_stats("val", val_ids)
        _split_stats("test", test_ids)

```

Indexed frames available: 289675

Planes in index: ['F0']

Splits cached: splits_embryo_phase_ctxk3_seed42_v0.1_t0.1_train.csv,

splits_embryo_phase_ctxk3_seed42_v0.1_t0.1_val.csv,

splits_embryo_phase_ctxk3_seed42_v0.1_t0.1_test.csv

Train embryos: 565 | Val embryos: 69 | Test embryos: 69

[train] embryos=565 | planes=1 | sequences=565 | frames=231119

[val] embryos=69 | planes=1 | sequences=69 | frames=29963

[test] embryos=69 | planes=1 | sequences=69 | frames=28593

1.7 6. Diffusion schedules

We use **DDPM** for training noise schedules and **DDIM** for fast sampling.

```

[ ]: # Diffusion schedules using diffusers
#
# Cosine schedule (squaredcos_cap_v2) is better suited than scaled_linear for
# near-static signals: it allocates more capacity in the mid-SNR regime and
# avoids over-noising small inter-frame changes.
#
# prediction_type is set dynamically from USE_V_PREDICTION in config:
# 'epsilon' -> model predicts noise eps (current default,  

↳USE_V_PREDICTION=False)
# 'v_prediction' -> model predicts velocity v = sqrt_alpha * eps -  

↳sqrt_one_minus * x0
# CRITICAL: this flag MUST match USE_V_PREDICTION, otherwise DDIM sampling  

↳misinterprets

```

```

# the model output while the training loss still decreases normally (p_losses_
↳ never calls
# ddim_scheduler.step()), creating a silent mismatch that only shows up in_
↳ rollout metrics.

# Training schedule (DDPM)
_sched_pred_type = 'v_prediction' if USE_V_PREDICTION else 'epsilon'
scheduler = DDPMSScheduler(
    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='squaredcos_cap_v2',
    variance_type='fixed_small',
    prediction_type=_sched_pred_type,
)

# Sampling schedule (DDIM for fast inference)
ddim_scheduler = DDIMScheduler(
    num_train_timesteps=TIMESTEPS,
    beta_start=BETA_START,
    beta_end=BETA_END,
    beta_schedule='squaredcos_cap_v2',
    clip_sample=False,
    set_alpha_to_one=False,
    prediction_type=_sched_pred_type,
)

print(f'Scheduler prediction type: {_sched_pred_type}')

# Pre-compute noise schedule for training
betas = scheduler.betas.to(device)
alphas = (1 - betas).to(device)
alphas_cumprod = torch.cumprod(alphas, dim=0).to(device)
sqrt_alphas_cumprod = torch.sqrt(alphas_cumprod)
sqrt_one_minus_alphas_cumprod = torch.sqrt(1.0 - alphas_cumprod)
alphas_cumprod_prev = torch.cat([torch.ones(1, device=device), alphas_cumprod[:
↳ -1]])

```

Scheduler prediction type: epsilon

1.8 7. Model: Context-Conditioned UNet

The model predicts diffusion noise for a single grayscale next frame.

Conditioning inputs are: - context image stack [B, k, H, W] - optional absolute frame-index time map

Phase conditioning has been removed from the model. The phase classifier is used only at sampling time for labelling generated frames and for next-phase guidance in the evaluation notebook.

```
[ ]: # Diffusion model using diffusers UNet2DConditionModel (context cross-attention)
from models import DiffusionCond as _DiffusionCondBase

class DiffusionCond(_DiffusionCondBase):
    """Training wrapper that adds q_sample / p_losses on top of the shared_
    ↪architecture."""
    def __init__(self, context_k: int = MODEL_CONTEXT_K, use_time_map: bool =
    ↪USE_TIME_MAP):
        super().__init__(
            context_k=context_k,
            use_time_map=use_time_map,
            img_size=IMG_SIZE,
            context_emb_dim=CONTEXT_EMB_DIM,
            ctx_img_channels=CTX_IMG_CHANNELS,
            ctx_img_hidden=CTX_IMG_HIDDEN,
            output_channels=MODEL_OUT_CHANNELS,
            max_frame_index=MAX_FRAME_INDEX,
            time_emb_dim=TIME_EMB_DIM,
            block_out_channels=MODEL_BLOCK_OUT_CHANNELS,
            layers_per_block=MODEL_LAYERS_PER_BLOCK,
        )

    def q_sample(self, x0, t, noise=None):
        if noise is None:
            noise = torch.randn_like(x0)
        sqrt_alpha = sqrt_alphas_cumprod[t][:, None, None, None]
        sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t][:, None, None, None]
        return sqrt_alpha * x0 + sqrt_one_minus * noise, noise

    def p_losses(self, x_tgt, x_ctx, time_map: torch.Tensor | None = None,
    ↪frame_idx: torch.Tensor | None = None, return_x0_pred: bool = False,
    ↪return_per_sample: bool = False):
        """Training loss.

        With USE_RESIDUAL_PREDICTION the model denoises
            delta = (x_tgt - x_last) / 2 in [-1, 1]
        instead of the absolute target frame. At inference time_
    ↪sample_from_model
            adds the last context frame back to recover the absolute prediction.

        With USE_V_PREDICTION the network predicts the velocity
            v = sqrt_alpha * eps - sqrt_one_minus * x0
        rather than the noise eps directly. The DDIM scheduler is configured_
    ↪with
            prediction_type='v_prediction' so its step() call is also consistent.
        """
        B = x_tgt.size(0)
```

```

t = torch.randint(0, Timesteps, (B,)), device=x_tgt.device, dtype=torch.
↳long)

# ---- Select denoising target ----
if USE_RESIDUAL_PREDICTION:
    x_last = x_ctx[:, -1:] # [B, 1, H, W] - most
↳recent context frame
    x_target = (x_tgt - x_last) * 0.5 # normalize delta to [-1,
↳1]
else:
    x_target = x_tgt

x_noisy, noise = self.q_sample(x_target, t)

sqrt_alpha = sqrt_alphas_cumprod[t][:, None, None, None]
sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t][:, None, None, None]

pred = self.forward(x_noisy, t, x_ctx, time_map=time_map,
↳frame_idx=frame_idx)

# ---- Compute prediction target and raw per-sample MSE ----
if USE_V_PREDICTION:
    # v-target = sqrt_alpha * eps - sqrt_one_minus * x0
    pred_target = sqrt_alpha * noise - sqrt_one_minus * x_target
else:
    pred_target = noise

raw_per = (pred - pred_target).pow(2).mean(dim=[1, 2, 3])

# ---- Min-SNR loss weighting ----
if USE_MIN_SNR:
    snr = alphas_cumprod[t] / (1 - alphas_cumprod[t])
    snr = torch.clamp(snr, min=1e-8)
    weights = torch.minimum(snr, snr.new_tensor(MIN_SNR_GAMMA)) / snr
    mse_per_weighted = raw_per * weights
    mse = mse_per_weighted.mean()
else:
    mse_per_weighted = raw_per
    mse = raw_per.mean()

# ---- Recover predicted clean target (x0_pred) for the auxiliary L1
↳loss ----
if USE_V_PREDICTION:
    # x0 = sqrt_alpha * x_noisy - sqrt_one_minus * v
    x0_pred = sqrt_alpha * x_noisy - sqrt_one_minus * pred
else:
    x0_pred = (x_noisy - sqrt_one_minus * pred) / (sqrt_alpha + 1e-8)

```

```

# x0_pred and x_target are both in the same space (delta or absolute)
l1_per = (x0_pred - x_target).abs().mean(dim=[1, 2, 3])
l1 = l1_per.mean()

if return_x0_pred and return_per_sample:
    return mse, l1, x0_pred, mse_per_weighted, l1_per
if return_x0_pred:
    return mse, l1, x0_pred
if return_per_sample:
    return mse, l1, mse_per_weighted, l1_per
return mse, l1

```

1.9 8. Image preprocessing and dataset

We center-crop and resize frames, convert to grayscale tensors in $[-1,1]$, and build **context**→**target** pairs (strictly adjacent in time).

```

[ ]: def _preprocess_pil(img: Image.Image, size: int = IMG_SIZE) -> Image.Image:
    """Center-crop to square and resize to model input size."""
    w, h = img.size
    m = min(w, h)
    left = (w - m) // 2
    top = (h - m) // 2
    img = img.crop((left, top, left + m, top + m))
    if size is not None and (img.size[0] != size or img.size[1] != size):
        img = img.resize((size, size), Image.BICUBIC)
    return img

def load_pil(path: str, size: int = IMG_SIZE) -> Image.Image:
    """Load grayscale image and preprocess."""
    img = Image.open(path).convert('L')
    return _preprocess_pil(img, size=size)

def pil_to_tensor(img: Image.Image) -> torch.Tensor:
    """Convert PIL to float tensor in [-1, 1]."""
    arr = np.array(img, dtype=np.float32) / 255.0
    arr = arr * 2 - 1 # [-1, 1]
    return torch.from_numpy(arr).unsqueeze(0)

def _augment_stack(ctx_list: list[Image.Image], tgt):
    """Apply identical random augmentation to every context frame and target_
    ↪ frame(s)."""
    if not AUGMENT:
        return ctx_list, tgt

    do_hflip = AUGMENT_HFLIP > 0 and random.random() < AUGMENT_HFLIP

```

```

do_vflip = AUGMENT_VFLIP > 0 and random.random() < AUGMENT_VFLIP
angle = random.uniform(-AUGMENT_MAX_ROT, AUGMENT_MAX_ROT) if
↪AUGMENT_MAX_ROT and AUGMENT_MAX_ROT > 0 else 0.0
brightness = 1.0 + random.uniform(-AUGMENT_BRIGHTNESS, AUGMENT_BRIGHTNESS)
↪if AUGMENT_BRIGHTNESS and AUGMENT_BRIGHTNESS > 0 else None
contrast = 1.0 + random.uniform(-AUGMENT_CONTRAST, AUGMENT_CONTRAST) if
↪AUGMENT_CONTRAST and AUGMENT_CONTRAST > 0 else None

def _apply(img: Image.Image) -> Image.Image:
    if do_hflip:
        img = TF.hflip(img)
    if do_vflip:
        img = TF.vflip(img)
    if angle != 0.0:
        img = TF.rotate(img, angle, interpolation=TF.InterpolationMode.
↪BILINEAR)
    if brightness is not None:
        img = TF.adjust_brightness(img, brightness)
    if contrast is not None:
        img = TF.adjust_contrast(img, contrast)
    return img

ctx_list = [_apply(img) for img in ctx_list]
if isinstance(tgt, list):
    tgt = [_apply(img) for img in tgt]
else:
    tgt = _apply(tgt)
return ctx_list, tgt

def _perturb_context_frames_tensor(x_ctx: torch.Tensor) -> torch.Tensor:
    """Mildly degrade context frames with recency-weighted probabilities.

    The newest frame gets CONTEXT_PERTURB_PROB. Older frames are perturbed with
    CONTEXT_PERTURB_PROB * CONTEXT_PERTURB_DECAY**age, where age=1 is the
    second-newest frame. This matches rollout drift without corrupting the whole
    context window too aggressively.
    """
    if not PERTURB_CONTEXT or CONTEXT_PERTURB_PROB <= 0:
        return x_ctx
    if x_ctx.dim() != 3 or x_ctx.shape[0] <= 0:
        return x_ctx

    base_prob = max(0.0, min(1.0, float(CONTEXT_PERTURB_PROB)))
    decay = max(0.0, min(1.0, float(CONTEXT_PERTURB_DECAY)))
    x_out = None
    k = int(x_ctx.shape[0])

```

```

for idx in range(k):
    age = (k - 1) - idx # newest frame has age 0
    prob = base_prob * (decay ** age)
    if prob <= 0 or random.random() >= prob:
        continue

    if x_out is None:
        x_out = x_ctx.clone()
    frame = x_out[idx:idx + 1].unsqueeze(0) # [1,1,H,W]

    # Downsample/upsample adds a mild blur that is closer to rollout
    ↳ artifacts than strong augmentation.
    scale = random.uniform(float(CONTEXT_PERTURB_MIN_SCALE), 1.0)
    if scale < 0.999:
        h, w = frame.shape[-2:]
        h_small = max(16, int(round(h * scale)))
        w_small = max(16, int(round(w * scale)))
        frame = F.interpolate(frame, size=(h_small, w_small),
    ↳ mode='bilinear', align_corners=False)
        frame = F.interpolate(frame, size=(h, w), mode='bilinear',
    ↳ align_corners=False)

    frame = F.avg_pool2d(frame, kernel_size=3, stride=1, padding=1)

    if CONTEXT_PERTURB_NOISE_STD and CONTEXT_PERTURB_NOISE_STD > 0:
        frame = frame + torch.randn_like(frame) *
    ↳ float(CONTEXT_PERTURB_NOISE_STD)

    x_out[idx] = frame[0, 0].clamp(-1, 1)

return x_ctx if x_out is None else x_out

def load_image(path: str, size: int = IMG_SIZE):
    """Load image and return tensor [1,H,W] in [-1, 1]."""
    img = load_pil(path, size=size)
    return pil_to_tensor(img)

def denorm(x):
    """Convert [-1,1] -> [0,1] for visualization."""
    return (x.clamp(-1,1) + 1) / 2

```

1.9.1 8.1 Dataset (Context -> Target Windows)

PairDataset builds deterministic windows per (embryo_id, plane) stream: - Context: frames [t-k+1..t] from one focal plane - Target: frame t+1 from the same focal plane

This keeps each sample single-plane while training still covers all planes.

For training only, the recency-weighted context frames can be **mildly degraded** before the batch is returned. This is a deliberate rollout-robustness trick: the model learns to tolerate context that is slightly worse than perfect ground truth.

Phase-label robustness is applied later in the training objective, not here, so transition labels and metadata remain ground truth.

```
[ ]: class PairDataset(Dataset):
    """Windowed next-frame pairs built from independent (embryo, plane)
    ↪sequences."""

    def __init__(
        self,
        index_df: pd.DataFrame,
        embryo_ids: set[str],
        split_seed: int,
        split_name: str,
        planes: list[str] | None = None,
        build_transition_index: bool = True,
    ):
        self.planes = list(planes) if planes is not None else list(FOCAL_PLANES)

        self.df = index_df[
            index_df['embryo_id'].isin(set(embryo_ids)) & index_df['plane'].
            ↪isin(set(self.planes))
        ].copy().reset_index(drop=True)

        self.split_seed = int(split_seed)
        self.split_name = split_name
        self.build_transition_index = build_transition_index
        self.rng = np.random.RandomState(split_seed + {'train': 0, 'val': 1,
            ↪'test': 2}.get(split_name, 0))

        self.sequences: list[str] = []
        self.sequences_by_plane: dict[str, list[str]] = {}
        self.frames_by_seq: dict[str, list[int]] = {}
        self.path_by_seq: dict[str, dict[int, str]] = {}
        self.phase_by_seq: dict[str, dict[int, int]] = {}
        self.valid_t_ends_by_seq: dict[str, list[int]] = {}
        self.transition_t_ends_by_seq: dict[str, list[int]] = {}
        self.non_transition_t_ends_by_seq: dict[str, list[int]] = {}
        self.max_frame_by_seq: dict[str, int] = {}

        self.windows: list[dict] = []
        self.windows_by_plane: dict[str, list[dict]] = {}
        self.transition_windows_by_plane: dict[str, list[dict]] = {}
        self.non_transition_windows_by_plane: dict[str, list[dict]] = {}
```

```

self._skipped_short = 0
self._skipped_no_tgt = 0
self._build_index()

def _build_index(self):
    """Build valid context->target windows for every selected plane stream.
    ↪ """
    for eid, g_eid in self.df.groupby('embryo_id'):
        by_plane = {plane: g.sort_values('frame_num').
        ↪ reset_index(drop=True) for plane, g in g_eid.groupby('plane')}

        frame_set_by_plane: dict[str, set[int]] = {}
        path_by_plane: dict[str, dict[int, str]] = {}
        phase_by_plane: dict[str, dict[int, int]] = {}
        for plane, g_plane in by_plane.items():
            if plane not in self.planes:
                continue
            frame_nums = g_plane['frame_num'].astype(int).tolist()
            frame_set_by_plane[plane] = set(frame_nums)
            path_by_plane[plane] = dict(zip(frame_nums, g_plane['path'].
        ↪ tolist()))
            phase_by_plane[plane] = dict(zip(frame_nums,
        ↪ g_plane['phase_id'].tolist()))

        for plane in self.planes:
            if plane not in frame_set_by_plane:
                continue

            tgt_frames = sorted(frame_set_by_plane[plane])
            if len(tgt_frames) <= CONTEXT_K:
                self._skipped_short += 1
                continue

            valid_t_ends = []
            for t_end in tgt_frames:
                if (t_end + 1) not in frame_set_by_plane[plane]:
                    continue
                ctx_ok = True
                for f in range(t_end - CONTEXT_K + 1, t_end + 1):
                    if f not in frame_set_by_plane[plane]:
                        ctx_ok = False
                        break
                if ctx_ok:
                    valid_t_ends.append(t_end)

            if not valid_t_ends:
                self._skipped_no_tgt += 1

```

```

        continue

    phase_map = phase_by_plane[plane]
    if self.build_transition_index:
        transition_t_ends = []
        non_transition_t_ends = []
        for t in valid_t_ends:
            phase_t = phase_map.get(t)
            phase_next = phase_map.get(t + 1)
            if phase_next is None:
                continue
            if phase_next != phase_t:
                transition_t_ends.append(t)
            else:
                non_transition_t_ends.append(t)
        else:
            transition_t_ends = []
            non_transition_t_ends = []

    seq_key = f"{eid}|{plane}"
    max_frame = max(tgt_frames)
    self.sequences.append(seq_key)
    self.sequences_by_plane.setdefault(plane, []).append(seq_key)
    self.frames_by_seq[seq_key] = tgt_frames
    self.path_by_seq[seq_key] = path_by_plane[plane]
    self.phase_by_seq[seq_key] = phase_map
    self.valid_t_ends_by_seq[seq_key] = valid_t_ends
    self.transition_t_ends_by_seq[seq_key] = transition_t_ends
    self.non_transition_t_ends_by_seq[seq_key] = non_transition_t_ends
    self.max_frame_by_seq[seq_key] = max_frame

    for t_end in valid_t_ends:
        # prev_last_path: the frame immediately before the context
        window (t - k).
        # Used for scheduled sampling to generate a realistic
        "fake" last context frame.
        prev_last_t = t_end - CONTEXT_K
        prev_last_path = path_by_plane[plane].get(prev_last_t, '')
        if SCHEDULED_SAMPLING_ENABLED else ''
        row = {
            'embryo_id': str(eid),
            'seq_key': seq_key,
            'plane': str(plane),
            't_end': int(t_end),
            'tgt_frame': int(t_end + 1),
            'max_frame': int(max_frame),

```

```

        'prev_last_path': prev_last_path,
    }
    self.windows.append(row)
    self.windows_by_plane.setdefault(plane, []).append(row)
    if self.build_transition_index and t_end in_
↪transition_t_ends:
        self.transition_windows_by_plane.setdefault(plane, []).
↪append(row)
        elif self.build_transition_index:
            self.non_transition_windows_by_plane.setdefault(plane,
↪[]).append(row)

    if VERBOSE:
        frames_per_seq = [len(self.frames_by_seq[s]) for s in self.
↪sequences]
        valid_tgts_per_seq = [len(self.valid_t_ends_by_seq[s]) for s in_
↪self.sequences]
        trans_tgts_per_seq = [len(self.transition_t_ends_by_seq[s]) for s_
↪in self.sequences]
        total_frames = int(np.sum(frames_per_seq)) if frames_per_seq else 0

        def _avg(vals):
            return float(np.mean(vals)) if vals else 0.0

        def _minmax(vals):
            return (int(np.min(vals)), int(np.max(vals))) if vals else (0,
↪0)

        fp_min, fp_max = _minmax(frames_per_seq)
        vt_min, vt_max = _minmax(valid_tgts_per_seq)
        tr_min, tr_max = _minmax(trans_tgts_per_seq)
        print(
            f"[PairDataset:{self.split_name}] sequences={len(self.
↪sequences)} | "
            f"frames={total_frames} | frames/seq avg={_avg(frames_per_seq):.
↪1f} (min={fp_min}, max={fp_max}) | "
            f"valid windows/seq avg={_avg(valid_tgts_per_seq):.1f}_
↪(min={vt_min}, max={vt_max}) | "
            f"transition windows/seq avg={_avg(trans_tgts_per_seq):.1f}_
↪(min={tr_min}, max={tr_max}) | "
            f"skipped short={self._skipped_short}, skipped no-target={self.
↪_skipped_no_tgt}"
        )

    def __len__(self):
        return len(self.windows)

```

```

def _build_row(self, row: dict):
    """Expand one metadata row into file paths and phase ids used by
    ↪ loaders/plots."""
    seq_key = str(row['seq_key'])
    plane_name = str(row['plane'])
    t_end = int(row['t_end'])

    ctx_frames = [t_end - i for i in range(CONTEXT_K - 1, -1, -1)]
    ctx_paths = [self.path_by_seq[seq_key][f] for f in ctx_frames]
    phase_seq = [self.phase_by_seq[seq_key].get(f) for f in ctx_frames]

    tgt_path = self.path_by_seq[seq_key][t_end + 1]
    tgt_phase_id = self.phase_by_seq[seq_key].get(t_end + 1)

    out = dict(row)
    out['ctx_frames'] = ctx_frames
    out['ctx_paths'] = ctx_paths
    out['ctx_paths_vis'] = list(ctx_paths)
    out['ctx_frames_vis'] = list(ctx_frames)
    out['ctx_planes_vis'] = [plane_name] * len(ctx_frames)
    out['phase_seq'] = phase_seq
    out['phase_seq_vis'] = phase_seq
    out['ctx_planes'] = [plane_name]
    out['model_context_k'] = len(ctx_paths)
    out['tgt_path'] = tgt_path
    out['tgt_phase_id'] = tgt_phase_id
    out['prev_last_path'] = row.get('prev_last_path', '')
    return out

def sample_pairs_for_plane(self, plane: str, n: int, mode: str = 'any'):
    """Utility for qualitative sampling: transitions, non-transitions, or
    ↪ all."""
    if mode == 'transition':
        rows = self.transition_windows_by_plane.get(plane, [])
    elif mode == 'non_transition':
        rows = self.non_transition_windows_by_plane.get(plane, [])
    else:
        rows = self.windows_by_plane.get(plane, [])
    if not rows or n <= 0:
        return []
    picks = [rows[self.rng.randint(len(rows))] for _ in range(n)]
    return [self._build_row(r) for r in picks]

def sample_transition_pairs_for_plane(self, plane: str, n: int):
    return self.sample_pairs_for_plane(plane, n, mode='transition')

```

```

def sample_non_transition_pairs_for_plane(self, plane: str, n: int):
    return self.sample_pairs_for_plane(plane, n, mode='non_transition')

def __getitem__(self, idx):
    # Returns tensors for one training example: x_ctx [k,H,W], x_tgt [1,H,W].
    # NOTE: context perturbation is NOT applied here - it is applied
    # per-batch in
    # one_epoch (main process) so that adaptive parameter updates take
    # effect immediately.
    #
    # For scheduled sampling (training split only): also loads the frame
    # immediately
    # before the context window (prev_last at t-k) so that one_epoch can
    # generate a
    # model-predicted "fake" last context frame and replace x_ctx[:, -1].
    row = self._build_row(self.windows[idx])
    ctx_pils = [load_pil(p) for p in row['ctx_paths']]
    tgt_pil = load_pil(row['tgt_path'])

    # Load prev_last frame for scheduled sampling (training split only).
    # Augmentation is applied to all frames together for consistency.
    prev_last_pil = None
    prev_last_path = row.get('prev_last_path', '')
    if SCHEDULED_SAMPLING_ENABLED and self.split_name == 'train' and
    prev_last_path:
        try:
            prev_last_pil = load_pil(prev_last_path)
        except Exception:
            prev_last_pil = None

    if prev_last_pil is not None:
        # Include prev_last at the front of ctx_list so it receives the same
        # random flip/rotation/brightness transforms as the rest of the
        # context.
        aug_all, tgt_pil = _augment_stack([prev_last_pil] + ctx_pils,
        tgt_pil)
        prev_last_pil = aug_all[0]
        ctx_pils = aug_all[1:]
    else:
        ctx_pils, tgt_pil = _augment_stack(ctx_pils, tgt_pil)

    x_ctx = torch.stack([pil_to_tensor(p).squeeze(0) for p in ctx_pils],
    dim=0)
    x_tgt = pil_to_tensor(tgt_pil)

```

```

# Encode prev_last; zeros sentinel when the frame is unavailable.
if prev_last_pil is not None:
    x_prev_last = pil_to_tensor(prev_last_pil)      # [1, H, W]
    has_prev_last = torch.tensor(True, dtype=torch.bool)
else:
    x_prev_last = torch.zeros(1, IMG_SIZE, IMG_SIZE)
    has_prev_last = torch.tensor(False, dtype=torch.bool)

phase_seq_list = row['phase_seq']
if any(p is None for p in phase_seq_list):
    raise ValueError(f"Missing phase labels in context window for_
↳{row['embryo_id']} ({row['plane']}) at t={row['t_end']}")
phase_seq = torch.tensor(phase_seq_list, dtype=torch.long)

tgt_phase_id = row.get('tgt_phase_id')
if tgt_phase_id is None:
    raise ValueError(f"Missing target phase label for_
↳{row['embryo_id']} ({row['plane']}) at t={row['t_end']}")
tgt_phase = torch.tensor(int(tgt_phase_id), dtype=torch.long)

if USE_TIME_MAP:
    frame_idx = min(int(row['tgt_frame']), int(MAX_FRAME_INDEX))
    frame_idx = torch.tensor(frame_idx, dtype=torch.long)
    return x_ctx, x_tgt, frame_idx, phase_seq, tgt_phase, x_prev_last,
↳has_prev_last
return x_ctx, x_tgt, phase_seq, tgt_phase, x_prev_last, has_prev_last

```

1.10 9. DataLoaders

We build one standard train/val/test DataLoader set for next-frame prediction.

Training still uses the one-step diffusion objective. Rollout-aware evaluation is handled later from fixed validation rows, not through a separate training loader.

```

[ ]: # Build datasets/loaders once; pairs are sampled on the fly
train_dataset = PairDataset(index_for_pairs, train_ids, SPLIT_SEED, 'train',
↳planes=FOCAL_PLANES, build_transition_index=True)
val_dataset = PairDataset(index_for_pairs, val_ids, SPLIT_SEED, 'val',
↳planes=FOCAL_PLANES, build_transition_index=True)
test_dataset = PairDataset(index_for_pairs, test_ids, SPLIT_SEED, 'test',
↳planes=FOCAL_PLANES, build_transition_index=True)

# Transition-aware weighted sampling for train loader
train_transition_keys = {
    (r['embryo_id'], r['plane'], r['t_end'])
    for rows in train_dataset.transition_windows_by_plane.values()
    for r in rows
}

```

```

}
train_is_transition = np.array([
    1.0 if (r['embryo_id'], r['plane'], r['t_end']) in train_transition_keys
    ↪else 0.0
    for r in train_dataset.windows
], dtype=np.float32)
train_trans_count = int(train_is_transition.sum())
train_non_count = int(len(train_is_transition) - train_trans_count)
train_base_transition_frac = (train_trans_count / max(1,
    ↪len(train_is_transition)))

train_sampler = None
if train_trans_count > 0 and train_non_count > 0 and 0.0 <
    ↪TRAIN_TARGET_TRANSITION_FRAC < 1.0:
    w_trans = TRAIN_TARGET_TRANSITION_FRAC / train_trans_count
    w_non = (1.0 - TRAIN_TARGET_TRANSITION_FRAC) / train_non_count
    train_weights = np.where(train_is_transition > 0.5, w_trans, w_non).
    ↪astype(np.float64)
    train_sampler = WeightedRandomSampler(
        weights=torch.from_numpy(train_weights),
        num_samples=len(train_dataset),
        replacement=True,
    )

loader_kwargs = {
    'batch_size': BATCH_SIZE,
    'num_workers': NUM_WORKERS,
    'pin_memory': True,
}
if NUM_WORKERS > 0:
    if TRAIN_PERSISTENT_WORKERS:
        loader_kwargs['persistent_workers'] = True
    if DATALOADER_PREFETCH_FACTOR is not None and
    ↪int(DATALOADER_PREFETCH_FACTOR) > 0:
        loader_kwargs['prefetch_factor'] = int(DATALOADER_PREFETCH_FACTOR)

train_loader = DataLoader(
    train_dataset,
    **loader_kwargs,
    shuffle=(train_sampler is None),
    sampler=train_sampler,
    drop_last=TRAIN_DROP_LAST,
)
val_loader = DataLoader(val_dataset, shuffle=False, **loader_kwargs)
test_loader = DataLoader(test_dataset, shuffle=False, **loader_kwargs)

print(

```

```

        f"Train transition windows: {train_trans_count} / {len(train_dataset)} "
        f"({100.0 * train_base_transition_frac:.2f}%) | target sampler_
↪fraction={100.0 * TRAIN_TARGET_TRANSITION_FRAC:.2f}%"
    )
    print(f"Train pairs: {len(train_dataset)} | Val pairs: {len(val_dataset)} |_
↪Test pairs: {len(test_dataset)}")
    print(f"Train loader batches: {len(train_loader)} | Val loader batches:_
↪{len(val_loader)} | Test loader batches: {len(test_loader)}")

```

```

[PairDataset:train] sequences=565 | frames=231119 | frames/seq avg=409.1
(min=106, max=647) | valid windows/seq avg=405.1 (min=103, max=644) | transition
windows/seq avg=10.8 (min=3, max=14) | skipped short=0, skipped no-target=0
[PairDataset:val] sequences=69 | frames=29963 | frames/seq avg=434.2 (min=145,
max=639) | valid windows/seq avg=430.2 (min=142, max=636) | transition
windows/seq avg=11.6 (min=5, max=14) | skipped short=0, skipped no-target=0
[PairDataset:test] sequences=69 | frames=28593 | frames/seq avg=414.4 (min=179,
max=563) | valid windows/seq avg=410.5 (min=176, max=558) | transition
windows/seq avg=10.5 (min=5, max=14) | skipped short=0, skipped no-target=0
Train transition windows: 6097 / 228865 (2.66%) | target sampler fraction=20.00%
Train pairs: 228865 | Val pairs: 29685 | Test pairs: 28323
Train loader batches: 7152 | Val loader batches: 928 | Test loader batches: 886

```

1.11 10. Utilities, Metrics & Sampling Helpers

This section initialises the model, optimiser, and all helper functions used by the training loop and inference utilities.

Sub-sections: - **10.1** Sampling helpers — model init, DDIM sampling. - **10.2** Perceptual metrics — SSIM & LPIPS. - **10.3** Phase classifier architecture — rollout guidance only. - **10.4** Classifier helpers & rollout policy.

```

[ ]: def _unwrap_diffusion_model(model: nn.Module) -> nn.Module:
        return model._orig_mod if hasattr(model, '_orig_mod') else model

base_diffusion = DiffusionCond(context_k=MODEL_CONTEXT_K,_
↪use_time_map=USE_TIME_MAP).to(device)
ema_model = None
if USE_EMA:
    ema_model = copy.deepcopy(base_diffusion).to(device)
    ema_model.eval()
    for param in ema_model.parameters():
        param.requires_grad_(False)

diffusion = base_diffusion
if USE_GRAD_CHECKPOINTING:
    diffusion.unet.enable_gradient_checkpointing()

```

```

# Speed: keep channels_last on input tensors (set in train/val loops).
# Do not call module.to(memory_format=...) on this model; it contains non-4D
↳params.
torch.backends.cudnn.benchmark = True

if ENABLE_TORCH_COMPILE and torch.__version__.startswith('2'):
    try:
        diffusion = torch.compile(diffusion, mode='reduce-overhead')
        print('torch.compile enabled (mode=reduce-overhead)')
    except Exception as e:
        print(f'torch.compile unavailable, proceeding without compile: {e}')

try:
    optimizer = torch.optim.AdamW(diffusion.parameters(), lr=LR, betas=(0.9, 0.
↳999), fused=True)
    print('AdamW fused=True')
except TypeError:
    optimizer = torch.optim.AdamW(diffusion.parameters(), lr=LR, betas=(0.9, 0.
↳999))
    print('AdamW fused not supported; using standard AdamW')

# Model summary & parameter counts
total_params = sum(p.numel() for p in diffusion.parameters())
trainable_params = sum(p.numel() for p in diffusion.parameters() if p.
↳requires_grad)
print(diffusion)
print(f"Total parameters: {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
if ema_model is not None:
    print(f"EMA enabled with decay={EMA_DECAY}")

```

AdamW fused=True

```

DiffusionCond(
  (frame_emb): Embedding(801, 8)
  (frame_proj): Linear(in_features=8, out_features=1, bias=True)
  (ctx_encoder): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (1): GELU(approximate='none')
    (2): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (3): GELU(approximate='none')
    (4): Conv2d(64, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (5): GELU(approximate='none')
  )
  (ctx_to_img): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): GELU(approximate='none')
    (2): Conv2d(32, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  )
)

```

```

)
(ctx_pos_emb): Embedding(3, 96)
(ctx_time_emb): Embedding(801, 96)
(unet): UNet2DConditionModel(
  (conv_in): Conv2d(10, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (time_proj): Timesteps()
  (time_embedding): TimestepEmbedding(
    (linear_1): Linear(in_features=32, out_features=128, bias=True)
    (act): SiLU()
    (linear_2): Linear(in_features=128, out_features=128, bias=True)
  )
  (down_blocks): ModuleList(
    (0): DownBlock2D(
      (resnets): ModuleList(
        (0): ResnetBlock2D(
          (norm1): GroupNorm(8, 32, eps=1e-05, affine=True)
          (conv1): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
          (time_emb_proj): Linear(in_features=128, out_features=32, bias=True)
          (norm2): GroupNorm(8, 32, eps=1e-05, affine=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
          (nonlinearity): SiLU()
        )
      )
      (downsamplers): ModuleList(
        (0): Downsample2D(
          (conv): Conv2d(32, 32, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
        )
      )
    )
    (1): DownBlock2D(
      (resnets): ModuleList(
        (0): ResnetBlock2D(
          (norm1): GroupNorm(8, 32, eps=1e-05, affine=True)
          (conv1): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
          (time_emb_proj): Linear(in_features=128, out_features=64, bias=True)
          (norm2): GroupNorm(8, 64, eps=1e-05, affine=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
          (nonlinearity): SiLU()
          (conv_shortcut): Conv2d(32, 64, kernel_size=(1, 1), stride=(1, 1))
        )
      )
    )
  )
)

```

```

        (downsamplers): ModuleList(
          (0): Downsample2D(
            (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
          )
        )
      (2): DownBlock2D(
        (resnets): ModuleList(
          (0): ResnetBlock2D(
            (norm1): GroupNorm(8, 64, eps=1e-05, affine=True)
            (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
            (time_emb_proj): Linear(in_features=128, out_features=128,
bias=True)
            (norm2): GroupNorm(8, 128, eps=1e-05, affine=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
            (nonlinearity): SiLU()
            (conv_shortcut): Conv2d(64, 128, kernel_size=(1, 1), stride=(1, 1))
          )
        )
        (downsamplers): ModuleList(
          (0): Downsample2D(
            (conv): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
          )
        )
      (3): DownBlock2D(
        (resnets): ModuleList(
          (0): ResnetBlock2D(
            (norm1): GroupNorm(8, 128, eps=1e-05, affine=True)
            (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
            (time_emb_proj): Linear(in_features=128, out_features=256,
bias=True)
            (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
            (dropout): Dropout(p=0.0, inplace=False)
            (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
            (nonlinearity): SiLU()
            (conv_shortcut): Conv2d(128, 256, kernel_size=(1, 1), stride=(1, 1))
          )
        )
        (downsamplers): ModuleList(
          (0): Downsample2D(

```

```

        (conv): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1))
    )
)
(4): CrossAttnDownBlock2D(
  (attentions): ModuleList(
    (0): Transformer2DModel(
      (norm): GroupNorm(8, 256, eps=1e-06, affine=True)
      (proj_in): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
      (transformer_blocks): ModuleList(
        (0): BasicTransformerBlock(
          (norm1): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
          (attn1): Attention(
            (to_q): Linear(in_features=256, out_features=256, bias=False)
            (to_k): Linear(in_features=256, out_features=256, bias=False)
            (to_v): Linear(in_features=256, out_features=256, bias=False)
            (to_out): ModuleList(
              (0): Linear(in_features=256, out_features=256, bias=True)
              (1): Dropout(p=0.0, inplace=False)
            )
          )
        )
        (norm2): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
        (attn2): Attention(
          (to_q): Linear(in_features=256, out_features=256, bias=False)
          (to_k): Linear(in_features=96, out_features=256, bias=False)
          (to_v): Linear(in_features=96, out_features=256, bias=False)
          (to_out): ModuleList(
            (0): Linear(in_features=256, out_features=256, bias=True)
            (1): Dropout(p=0.0, inplace=False)
          )
        )
        (norm3): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
        (ff): FeedForward(
          (net): ModuleList(
            (0): GEGLU(
              (proj): Linear(in_features=256, out_features=2048,
bias=True)
            )
            (1): Dropout(p=0.0, inplace=False)
            (2): Linear(in_features=1024, out_features=256, bias=True)
          )
        )
      )
    )
  )
  (proj_out): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
)
)

```

```

(resnets): ModuleList(
  (0): ResnetBlock2D(
    (norm1): GroupNorm(8, 256, eps=1e-05, affine=True)
    (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (time_emb_proj): Linear(in_features=128, out_features=256,
bias=True)
    (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
    (dropout): Dropout(p=0.0, inplace=False)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (nonlinearity): SiLU()
  )
)
)
)
(up_blocks): ModuleList(
  (0): CrossAttnUpBlock2D(
    (attentions): ModuleList(
      (0-1): 2 x Transformer2DModel(
        (norm): GroupNorm(8, 256, eps=1e-06, affine=True)
        (proj_in): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
        (transformer_blocks): ModuleList(
          (0): BasicTransformerBlock(
            (norm1): LayerNorm((256,)), eps=1e-05, elementwise_affine=True)
            (attn1): Attention(
              (to_q): Linear(in_features=256, out_features=256, bias=False)
              (to_k): Linear(in_features=256, out_features=256, bias=False)
              (to_v): Linear(in_features=256, out_features=256, bias=False)
              (to_out): ModuleList(
                (0): Linear(in_features=256, out_features=256, bias=True)
                (1): Dropout(p=0.0, inplace=False)
              )
            )
          )
          (norm2): LayerNorm((256,)), eps=1e-05, elementwise_affine=True)
          (attn2): Attention(
            (to_q): Linear(in_features=256, out_features=256, bias=False)
            (to_k): Linear(in_features=96, out_features=256, bias=False)
            (to_v): Linear(in_features=96, out_features=256, bias=False)
            (to_out): ModuleList(
              (0): Linear(in_features=256, out_features=256, bias=True)
              (1): Dropout(p=0.0, inplace=False)
            )
          )
          (norm3): LayerNorm((256,)), eps=1e-05, elementwise_affine=True)
          (ff): FeedForward(
            (net): ModuleList(
              (0): GEGLU(

```

```

        (proj): Linear(in_features=256, out_features=2048,
bias=True)
    )
    (1): Dropout(p=0.0, inplace=False)
    (2): Linear(in_features=1024, out_features=256, bias=True)
    )
    )
    )
    (proj_out): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
    )
    )
    (resnets): ModuleList(
      (0-1): 2 x ResnetBlock2D(
        (norm1): GroupNorm(8, 512, eps=1e-05, affine=True)
        (conv1): Conv2d(512, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
        (time_emb_proj): Linear(in_features=128, out_features=256,
bias=True)
        (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
        (nonlinearity): SiLU()
        (conv_shortcut): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1))
      )
    )
    (upsamplers): ModuleList(
      (0): Upsample2D(
        (conv): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      )
    )
    )
    (1): UpBlock2D(
      (resnets): ModuleList(
        (0): ResnetBlock2D(
          (norm1): GroupNorm(8, 512, eps=1e-05, affine=True)
          (conv1): Conv2d(512, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
          (time_emb_proj): Linear(in_features=128, out_features=256,
bias=True)
          (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
          (dropout): Dropout(p=0.0, inplace=False)
          (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
          (nonlinearity): SiLU()
          (conv_shortcut): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1))
        )
      )
    )
  )
)

```

```

    )
    (1): ResnetBlock2D(
      (norm1): GroupNorm(8, 384, eps=1e-05, affine=True)
      (conv1): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (time_emb_proj): Linear(in_features=128, out_features=256,
bias=True)
      (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
      (dropout): Dropout(p=0.0, inplace=False)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (nonlinearity): SiLU()
      (conv_shortcut): Conv2d(384, 256, kernel_size=(1, 1), stride=(1, 1))
    )
  )
  (upsamplers): ModuleList(
    (0): Upsample2D(
      (conv): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    )
  )
)
(2): UpBlock2D(
  (resnets): ModuleList(
    (0): ResnetBlock2D(
      (norm1): GroupNorm(8, 384, eps=1e-05, affine=True)
      (conv1): Conv2d(384, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (time_emb_proj): Linear(in_features=128, out_features=128,
bias=True)
      (norm2): GroupNorm(8, 128, eps=1e-05, affine=True)
      (dropout): Dropout(p=0.0, inplace=False)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (nonlinearity): SiLU()
      (conv_shortcut): Conv2d(384, 128, kernel_size=(1, 1), stride=(1, 1))
    )
    (1): ResnetBlock2D(
      (norm1): GroupNorm(8, 192, eps=1e-05, affine=True)
      (conv1): Conv2d(192, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (time_emb_proj): Linear(in_features=128, out_features=128,
bias=True)
      (norm2): GroupNorm(8, 128, eps=1e-05, affine=True)
      (dropout): Dropout(p=0.0, inplace=False)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (nonlinearity): SiLU()

```

```

        (conv_shortcut): Conv2d(192, 128, kernel_size=(1, 1), stride=(1, 1))
    )
)
(upsamplers): ModuleList(
  (0): Upsample2D(
    (conv): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  )
)
)
(3): UpBlock2D(
  (resnets): ModuleList(
    (0): ResnetBlock2D(
      (norm1): GroupNorm(8, 192, eps=1e-05, affine=True)
      (conv1): Conv2d(192, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (time_emb_proj): Linear(in_features=128, out_features=64, bias=True)
      (norm2): GroupNorm(8, 64, eps=1e-05, affine=True)
      (dropout): Dropout(p=0.0, inplace=False)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (nonlinearity): SiLU()
      (conv_shortcut): Conv2d(192, 64, kernel_size=(1, 1), stride=(1, 1))
    )
    (1): ResnetBlock2D(
      (norm1): GroupNorm(8, 96, eps=1e-05, affine=True)
      (conv1): Conv2d(96, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (time_emb_proj): Linear(in_features=128, out_features=64, bias=True)
      (norm2): GroupNorm(8, 64, eps=1e-05, affine=True)
      (dropout): Dropout(p=0.0, inplace=False)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
      (nonlinearity): SiLU()
      (conv_shortcut): Conv2d(96, 64, kernel_size=(1, 1), stride=(1, 1))
    )
  )
)
(upsamplers): ModuleList(
  (0): Upsample2D(
    (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  )
)
)
(4): UpBlock2D(
  (resnets): ModuleList(
    (0): ResnetBlock2D(
      (norm1): GroupNorm(8, 96, eps=1e-05, affine=True)

```

```

        (conv1): Conv2d(96, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
        (time_emb_proj): Linear(in_features=128, out_features=32, bias=True)
        (norm2): GroupNorm(8, 32, eps=1e-05, affine=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
        (nonlinearity): SiLU()
        (conv_shortcut): Conv2d(96, 32, kernel_size=(1, 1), stride=(1, 1))
    )
    (1): ResnetBlock2D(
        (norm1): GroupNorm(8, 64, eps=1e-05, affine=True)
        (conv1): Conv2d(64, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
        (time_emb_proj): Linear(in_features=128, out_features=32, bias=True)
        (norm2): GroupNorm(8, 32, eps=1e-05, affine=True)
        (dropout): Dropout(p=0.0, inplace=False)
        (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
        (nonlinearity): SiLU()
        (conv_shortcut): Conv2d(64, 32, kernel_size=(1, 1), stride=(1, 1))
    )
)
)
)
)
(mid_block): UNetMidBlock2DCrossAttn(
  (attentions): ModuleList(
    (0): Transformer2DModel(
      (norm): GroupNorm(8, 256, eps=1e-06, affine=True)
      (proj_in): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
      (transformer_blocks): ModuleList(
        (0): BasicTransformerBlock(
          (norm1): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
          (attn1): Attention(
            (to_q): Linear(in_features=256, out_features=256, bias=False)
            (to_k): Linear(in_features=256, out_features=256, bias=False)
            (to_v): Linear(in_features=256, out_features=256, bias=False)
            (to_out): ModuleList(
              (0): Linear(in_features=256, out_features=256, bias=True)
              (1): Dropout(p=0.0, inplace=False)
            )
          )
        )
      )
      (norm2): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
      (attn2): Attention(
        (to_q): Linear(in_features=256, out_features=256, bias=False)
        (to_k): Linear(in_features=96, out_features=256, bias=False)
        (to_v): Linear(in_features=96, out_features=256, bias=False)
        (to_out): ModuleList(

```

```

        (0): Linear(in_features=256, out_features=256, bias=True)
        (1): Dropout(p=0.0, inplace=False)
    )
)
(norm3): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
(ff): FeedForward(
  (net): ModuleList(
    (0): GEGLU(
      (proj): Linear(in_features=256, out_features=2048,
bias=True)
    )
    (1): Dropout(p=0.0, inplace=False)
    (2): Linear(in_features=1024, out_features=256, bias=True)
  )
)
)
)
(proj_out): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
)
)
(resnets): ModuleList(
  (0-1): 2 x ResnetBlock2D(
    (norm1): GroupNorm(8, 256, eps=1e-05, affine=True)
    (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (time_emb_proj): Linear(in_features=128, out_features=256, bias=True)
    (norm2): GroupNorm(8, 256, eps=1e-05, affine=True)
    (dropout): Dropout(p=0.0, inplace=False)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (nonlinearity): SiLU()
  )
)
)
(conv_norm_out): GroupNorm(8, 32, eps=1e-05, affine=True)
(conv_act): SiLU()
(conv_out): Conv2d(32, 1, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
)
)
Total parameters: 21,586,682
Trainable parameters: 21,586,682
EMA enabled with decay=0.9995

```

1.11.1 10.1 Sampling & Guidance Helpers

Low-level helpers that wrap the UNet forward pass and the DDIM scheduler step.

- `_eps_from_model`: single forward pass that returns noise prediction (or velocity when

USE_V_PREDICTION=True).

- `ddim_sample_from_model`: full DDIM loop; when `USE_RESIDUAL_PREDICTION=True` it decodes the model's delta output back to an absolute frame.
- `sample_from_model`: thin wrapper with default `DDIM_STEPS / DDIM_ETA` for use in training and validation loops.

```
[ ]: def _repeat_channels(x, target_channels=3):
    """Repeat single-channel tensors along channel dim."""
    if x.shape[1] == target_channels:
        return x
    return x.repeat(1, target_channels, 1, 1)

def _target_channels() -> int:
    """Current diffusion output channel count."""
    return int(getattr(diffusion, 'output_channels', MODEL_OUT_CHANNELS))

def _eps_from_model(
    x_noisy: torch.Tensor,
    x_ctx: torch.Tensor,
    t_tensor: torch.Tensor,
    time_map: torch.Tensor | None = None,
    frame_idx: torch.Tensor | None = None,
):
    """Query UNet to predict noise epsilon (or velocity v when
    USE_V_PREDICTION=True)."""
    return diffusion.forward(
        x_noisy,
        t_tensor,
        x_ctx,
        time_map=time_map,
        frame_idx=frame_idx,
    )

def _ensure_context_batch(x_ctx: torch.Tensor | None, shape):
    """Ensure context tensor is [B, k, H, W]."""
    B, H, W = shape[0], shape[2], shape[3]
    expected_k = int(getattr(diffusion, 'context_k', MODEL_CONTEXT_K))
    if x_ctx is None:
        raise ValueError("Context tensor x_ctx is required for conditioned_
    generation")
    x_ctx = x_ctx.to(device)
    if x_ctx.dim() == 3:
        x_ctx = x_ctx.unsqueeze(0)
    if x_ctx.shape[1] != expected_k:
```

```

        raise ValueError(f"Expected context with k={expected_k} channels, got_{x_ctx.shape[1]}")
    return x_ctx

def _ensure_time_map(frame_idx: torch.Tensor | int | None, shape):
    """Build time map if enabled; otherwise return None."""
    if not USE_TIME_MAP:
        return None
    if frame_idx is None:
        raise ValueError("frame_idx is required when USE_TIME_MAP=True")
    if not torch.is_tensor(frame_idx):
        frame_idx = torch.tensor(frame_idx, device=device, dtype=torch.long)
    if frame_idx.dim() == 0:
        frame_idx = frame_idx.view(1)
    frame_idx = frame_idx.clamp(0, MAX_FRAME_INDEX).long()
    return diffusion.build_time_map(frame_idx, shape[2], shape[3])

def ddim_sample_from_model(
    shape,
    x_ctx,
    frame_idx=None,
    steps=DDIM_STEPS,
    eta=DDIM_ETA,
    x_init=None,
):
    """DDIM sampling. Always returns the absolute predicted frame in [-1, 1].

    When USE_RESIDUAL_PREDICTION=True the network denoises a normalized delta
    (x_tgt - x_last) / 2. The last context frame is added back here so the
    caller always receives an absolute-frame tensor, regardless of training_
    ↪mode.
    """
    B = shape[0]
    if x_init is None:
        x = torch.randn((B, shape[1], shape[2], shape[3]), device=device)
    else:
        x = x_init
    x_ctx = _ensure_context_batch(x_ctx, shape)
    x_last = x_ctx[:, -1:] # [B, 1, H, W] - save before the loop for residual_
    ↪reconstruction
    time_map = _ensure_time_map(frame_idx, shape)
    ddim_scheduler.set_timesteps(steps, device=device)
    for t in ddim_scheduler.timesteps:
        t_tensor = torch.full((B,), int(t), device=device, dtype=torch.long)
        with torch.no_grad():

```

```

        eps = _eps_from_model(
            x,
            x_ctx,
            t_tensor,
            time_map=time_map,
            frame_idx=frame_idx,
        )
    x = ddim_scheduler.step(eps, t, x, eta=eta).prev_sample

    # Residual prediction: model output is the normalized delta; reconstruct
    ↪ absolute frame.
    if USE_RESIDUAL_PREDICTION:
        x = (x * 2.0 + x_last).clamp(-1, 1)

    return x.detach()

def sample_from_model(shape, x_ctx, frame_idx=None, x_init=None):
    """Generate samples using DDIM."""
    return ddim_sample_from_model(
        shape,
        x_ctx,
        frame_idx=frame_idx,
        steps=DDIM_STEPS,
        eta=DDIM_ETA,
        x_init=x_init,
    )

```

```

[ ]: # Save a grayscale tensor to disk as a PNG/JPEG.
def save_tensor_image(tensor: torch.Tensor, path):
    """Save a single-plane tensor ([1,H,W] or [H,W]) as a grayscale image."""
    if tensor.dim() == 3 and tensor.shape[0] == 1:
        tensor = tensor[0]
    if tensor.dim() != 2:
        raise ValueError(f"save_tensor_image expects [1,H,W] or [H,W], got
        ↪ {tuple(tensor.shape)}")
    arr = (denorm(tensor).clamp(0, 1).cpu().numpy() * 255).astype(np.uint8)
    Image.fromarray(arr).save(path)

# LPIPS is only loaded when COMPUTE_VAL_METRICS=True to avoid consuming GPU
↪ memory
# during pure training runs where perceptual metrics are not needed.
lpips_metric = None
if COMPUTE_VAL_METRICS:
    lpips_metric = LPIPS(net='alex').to(device)
    lpips_metric.eval()

```

```

for p in lpips_metric.parameters():
    p.requires_grad_(False)

```

1.11.2 10.2 Perceptual Metrics (SSIM & LPIPS)

SSIM and LPIPS are computed on fixed validation subsets controlled by COMPUTE_VAL_METRICS and VAL_METRIC_SAMPLES. LPIPS requires the metric model to be loaded at setup time (see cell above).

```

[ ]: # Gaussian kernel cache - reused across SSIM calls for the same window/sigma/
    ↪device.
_gaussian_kernel_cache: dict = {}

def _gaussian_kernel(window_size: int, sigma: float, device: torch.device,
    ↪dtype: torch.dtype) -> torch.Tensor:
    """Build and cache a 2D Gaussian window for SSIM convolutions."""
    key = (window_size, sigma, device.type, device.index, dtype)
    kernel = _gaussian_kernel_cache.get(key)
    if kernel is None:
        coords = torch.arange(window_size, device=device, dtype=dtype) -
    ↪window_size // 2
        gauss = torch.exp(-(coords ** 2) / (2 * sigma ** 2))
        gauss /= gauss.sum()
        kernel_1d = gauss.unsqueeze(0)
        kernel_2d = kernel_1d.t() @ kernel_1d
        kernel_2d /= kernel_2d.sum()
        kernel = kernel_2d.unsqueeze(0).unsqueeze(0)
        _gaussian_kernel_cache[key] = kernel
    return kernel

def compute_ssim_metric(pred: torch.Tensor, target: torch.Tensor, window_size:
    ↪int = 11, sigma: float = 1.5) -> torch.Tensor:
    """Compute SSIM between two tensors in [-1, 1] space (internally mapped to
    ↪[0, 1])."""
    pred_01 = ((pred.clamp(-1, 1) + 1) / 2).float()
    target_01 = ((target.clamp(-1, 1) + 1) / 2).float()
    device, dtype, channels = pred_01.device, pred_01.dtype, pred_01.size(1)

    kernel = _gaussian_kernel(window_size, sigma, device, dtype).
    ↪expand(channels, 1, window_size, window_size)
    pad = window_size // 2

    mu_p = F.conv2d(pred_01, kernel, padding=pad, groups=channels)
    mu_t = F.conv2d(target_01, kernel, padding=pad, groups=channels)

```

```

        sigma_p = torch.clamp_min(F.conv2d(pred_01 * pred_01, kernel, padding=pad,
↪groups=channels) - mu_p.pow(2), 0.0)
        sigma_t = torch.clamp_min(F.conv2d(target_01 * target_01, kernel,
↪padding=pad, groups=channels) - mu_t.pow(2), 0.0)
        sigma_pt = F.conv2d(pred_01 * target_01, kernel, padding=pad,
↪groups=channels) - mu_p * mu_t

        c1, c2 = (0.01) ** 2, (0.03) ** 2
        numerator = (2 * mu_p * mu_t + c1) * (2 * sigma_pt + c2)
        denominator = (mu_p.pow(2) + mu_t.pow(2) + c1) * (sigma_p + sigma_t + c2)
        return (numerator / (denominator + 1e-6)).mean()

def compute_lpips_metric(pred: torch.Tensor, target: torch.Tensor) -> torch.
↪Tensor:
    """Compute LPIPS on grayscale inputs by repeating to 3 channels as required,
↪by the network."""
    if lpips_metric is None:
        raise RuntimeError('LPIPS metric not initialised - set
↪COMPUTE_VAL_METRICS=True.')
    pred = pred.clamp(-1, 1)
    target = target.clamp(-1, 1)
    with torch.no_grad():
        if pred.shape[1] == 1:
            return lpips_metric(_repeat_channels(pred),
↪_repeat_channels(target), normalize=False).mean()
            # Multi-channel: average per-channel LPIPS
            vals = [
                lpips_metric(_repeat_channels(pred[:, ci:ci+1]),
↪_repeat_channels(target[:, ci:ci+1]), normalize=False).mean()
                for ci in range(pred.shape[1])
            ]
            return torch.stack(vals).mean()

```

1.11.3 10.3 Phase Classifier Architecture

The rollout classifier reuses the same EfficientNetV2 architecture as the external phase-classifier notebook, extended with a sinusoidal timestep embedding. It is **not** part of the diffusion training objective — it is only used at sampling time to drive the monotonic phase-update policy during qualitative rollouts.

```

[ ]: # EfficientNetV2 adapted for single-channel (grayscale) input with optional
# diffusion-timestep conditioning. Used only for rollout phase guidance;
# not part of the diffusion training objective.

```

```

class SinusoidalTimeEmbedding(nn.Module):
    """Sinusoidal positional embedding for diffusion timesteps."""

```

```

    def __init__(self, dim: int, max_timesteps: int = 1000, max_period: float = 10000.0):
        super().__init__()
        self.dim = dim
        self.max_timesteps = max_timesteps
        self.max_period = max_period

    def forward(self, t: torch.Tensor) -> torch.Tensor:
        half = self.dim // 2
        t = t.float() / max(1, self.max_timesteps - 1)
        freqs = torch.exp(
            -math.log(self.max_period) * torch.arange(0, half, device=t.device).
            float() / max(1, half)
        )
        args = t[:, None] * freqs[None] * 2 * math.pi
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=1)
        if self.dim % 2 == 1:
            emb = torch.cat([emb, torch.zeros_like(emb[:, :1])], dim=1)
        return emb


class EfficientNetV2MGray(nn.Module):
    """EfficientNetV2 adapted for grayscale input with diffusion timestep
    conditioning."""

    def __init__(self, n_classes: int, time_dim: int = 32, max_timesteps: int = 1000,
                 backbone: str = "efficientnet_v2_m", dropout: float = 0.5):
        super().__init__()
        if backbone == "efficientnet_v2_s":
            self.base = efficientnet_v2_s(weights=None)
        else:
            self.base = efficientnet_v2_m(weights=None)

        # Replace the first convolution to accept 1-channel (grayscale) input.
        old_conv = self.base.features[0][0]
        new_conv = nn.Conv2d(1, old_conv.out_channels, kernel_size=3, stride=2,
                             padding=1, bias=False)
        self.base.features[0][0] = new_conv

        in_feats = self.base.classifier[1].in_features
        self.base.classifier = nn.Identity()

        self.time_embed = SinusoidalTimeEmbedding(time_dim,
            max_timesteps=max_timesteps)

```

```

        self.time_mlp = nn.Sequential(
            nn.Linear(time_dim, time_dim),
            nn.SiLU(),
            nn.Linear(time_dim, time_dim),
        )
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout),
            nn.Linear(in_feats + time_dim, n_classes),
        )

    def forward(self, x: torch.Tensor, t: torch.Tensor | None = None) -> torch.
↳Tensor:
        feats = self.base.features(x)
        feats = self.base.avgpool(feats)
        feats = torch.flatten(feats, 1)
        if t is None:
            t = torch.zeros(x.size(0), device=x.device, dtype=torch.long)
        t_emb = self.time_mlp(self.time_embed(t.long()))
        feats = torch.cat([feats, t_emb], dim=1)
        return self.classifier(feats)

```

1.11.4 10.4 Classifier Helpers and Rollout Phase Policy

Helpers for loading/unloading the classifier, running predictions, and implementing the PHASE_POLICY_MODE logic used during qualitative rollout generation. The classifier is unloaded after each epoch's sample generation to reclaim VRAM for the next training epoch.

```

[ ]: # Globals for the lazy-loaded rollout classifier.
# Use load_phase_classifier() / unload_phase_classifier() to manage lifetime.
phase_classifier = None
phase_classifier_meta = None

def unload_phase_classifier():
    """Release the rollout-only classifier to free VRAM after qualitative_
↳sampling."""
    global phase_classifier, phase_classifier_meta
    if phase_classifier is None and phase_classifier_meta is None:
        return
    phase_classifier = None
    phase_classifier_meta = None
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

def _resolve_classifier_ckpt(path) -> Path | None:

```

```

    """Return the checkpoint path if it exists, else fall back to the most
    ↪recent in the model dir."""
    p = Path(path)
    if p.exists():
        return p
    ckpts = sorted(CLASSIFIER_MODEL_DIR.glob('phase_classifier*.pth'),
    ↪key=lambda q: q.stat().st_mtime, reverse=True)
    return ckpts[0] if ckpts else None

def _classifier_label_to_phase_id(label: str) -> int | None:
    """Normalize a classifier label string to a notebook phase ID."""
    p = normalize_phase_label(label)
    return PHASE_TO_ID.get(p)

def load_phase_classifier(ckpt_path=None):
    """Lazy-load the rollout phase classifier only when a qualitative helper
    ↪needs it."""
    global phase_classifier, phase_classifier_meta
    resolved = _resolve_classifier_ckpt(ckpt_path or CLASSIFIER_CKPT_PATH)
    if resolved is None or not resolved.exists():
        print('Phase classifier checkpoint not found. Rollout policy disabled.')
        phase_classifier = None
        phase_classifier_meta = None
        return None

    # Skip reload if the same checkpoint is already loaded.
    if phase_classifier is not None and phase_classifier_meta is not None:
        if Path(phase_classifier_meta.get('path')) == resolved:
            return phase_classifier
        unload_phase_classifier()

    payload = torch.load(resolved, map_location='cpu', weights_only=False)
    state_dict = payload.get('model_state_dict', payload.get('model', payload))
    cfg = payload.get('config', {})
    clf_labels = cfg.get('phases', [
        ↪
    ↪'tPB2', 'tPNa', 'tPNf', 't2', 't3', 't4', 't5', 't6', 't7', 't8', 't9+', 'tM', 'tSB', 'tB', 'tEB'
    ])
    clf_alias = cfg.get('phase_alias', {'tHB': 'tEB'})
    time_dim = int(cfg.get('time_emb_dim', 32))
    max_timesteps = int(cfg.get('timesteps', 1000))
    backbone = str(cfg.get('backbone', 'efficientnet_v2_m'))
    dropout = float(cfg.get('dropout', 0.5))

    model = EfficientNetV2MGray(

```

```

        n_classes=len(clf_labels),
        time_dim=time_dim,
        max_timesteps=max_timesteps,
        backbone=backbone,
        dropout=dropout,
    ).to(device)
    model.load_state_dict(state_dict, strict=True)
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)

    phase_classifier = model
    phase_classifier_meta = {'path': resolved, 'labels': list(clf_labels),
↪ 'alias': dict(clf_alias)}
    print(f'Loaded phase classifier: {resolved}')
    return model

@torch.no_grad()
def predict_phase_logits_from_generated(x: torch.Tensor) -> torch.Tensor | None:
    """Run the rollout classifier on generated frames and return raw logits."""
    if phase_classifier is None or phase_classifier_meta is None:
        if not USE_CLASSIFIER_PHASE_POLICY:
            return None
        if load_phase_classifier(CLASSIFIER_CKPT_PATH) is None:
            return None
    x_in = x.clamp(-1, 1).to(device)
    if x_in.dim() != 4:
        raise ValueError(f"Expected [B,C,H,W], got {tuple(x_in.shape)}")
    if x_in.shape[1] > 1:
        x_in = x_in[:, :1] # use primary channel only
    return phase_classifier(x_in)

def _phase_probs_from_classifier_logits(logits: torch.Tensor) -> torch.Tensor:
    """Map classifier output probabilities onto the notebook phase-id space,
↪ merging aliases."""
    probs = torch.softmax(logits, dim=1)
    phase_probs = torch.zeros((logits.shape[0], NUM_PHASES), device=logits.
↪ device, dtype=probs.dtype)
    for idx, raw_label in enumerate(phase_classifier_meta['labels']):
        mapped_label = phase_classifier_meta['alias'].get(str(raw_label),
↪ str(raw_label))
        phase_id = _classifier_label_to_phase_id(mapped_label)
        if phase_id is None:
            phase_id = PHASE_TO_ID['pEB']
        phase_probs[:, int(phase_id)] += probs[:, idx]

```

```

    return phase_probs

@torch.no_grad()
def predict_phase_ids_from_generated(x: torch.Tensor) -> torch.Tensor | None:
    """Predict phase IDs from generated outputs using the rollout classifier."""
    logits = predict_phase_logits_from_generated(x)
    if logits is None:
        return None
    phase_probs = _phase_probs_from_classifier_logits(logits)
    return torch.argmax(phase_probs, dim=1).to(device=x.device, dtype=torch.
↳long)

@torch.no_grad()
def choose_rollout_next_phase(current_phase_seq: torch.Tensor | None, x_gen:
↳torch.Tensor) -> torch.Tensor | None:
    """Apply the configured rollout phase policy to update the temporal phase
↳context.

    Policies (PHASE_POLICY_MODE):
        'argmax' - use the classifier's argmax prediction
↳directly.
        'monotonic_adjacent' - advance by one step if the classifier
↳predicts a later phase.
        'monotonic_adjacent_confidence' - advance only when later-phase
↳probability exceeds threshold.
    """
    next_phase_tensor = current_phase_seq[:, -1:] if current_phase_seq is not
↳None else None
    if not USE_CLASSIFIER_PHASE_POLICY:
        return next_phase_tensor

    logits = predict_phase_logits_from_generated(x_gen)
    if logits is None:
        return next_phase_tensor

    phase_probs = _phase_probs_from_classifier_logits(logits)
    pred_phase = torch.argmax(phase_probs, dim=1)

    if current_phase_seq is None or PHASE_POLICY_MODE == 'argmax':
        return pred_phase.view(-1, 1).to(device=phase_probs.device, dtype=torch.
↳long)

    current_phase = current_phase_seq[:, -1]
    next_adjacent = torch.clamp(current_phase + 1, max=NUM_PHASES - 1)

```

```

    if PHASE_POLICY_MODE == 'monotonic_adjacent':
        chosen = torch.where(pred_phase > current_phase, next_adjacent,
↪current_phase)
        return chosen.view(-1, 1).to(device=phase_probs.device, dtype=torch.
↪long)

    if PHASE_POLICY_MODE == 'monotonic_adjacent_confidence':
        phase_ids = torch.arange(NUM_PHASES, device=phase_probs.device).view(1,
↪NUM_PHASES)
        later_prob = (phase_probs * (phase_ids > current_phase.view(-1, 1))).
↪sum(dim=1)
        advance_mask = (next_adjacent > current_phase) & (later_prob >=
↪float(PHASE_POLICY_ADVANCE_THRESHOLD))
        chosen = torch.where(advance_mask, next_adjacent, current_phase)
        return chosen.view(-1, 1).to(device=phase_probs.device, dtype=torch.
↪long)

    raise ValueError(f'Unknown PHASE_POLICY_MODE: {PHASE_POLICY_MODE}')

@torch.no_grad()
def infer_phase_sequence_from_context_paths(ctx_paths: list[str]) -> torch.
↪Tensor | None:
    """Classify context frames to initialize phase_seq for ad-hoc rollout when
↪ground-truth labels are unavailable."""
    if phase_classifier is None or phase_classifier_meta is None:
        if not USE_CLASSIFIER_PHASE_POLICY:
            return None
        if load_phase_classifier(CLASSIFIER_CKPT_PATH) is None:
            return None
    imgs = [load_image(p) for p in ctx_paths]
    x = torch.stack(imgs, dim=0).to(device)
    phase_ids = predict_phase_ids_from_generated(x)
    if phase_ids is None:
        return None
    return phase_ids.view(1, -1)

```

1.12 12. Training Loop

Each training batch predicts diffusion noise for one next-frame target with temporal context from the same plane stream.

This section now has three quality-oriented additions: - **EMA weights**: the exponential moving average can be used for validation, sample generation, and best-model checkpoints. - **Weighted validation selection**: the main best checkpoint and LR scheduler use `val_loss + VAL_ROLLOUT_LOSS_WEIGHT * rollout_h3_loss` when rollout validation is available. - **Adjacent**

phase-label jitter: training can feed a slightly corrupted copy of the context phase labels to the model while preserving true labels for transition-loss bookkeeping. - **Conservative rollout phase policy:** classifier-driven phase updates can use raw argmax, monotonic-adjacent updates, or a confidence-gated monotonic-adjacent policy. - **Rollout-aware validation:** a fixed validation subset measures free-running H=3 drift separately from the one-step validation pass, so we can inspect both signals.

Training state & scheduler setup

```
[ ]: # Train loop - context-conditioned diffusion model
best_val = None
best_rollout = None
best_one_step_val = None
ckpt_path = BEST_CKPT_PATH

# Early stopping config.
# The cosine LR doesn't decay meaningfully until epoch ~500. Patience must be
↳ large enough
# to survive rollout-estimate noise between genuine new bests. At no_imp=12
↳ with patience=150
# this fires around epoch 173 (LR still ~99.3% of peak) - a last resort, not a
↳ convergence signal.
EARLY_STOPPING_PATIENCE = 150 # raised from 75
epochs_no_improve = 0
global_step = 0

# GradScaler for proper mixed precision training (prevents NaN from underflow/
↳ overflow)
use_amp = USE_AMP
# BF16 has the same dynamic range as FP32: no scaler needed on Ampere+.
_amp_dtype = torch.bfloat16 if (use_amp and torch.cuda.is_bf16_supported())
↳ else torch.float16
scaler = torch.amp.GradScaler('cuda', enabled=use_amp and _amp_dtype == torch.
↳ float16)
metric_history = []

# Learning-rate scheduler setup (epoch-wise warmup + cosine decay)
lr_scheduler = None
if USE_LR_SCHEDULER:
    warmup_epochs = max(0, int(LR_WARMUP_EPOCHS))
    warmup_epochs = min(warmup_epochs, max(0, EPOCHS - 1))
    if warmup_epochs > 0:
        warmup_scheduler = torch.optim.lr_scheduler.LinearLR(
            optimizer,
            start_factor=float(LR_WARMUP_START_FACTOR),
            end_factor=1.0,
            total_iters=warmup_epochs,
        )
```

```

        cosine_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
            optimizer,
            T_max=max(1, EPOCHS - warmup_epochs),
            eta_min=float(LR_MIN),
        )
        lr_scheduler = torch.optim.lr_scheduler.SequentialLR(
            optimizer,
            schedulers=[warmup_scheduler, cosine_scheduler],
            milestones=[warmup_epochs],
        )
    else:
        lr_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
            optimizer,
            T_max=max(1, EPOCHS),
            eta_min=float(LR_MIN),
        )

```

Logging, EMA & checkpoint helpers

```

[ ]: def _log(msg: str):
    print(msg)
    with open(LOG_PATH, 'a', encoding='utf-8') as f:
        f.write(str(msg) + "\n")

def _load_metric_history(path: Path):
    """Load per-epoch metrics from JSON; return None when no history exists yet."""
    ↪ """
    if not path.exists():
        return None
    try:
        with open(path, 'r', encoding='utf-8') as f:
            payload = json.load(f)
    except Exception as exc:
        _log(f"WARNING: failed to load metric history from {path}: {exc}")
        return []
    if isinstance(payload, list):
        return payload
    _log(f"WARNING: metric history at {path} is not a list; ignoring it")
    return []

def _save_metric_history(path: Path, history: list[dict]):
    """Keep the full metric history in JSON instead of growing every checkpoint."""
    ↪ """
    with open(path, 'w', encoding='utf-8') as f:
        json.dump(history, f, indent=2)

```

```

def _log_memory_stats(tag: str):
    if not LOG_MEMORY_STATS:
        return
    parts = []
    if torch.cuda.is_available():
        dev = torch.cuda.current_device()
        alloc = torch.cuda.memory_allocated(dev) / (1024 ** 3)
        reserv = torch.cuda.memory_reserved(dev) / (1024 ** 3)
        peak = torch.cuda.max_memory_allocated(dev) / (1024 ** 3)
        parts.append(f'cuda alloc={alloc:.2f}G reserved={reserv:.2f}G_
↪peak={peak:.2f}G')
    if psutil is not None:
        rss = psutil.Process(os.getpid()).memory_info().rss / (1024 ** 3)
        parts.append(f'host rss={rss:.2f}G')
    if parts:
        _log(f'[Mem] {tag} | ' + ' | '.join(parts))

def _phase_name(phase_id: int | None) -> str:
    if phase_id is None:
        return "-"
    phase_id = int(phase_id)
    if 0 <= phase_id < len(PHASE_LABELS):
        return PHASE_LABELS[phase_id]
    return "-"

@torch.no_grad()
def _update_ema_model(source_model: nn.Module, target_model: nn.Module | None,
↪decay: float):
    if target_model is None:
        return
    source_model = _unwrap_diffusion_model(source_model)
    for ema_param, src_param in zip(target_model.parameters(), source_model.
↪parameters()):
        ema_param.mul_(decay).add_(src_param.detach(), alpha=1.0 - decay)
    for ema_buffer, src_buffer in zip(target_model.buffers(), source_model.
↪buffers()):
        ema_buffer.copy_(src_buffer.detach())

def _state_dict_for_checkpoint(model: nn.Module | None):
    if model is None:
        return None
    return _unwrap_diffusion_model(model).state_dict()

def _eval_diffusion_model() -> nn.Module:

```

```

    if EMA_USE_FOR_EVAL and ema_model is not None:
        return ema_model
    return diffusion

@contextmanager
def _temporary_diffusion_model(model: nn.Module):
    global diffusion
    previous = diffusion
    if model is previous:
        yield
        return
    diffusion = model
    try:
        yield
    finally:
        diffusion = previous

# Resume if requested
start_epoch = 1
if RESUME_TRAINING and LAST_CKPT_PATH.exists():
    ckpt = torch.load(LAST_CKPT_PATH, map_location=device, weights_only=False)
    _unwrap_diffusion_model(diffusion).load_state_dict(ckpt.get('model_online',
↪ckpt['model']))
    if ema_model is not None:
        ema_state = ckpt.get('ema_model')
        if ema_state is not None:
            ema_model.load_state_dict(ema_state)
        else:
            ema_model.load_state_dict(_state_dict_for_checkpoint(diffusion))
    if 'optimizer' in ckpt:
        optimizer.load_state_dict(ckpt['optimizer'])
    if 'scaler' in ckpt and ckpt['scaler'] is not None:
        scaler.load_state_dict(ckpt['scaler'])
    weighted_val_active = bool(COMPUTE_VAL_ROLLOUT_METRICS and
↪VAL_ROLLOUT_LOSS_WEIGHT > 0)
    has_weighted_val_state = 'best_val_selection' in ckpt
    if 'lr_scheduler' in ckpt and lr_scheduler is not None and
↪ckpt['lr_scheduler'] is not None:
        try:
            lr_scheduler.load_state_dict(ckpt['lr_scheduler'])
        except Exception as exc:
            _log(f'Reset LR scheduler state due to scheduler mismatch: {exc}')
    if has_weighted_val_state:
        best_val = ckpt['best_val_selection']
        epochs_no_improve = ckpt.get('epochs_no_improve', 0)

```

```

elif weighted_val_active:
    # Older checkpoints stored raw one-step best_val, which is not
    ↪ comparable to the new weighted objective.
    best_val = None
    epochs_no_improve = 0
    _log('Reset weighted-val best tracker because checkpoint predates
    ↪ val_selection_loss.')
else:
    best_val = ckpt.get('best_val', best_val)
    epochs_no_improve = ckpt.get('epochs_no_improve', 0)
    best_rollout = ckpt.get('best_rollout_loss', best_rollout)
    best_one_step_val = ckpt.get('best_one_step_val', best_one_step_val)
    global_step = int(ckpt.get('global_step', 0))
    persisted_history = _load_metric_history(METRIC_HISTORY_PATH)
    if persisted_history is not None:
        metric_history = persisted_history
    elif isinstance(ckpt.get('metric_history'), list):
        metric_history = ckpt['metric_history']
    start_epoch = int(ckpt.get('epoch', 0)) + 1

    # Restore adaptive perturbation state so a resumed run continues from the
    # accumulated noise_std / prob rather than resetting to the config defaults.
    if PERTURB_ADAPT_ENABLED and 'config' in ckpt:
        _saved_cfg = ckpt['config']
        CONTEXT_PERTURB_NOISE_STD = float(_saved_cfg.
    ↪ get('CONTEXT_PERTURB_NOISE_STD', CONTEXT_PERTURB_NOISE_STD))
        CONTEXT_PERTURB_PROB = float(_saved_cfg.get('CONTEXT_PERTURB_PROB',
    ↪ CONTEXT_PERTURB_PROB))
        _log(f"Restored adaptive perturbation from checkpoint:
    ↪ noise_std={CONTEXT_PERTURB_NOISE_STD:.4f}, prob={CONTEXT_PERTURB_PROB:.4f}")

        _log(f"Resumed from {LAST_CKPT_PATH} at epoch {start_epoch}")

def _build_checkpoint_state(epoch: int, ckpt_cfg: dict, save_ema_as_primary:
    ↪ bool = False):
    """Bundle the current train state for best/last checkpoint saves."""
    online_state = _state_dict_for_checkpoint(diffusion)
    ema_state = _state_dict_for_checkpoint(ema_model)
    primary_state = ema_state if save_ema_as_primary and ema_state is not None
    ↪ else online_state
    return {
        'epoch': epoch,
        'model': primary_state,
        'model_online': online_state,
        'ema_model': ema_state,
        'optimizer': optimizer.state_dict(),

```

```

        'scaler': scaler.state_dict() if use_amp else None,
        'lr_scheduler': lr_scheduler.state_dict() if lr_scheduler is not None
    else None,
    'best_val': best_val,
    'best_val_selection': best_val,
    'best_rollout_loss': best_rollout,
    'best_one_step_val': best_one_step_val,
    'epochs_no_improve': epochs_no_improve,
    'global_step': global_step,
    'config': ckpt_cfg,
}

```

Training step (loss, jitter, one_epoch)

```

[ ]: def _compute_transition_focus(base_per_sample: torch.Tensor, is_transition:
    torch.Tensor) -> torch.Tensor:
    transition_loss = torch.zeros((), device=base_per_sample.device,
    dtype=base_per_sample.dtype)
    if TRANSITION_LOSS_WEIGHT > 0:
        transition_count = is_transition.sum()
        transition_focus = (base_per_sample * is_transition).sum() /
    transition_count.clamp_min(1.0)
        transition_loss = TRANSITION_LOSS_WEIGHT * transition_focus *
    (transition_count > 0).float()
    return transition_loss

def _compute_one_step_objective(
    x_ctx: torch.Tensor,
    x_tgt: torch.Tensor,
    frame_idx: torch.Tensor | None,
    phase_seq: torch.Tensor,
    tgt_phase: torch.Tensor,
    train: bool,
):
    time_map = diffusion.build_time_map(frame_idx, x_ctx.shape[-2], x_ctx.
    shape[-1]) if USE_TIME_MAP else None
    mse_loss, l1_loss, mse_per_sample, l1_per_sample = diffusion.p_losses(
        x_tgt,
        x_ctx,
        time_map=time_map,
        frame_idx=frame_idx,
        return_per_sample=True,
    )
    base_per_sample = mse_per_sample + L1_X0_WEIGHT * l1_per_sample
    base_loss = base_per_sample.mean()

```

```

is_transition = (phase_seq[:, -1] != tgt_phase).float()
transition_loss = _compute_transition_focus(base_per_sample, is_transition)
loss = (base_loss + transition_loss) if train else base_loss
# Return base_loss separately so the logged epoch average excludes the
# transition auxiliary term, making train and val losses directly
↪comparable.
    return loss, mse_loss, l1_loss, transition_loss, base_loss,
↪int(is_transition.sum().item()), int(x_tgt.size(0))

def _get_scheduled_sampling_prob(epoch_num: int) -> float:
    """Linearly ramp scheduled sampling probability from 0 to SS_MAX_PROB over
    SS_WARMUP_EPOCHS."""
    if not SCHEDULED_SAMPLING_ENABLED or SCHEDULED_SAMPLING_MAX_PROB <= 0:
        return 0.0
    if SCHEDULED_SAMPLING_WARMUP_EPOCHS <= 0:
        return float(SCHEDULED_SAMPLING_MAX_PROB)
    return float(SCHEDULED_SAMPLING_MAX_PROB) * min(1.0, epoch_num /
↪float(SCHEDULED_SAMPLING_WARMUP_EPOCHS))

def one_epoch(loader, train=True, desc: str | None = None, epoch_num: int = 0):
    """Run one epoch of training or validation over a loader.

    Args:
        loader: DataLoader for training or validation.
        train: If True, run in training mode with backward pass.
        desc: tqdm description string.
        epoch_num: current epoch number (used for scheduled sampling
    probability ramp).
    """
    global global_step
    diffusion.train(mode=train)
    if ema_model is not None:
        ema_model.eval()
    if LOG_MEMORY_STATS and torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()
    total = 0.0
    count = 0
    nan_batches = 0
    mse_total = 0.0
    l1_total = 0.0
    transition_total = 0.0
    transition_hits = 0
    sample_total = 0
    if desc is None:
        desc = "Train" if train else "Val"

```

```

# Scheduled sampling probability for this epoch (0 during validation).
cur_ss_prob = _get_scheduled_sampling_prob(epoch_num) if (train and
↳ SCHEDULED_SAMPLING_ENABLED) else 0.0

for batch_idx, batch in enumerate(tqdm_bar(loader, desc=desc)):
    if LOG_MEMORY_STATS and LOG_MEMORY_EVERY_N_BATCHES > 0 and (batch_idx +
↳ 1) % LOG_MEMORY_EVERY_N_BATCHES == 0:
        _log_memory_stats(f"{desc} batch {batch_idx + 1}/{len(loader)}")

    with torch.autocast(device_type='cuda', dtype=_amp_dtype,
↳ enabled=use_amp):
        # Unpack batch - all splits now return (x_prev_last, has_prev_last)
↳ as the
        # last two elements; x_prev_last is a zeros sentinel for val/test
↳ where
        # prev_last_path is always empty (SCHEDULED_SAMPLING_ENABLED checks
↳ split_name).
        if USE_TIME_MAP:
            x_ctx, x_tgt, frame_idx, phase_seq, tgt_phase, x_prev_last,
↳ has_prev_last = batch
            frame_idx = frame_idx.to(device, non_blocking=True)
        else:
            x_ctx, x_tgt, phase_seq, tgt_phase, x_prev_last, has_prev_last
↳ = batch
            frame_idx = None
            phase_seq = phase_seq.to(device, non_blocking=True, dtype=torch.
↳ long)
            tgt_phase = tgt_phase.to(device, non_blocking=True, dtype=torch.
↳ long)

        # Apply context perturbation in the main process so adaptive global
↳ updates
        # (CONTEXT_PERTURB_NOISE_STD / CONTEXT_PERTURB_PROB) take effect
↳ immediately.
        if train and PERTURB_CONTEXT:
            x_ctx = torch.stack(
                [_perturb_context_frames_tensor(x_ctx[i]) for i in
↳ range(x_ctx.size(0))],
                dim=0,
            )

            x_ctx = x_ctx.to(device, non_blocking=True, memory_format=torch.
↳ channels_last)
            x_tgt = x_tgt.to(device, non_blocking=True, memory_format=torch.
↳ channels_last)

```

```

B = x_ctx.size(0)

    # Scheduled sampling: replace the newest context frame with a
    ↪model-generated
    # version produced from the prior window [t-k, t-k+1, ..., t-1].
    # This trains the model to predict well when its own (imperfect)
    ↪output is
    # fed back as context, directly bridging the train/rollout
    ↪distribution gap.
    if train and cur_ss_prob > 0 and x_prev_last is not None and
    ↪has_prev_last is not None:
        has_prev_dev = has_prev_last.to(device)
        ss_apply = has_prev_dev & (torch.rand(B, device=device) <
    ↪cur_ss_prob)
        if ss_apply.any():
            # Only generate for the ss_apply subset - avoids running
            ↪full-batch
            # DDIM when only a fraction of samples need SS.
            ss_idx = ss_apply.nonzero(as_tuple=True)[0] # [n_ss]
            n_ss = ss_idx.numel()
            ctx_for_ss = torch.cat([
                x_prev_last[ss_idx.cpu()].to(device,
    ↪non_blocking=True), # [n_ss, 1, H, W]
                x_ctx[ss_idx, :-1],
    ↪ # [n_ss, k-1, H, W]
            ], dim=1) # [n_ss, k, H, W]
            ss_frame_idx = (frame_idx[ss_idx] - 1).clamp(0,
    ↪MAX_FRAME_INDEX) if USE_TIME_MAP else None
            with _temporary_diffusion_model(_eval_diffusion_model()):
                with torch.no_grad():
                    fake_last_ss = ddim_sample_from_model(
                        (n_ss, MODEL_OUT_CHANNELS, IMG_SIZE, IMG_SIZE),
                        x_ctx=ctx_for_ss,
                        frame_idx=ss_frame_idx,
                        steps=SCHEDULED_SAMPLING_DDIM_STEPS,
                    ) # [n_ss, 1, H, W]
                    # Scatter generated frames back into x_ctx for the ss_apply
                    ↪samples.
                    new_last = x_ctx[:, -1:].clone()
                    new_last[ss_idx] = fake_last_ss.detach().
    ↪to(memory_format=torch.channels_last)
                    x_ctx = torch.cat([x_ctx[:, :-1], new_last], dim=1)

    loss, mse_loss, l1_loss, transition_loss, base_loss,
    ↪batch_transition_hits, batch_sample_total = _compute_one_step_objective(

```

```

        x_ctx,
        x_tgt,
        frame_idx,
        phase_seq,
        tgt_phase,
        train=train,
    )

    transition_hits += batch_transition_hits
    sample_total += batch_sample_total

    # Check for NaN/Inf BEFORE backward (critical: don't corrupt model with
bad gradients)
    if torch.isnan(loss) or torch.isinf(loss):
        nan_batches += 1
        if nan_batches <= 3: # Only warn first few times
            _log(f"WARNING: NaN/Inf loss detected, skipping batch (total:
bad{nan_batches})")
            continue

    if train:
        optimizer.zero_grad(set_to_none=True)
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(diffusion.parameters(), GRAD_CLIP)
        scaler.step(optimizer)
        scaler.update()
        global_step += 1
        if ema_model is not None and global_step >=
badint(EMA_UPDATE_AFTER_STEP):
            _update_ema_model(diffusion, ema_model, decay=float(EMA_DECAY))

    # Accumulate base_loss (without transition term) so the logged average
# is symmetric with val loss, which also excludes the transition
badauxiliary.
    total += base_loss.item() * batch_sample_total
    count += batch_sample_total
    mse_total += mse_loss.item() * batch_sample_total
    l1_total += l1_loss.item() * batch_sample_total
    transition_total += transition_loss.item() * batch_sample_total

    if nan_batches > 0:
        _log(f" (Epoch had {nan_batches} NaN/Inf batches skipped)")
    avg = total / max(1, count)
    mse_avg = mse_total / max(1, count)
    l1_avg = l1_total / max(1, count)
    transition_avg = transition_total / max(1, count)

```

```

transition_frac = transition_hits / max(1, sample_total)
_log_memory_stats(f"{desc} end")
return avg, mse_avg, l1_avg, transition_avg, transition_frac

```

Validation metrics & fixed rows

```

[ ]: def _rows_to_tensors(rows: list[dict]):
    """Turn sampled metadata rows into batched tensors."""
    ctx_list = []
    tgt_list = []
    frame_idx_list = []
    for row in rows:
        ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']] # [k, H, W]
        x_ctx = torch.stack(ctx_tensors, dim=0)
        x_tgt = load_image(row['tgt_path']) # [1, H, W]
        ctx_list.append(x_ctx)
        tgt_list.append(x_tgt)
        if USE_TIME_MAP:
            frame_idx = min(int(row['tgt_frame']), int(MAX_FRAME_INDEX))
            frame_idx_list.append(frame_idx)
    x_ctx = torch.stack(ctx_list, dim=0)
    x_tgt = torch.stack(tgt_list, dim=0)
    if USE_TIME_MAP:
        frame_idx = torch.tensor(frame_idx_list, dtype=torch.long)
        return x_ctx, x_tgt, frame_idx
    return x_ctx, x_tgt

def _compute_metrics_from_rows(rows: list[dict], desc: str = "Val metrics",
    seed: int = METRIC_SEED):
    """Compute SSIM/LPIPS for a list of sampled pairs."""
    if not rows:
        return None, None
    ssim_vals = []
    lpips_vals = []
    gen = torch.Generator(device=device)
    gen.manual_seed(seed)
    batch_starts = list(range(0, len(rows), BATCH_SIZE))
    total_batches = len(batch_starts)
    if VAL_METRIC_MAX_BATCHES is not None and VAL_METRIC_MAX_BATCHES > 0:
        batch_starts = batch_starts[:VAL_METRIC_MAX_BATCHES]
        total_batches = len(batch_starts)
    for i in tqdm_bar(batch_starts, desc=desc, total=total_batches):
        batch_rows = rows[i:i + BATCH_SIZE]
        if USE_TIME_MAP:
            x_ctx, x_tgt, frame_idx = _rows_to_tensors(batch_rows)
            frame_idx = frame_idx.to(device, non_blocking=True)
        else:

```

```

        x_ctx, x_tgt = _rows_to_tensors(batch_rows)
        frame_idx = None
        x_ctx = x_ctx.to(device, non_blocking=True)
        x_tgt = x_tgt.to(device, non_blocking=True, memory_format=torch.
↪channels_last)
        out_ch = _target_channels()
        x_init = torch.randn((x_ctx.size(0), out_ch, IMG_SIZE, IMG_SIZE),
↪generator=gen, device=device)
        x_gen = sample_from_model(
            (x_ctx.size(0), out_ch, IMG_SIZE, IMG_SIZE),
            x_ctx=x_ctx,
            frame_idx=frame_idx,
            x_init=x_init,
        )
        ssim_vals.append(compute_ssim_metric(x_gen, x_tgt).item())
        if lpips_metric is not None:
            lpips_vals.append(compute_lpips_metric(x_gen, x_tgt).item())
        ssim = float(np.mean(ssim_vals)) if ssim_vals else None
        lpips = float(np.mean(lpips_vals)) if lpips_vals else None
        return ssim, lpips

def _collect_pairs_for_metrics(dataset: PairDataset, mode: str, max_samples:
↪int):
    """Sample rows across planes for metric evaluation."""
    rows = []
    planes = list(dataset.windows_by_plane.keys())
    for plane in planes:
        if len(rows) >= max_samples:
            break
        n_needed = max_samples - len(rows)
        if mode == 'transition':
            rows.extend(dataset.sample_transition_pairs_for_plane(plane,
↪n_needed))
        elif mode == 'non_transition':
            rows.extend(dataset.sample_non_transition_pairs_for_plane(plane,
↪n_needed))
        else:
            rows.extend(dataset.sample_pairs_for_plane(plane, n_needed))
    return rows

def _build_rollout_metric_row(dataset: PairDataset, base_row: dict, steps: int):
    """Expand one seed row with the next H real frames used as rollout targets.
↪"""
    if steps <= 0:
        return None

```

```

seq_key = str(base_row['seq_key'])
t_end = int(base_row['t_end'])
future_frames = [t_end + step for step in range(1, steps + 1)]
path_map = dataset.path_by_seq.get(seq_key, {})
phase_map = dataset.phase_by_seq.get(seq_key, {})
if any(frame not in path_map for frame in future_frames):
    return None

row = dataset._build_row(base_row)
row['rollout_tgt_frames'] = future_frames
row['rollout_tgt_paths'] = [path_map[frame] for frame in future_frames]
row['rollout_tgt_phase_ids'] = [phase_map.get(frame) for frame in
    ↪future_frames]
return row

def _collect_rollout_rows_for_metrics(dataset: PairDataset, steps: int,
    ↪max_samples: int, seed: int = METRIC_SEED):
    """Build a deterministic round-robin set of rollout seeds across planes."""
    if steps <= 0 or max_samples <= 0:
        return []

    rng = np.random.RandomState(seed + 1000 + int(steps))
    plane_buckets = {}
    for plane in sorted(dataset.windows_by_plane.keys()):
        base_rows = dataset.windows_by_plane.get(plane, [])
        candidates = []
        for base_row in base_rows:
            rollout_row = _build_rollout_metric_row(dataset, base_row, steps)
            if rollout_row is not None:
                candidates.append(rollout_row)
        if not candidates:
            continue
        order = rng.permutation(len(candidates)).tolist()
        plane_buckets[plane] = [candidates[int(i)] for i in order]

    rows = []
    plane_order = list(plane_buckets.keys())
    while len(rows) < max_samples:
        added = False
        for plane in plane_order:
            bucket = plane_buckets.get(plane, [])
            if not bucket:
                continue
            rows.append(bucket.pop())
            added = True
        if len(rows) >= max_samples:

```

```

        break
    if not added:
        break
    return rows

def _rows_to_rollout_tensors(rows: list[dict], steps: int):
    """Convert rollout-validation seed rows into batched context and H future_
    ↪targets."""
    if steps <= 0:
        raise ValueError(f'rollout steps must be positive, got {steps}')

    ctx_list = []
    t_end_list = []
    targets_by_step = [[] for _ in range(steps)]

    for row in rows:
        rollout_paths = row.get('rollout_tgt_paths')
        if rollout_paths is None or len(rollout_paths) != steps:
            raise ValueError(f"Expected {steps} rollout target paths for {row.
            ↪get('embryo_id')} ({row.get('plane')})")

        ctx_tensors = [load_image(p).squeeze(0) for p in row['ctx_paths']]
        ctx_list.append(torch.stack(ctx_tensors, dim=0))
        t_end_list.append(int(row['t_end']))

        for step_idx, tgt_path in enumerate(rollout_paths):
            targets_by_step[step_idx].append(load_image(tgt_path))

    x_ctx = torch.stack(ctx_list, dim=0)
    t_end = torch.tensor(t_end_list, dtype=torch.long)
    future_targets = [torch.stack(step_targets, dim=0) for step_targets in_
    ↪targets_by_step]
    return x_ctx, future_targets, t_end

@torch.no_grad()
def _compute_rollout_metrics_from_rows(rows: list[dict], steps: int, desc: str,
    ↪seed: int = METRIC_SEED):
    """Measure batched autoregressive rollout error on a fixed H-step_
    ↪validation subset."""
    if not rows or steps <= 0:
        return None, None, None

    out_ch = _target_channels()
    gen = torch.Generator(device=device)
    gen.manual_seed(seed + 2000 + int(steps))

```

```

total_loss = 0.0
total_mse = 0.0
total_l1 = 0.0
total_preds = 0

batch_starts = list(range(0, len(rows), BATCH_SIZE))
for i in tqdm_bar(batch_starts, desc=desc, total=len(batch_starts)):
    batch_rows = rows[i:i + BATCH_SIZE]
    x_ctx, future_targets, t_end = _rows_to_rollout_tensors(batch_rows,
↳steps=steps)

    current_ctx = x_ctx.to(device, non_blocking=True)
    current_t_end = t_end.to(device=device, non_blocking=True, dtype=torch.
↳long)

    for x_tgt in future_targets:
        x_tgt = x_tgt.to(device, non_blocking=True, memory_format=torch.
↳channels_last)

        if USE_TIME_MAP:
            frame_idx = torch.clamp(current_t_end + 1,
↳max=int(MAX_FRAME_INDEX))
        else:
            frame_idx = None

        x_init = torch.randn((current_ctx.size(0), out_ch, IMG_SIZE,
↳IMG_SIZE), generator=gen, device=device)
        x_gen = sample_from_model(
            (current_ctx.size(0), out_ch, IMG_SIZE, IMG_SIZE),
            x_ctx=current_ctx,
            frame_idx=frame_idx,
            x_init=x_init,
        )

        mse_per = (x_gen - x_tgt).pow(2).mean(dim=[1, 2, 3])
        l1_per = (x_gen - x_tgt).abs().mean(dim=[1, 2, 3])
        total_mse += mse_per.sum().item()
        total_l1 += l1_per.sum().item()
        total_loss += (mse_per + L1_X0_WEIGHT * l1_per).sum().item()
        total_preds += int(x_gen.size(0))

        # Feed the generated frame back into the temporal context.
        current_ctx = torch.cat([current_ctx[:, 1:], x_gen], dim=1)
        current_t_end = current_t_end + 1

if total_preds == 0:

```

```

        return None, None, None
    return total_loss / total_preds, total_mse / total_preds, total_l1 /
    ↪total_preds

# Fixed validation rows for deterministic metrics
fixed_val_rows = []
if COMPUTE_VAL_METRICS and VAL_METRIC_SAMPLES > 0:
    fixed_val_rows = _collect_pairs_for_metrics(val_dataset, mode='any',
    ↪max_samples=VAL_METRIC_SAMPLES)

# Fixed rollout seeds keep H-step validation comparable across epochs.
fixed_val_rollout_rows = []
if COMPUTE_VAL_ROLLOUT_METRICS and VAL_ROLLOUT_STEPS > 0 and
    ↪VAL_ROLLOUT_SAMPLES > 0:
    fixed_val_rollout_rows = _collect_rollout_rows_for_metrics(
        val_dataset,
        steps=VAL_ROLLOUT_STEPS,
        max_samples=VAL_ROLLOUT_SAMPLES,
        seed=METRIC_SEED,
    )

```

Rollout runner and grid saver `_advance_phase_sequence_for_rollout` updates the sliding phase context window at each rollout step. `_run_rollout` drives the autoregressive loop and returns per-step tensors and metadata. `_save_rollout_grid` wraps the loop and writes a context+rollout comparison figure.

```

[ ]: @torch.no_grad()
def _advance_phase_sequence_for_rollout(
    current_ctx: torch.Tensor,
    generated: torch.Tensor,
    phase_seq_tensor: torch.Tensor | None,
    plane_name: str | None = None,
):
    """Advance the rollout phase context and expose the chosen phase ID for
    ↪labelling.

    Returns:
        updated_phase_seq: new [1, k] phase tensor (oldest entry dropped, new
    ↪phase appended).
        phase_ids: dict mapping plane name to the predicted phase ID.
    """
    if phase_seq_tensor is None:
        return None, {}

    next_phase = choose_rollout_next_phase(phase_seq_tensor, generated)
    if next_phase is None:
        next_phase = phase_seq_tensor[:, -1:]

```

```

if next_phase.dim() == 1:
    next_phase = next_phase.view(-1, 1)
next_phase = next_phase.to(device=phase_seq_tensor.device, dtype=torch.long)

updated_phase_seq = torch.cat([phase_seq_tensor[:, 1:], next_phase], dim=1)
plane_key = str(plane_name or TARGET_PLANE)
phase_id = int(next_phase[0, 0].item()) if next_phase.numel() > 0 else None
return updated_phase_seq, {plane_key: phase_id}

@torch.no_grad()
def _run_rollout(
    ctx_tensors: list[torch.Tensor],
    t_end: int,
    steps: int,
    out_dir: Path,
    phase_seq_tensor: torch.Tensor | None = None,
    frame_index_cap: int | None = None,
    file_prefix: str = '',
    save_step_images: bool = True,
    plane_name: str | None = None,
):
    """Autoregressively generate `steps` future frames and optionally save each
    ↪ step.

    Args:
        ctx_tensors: list of k context tensors [1,H,W] or [H,W].
        t_end: frame index of the last context frame (used as the base for time
    ↪ conditioning).
        steps: number of autoregressive steps to generate.
        out_dir: directory where per-step images are saved (when
    ↪ save_step_images=True).
        phase_seq_tensor: optional [1, k] phase context for classifier guidance.
        frame_index_cap: clamp frame_idx at this value (defaults to
    ↪ MAX_FRAME_INDEX).
        file_prefix: prefix appended to each saved image filename.
        save_step_images: whether to write each generated frame to disk.
        plane_name: focal plane label used for logging and naming.

    Returns:
        List of dicts with 't_frame', 'path', 'images', and 'phase_ids' per
    ↪ step.
    """
    if not ctx_tensors:
        raise ValueError('ctx_tensors must not be empty')
    expected_k = int(getattr(diffusion, 'context_k', MODEL_CONTEXT_K))
    if len(ctx_tensors) != expected_k:

```

```

        raise ValueError(f"Need exactly {expected_k} context tensors, got {
↳len(ctx_tensors)}")

    out_dir.mkdir(parents=True, exist_ok=True)
    current_ctx = torch.stack([t.squeeze(0) if t.dim() == 3 else t for t in
↳ctx_tensors], dim=0).unsqueeze(0).to(device)
    current_t_end = int(t_end)
    generated_paths: list[dict] = []
    current_phase_seq = phase_seq_tensor.clone() if phase_seq_tensor is not
↳None else None
    row_plane = str(plane_name or TARGET_PLANE)

    for step_idx in range(int(steps)):
        frame_idx_tensor = None
        if USE_TIME_MAP:
            next_idx = current_t_end + 1
            if frame_index_cap is not None:
                next_idx = min(int(frame_index_cap), next_idx)
            frame_idx_tensor = torch.tensor([next_idx], device=device,
↳dtype=torch.long)

        x_gen = sample_from_model(
            (1, _target_channels(), IMG_SIZE, IMG_SIZE),
            current_ctx,
            frame_idx=frame_idx_tensor,
        )
        gen_cpu = x_gen[0, 0].detach().cpu() # [H, W]
        t_frame = current_t_end + 1

        phase_ids: dict[str, int | None] = {}
        if current_phase_seq is not None:
            current_phase_seq, phase_ids = _advance_phase_sequence_for_rollout(
                current_ctx=current_ctx,
                generated=x_gen,
                phase_seq_tensor=current_phase_seq,
                plane_name=row_plane,
            )

        save_path = out_dir / f"{file_prefix}gen_t{t_frame:05d}.png"
        if save_step_images:
            save_tensor_image(gen_cpu, save_path)

        generated_paths.append({
            't_frame': t_frame,
            'path': str(save_path),
            'images': {row_plane: gen_cpu},
            'phase_ids': phase_ids,

```

```

    })

    # Shift the context window: drop the oldest frame, append the generated
    ↪ one.
    current_ctx = torch.cat([current_ctx[:, 1:], x_gen], dim=1)
    current_t_end += 1

    return generated_paths

def _save_rollout_grid(row: dict, out_dir: Path, steps: int):
    """Autoregressively generate `steps` frames and save as a context-grid
    ↪ figure.

    Layout: context frames as columns (left), generated frames as columns
    ↪ (right).
    Rows correspond to distinct focal planes in the context (typically one for
    ↪ single-plane runs).
    """
    expected_k = int(getattr(diffusion, 'context_k', MODEL_CONTEXT_K))
    if len(row['ctx_paths']) != expected_k:
        raise ValueError(f"Need exactly {expected_k} context paths, got
        ↪ {len(row['ctx_paths'])}")

    row_plane = str(row.get('plane', TARGET_PLANE))
    ctx_tensors = [load_image(p) for p in row['ctx_paths']]
    t_end = int(row['t_end'])

    # Build phase context tensor if ground-truth labels are available.
    phase_seq_tensor = None
    row_phase_seq = row.get('phase_seq')
    if row_phase_seq is not None:
        if len(row_phase_seq) != expected_k:
            raise ValueError(f"phase_seq length {len(row_phase_seq)} must match
            ↪ context k={expected_k}")
        if any(p is None for p in row_phase_seq):
            raise ValueError(f"Missing phase labels in rollout row for
            ↪ {row['embryo_id']} ({row['plane']})")
        phase_seq_tensor = torch.tensor([row_phase_seq], device=device,
        ↪ dtype=torch.long)

    safe_embryo = re.sub(r'[^A-Za-z0-9_-]+', '_', str(row.get('embryo_id',
    ↪ 'embryo')))
    safe_plane = re.sub(r'[^A-Za-z0-9_-]+', '_', row_plane)
    rollout_prefix = f"{safe_embryo}_{safe_plane}_t{int(t_end):05d}_"

```

```

# Run the autoregressive rollout (images not saved individually; grid below
↳handles display).
generated_paths = _run_rollout(
    ctx_tensors=ctx_tensors,
    t_end=t_end,
    steps=steps,
    out_dir=out_dir,
    phase_seq_tensor=phase_seq_tensor,
    frame_index_cap=int(MAX_FRAME_INDEX),
    file_prefix=rollout_prefix,
    save_step_images=False,
    plane_name=row_plane,
)

# --- Build per-step lookup tables for the grid figure ---
gen_frame_order = [int(entry['t_frame']) for entry in generated_paths]
gen_lookup = {}
for entry in generated_paths:
    t_frame = int(entry['t_frame'])
    for plane_name, gen_tensor in entry.get('images', {}).items():
        gen_lookup[(t_frame, str(plane_name))] = (gen_tensor, entry.
↳get('phase_ids', {}).get(str(plane_name)))

# Resolve context display paths/metadata (fallback to model context if
↳vis-specific not set).
ctx_paths_vis = row.get('ctx_paths_vis', row['ctx_paths'])
ctx_frames_vis = row.get('ctx_frames_vis') or row.get('ctx_frames', [])
ctx_planes_vis = row.get('ctx_planes_vis') or [row_plane] *
↳len(ctx_paths_vis)
ctx_phases_vis = row.get('phase_seq_vis', row.get('phase_seq', [None] *
↳len(ctx_paths_vis)))
if len(ctx_phases_vis) != len(ctx_paths_vis):
    ctx_phases_vis = [None] * len(ctx_paths_vis)

frame_order_ctx = list(dict.fromkeys(ctx_frames_vis))
plane_order = list(dict.fromkeys(ctx_planes_vis)) or [row_plane]
if row_plane not in plane_order:
    plane_order.append(row_plane)

ctx_lookup = {
    (int(fr), str(pl)): (str(pth), ph)
    for pth, fr, pl, ph in zip(ctx_paths_vis, ctx_frames_vis,
↳ctx_planes_vis, ctx_phases_vis)
    if fr is not None and pl is not None
}

```

```

# --- Draw grid figure ---
n_rows = max(1, len(plane_order))
n_cols = max(1, len(frame_order_ctx) + len(gen_frame_order))
fig_w = min(max(2.1 * n_cols + 3.0, 10.0), 42.0)
fig_h = min(max(1.7 * n_rows + 2.2, 7.0), 28.0)
fig = plt.figure(figsize=(fig_w, fig_h), constrained_layout=True)
grid = fig.add_gridspec(n_rows, n_cols, wspace=0.02, hspace=0.02)

for r, plane_name in enumerate(plane_order):
    # Context columns
    for c, frame_val in enumerate(frame_order_ctx):
        ax = fig.add_subplot(grid[r, c])
        entry = ctx_lookup.get((int(frame_val), str(plane_name)))
        if entry is None:
            ax.axis('off')
            continue
        img_path, phase_id = entry
        img = denorm(load_image(img_path).squeeze(0).unsqueeze(0).
↳unsqueeze(0))[0, 0].cpu().numpy()
        ax.imshow(img, cmap='gray')
        if r == 0:
            ax.set_title(f"t={int(frame_val)}", fontsize=9)
        if c == 0:
            ax.set_ylabel(str(plane_name), fontsize=9)
        ax.text(0.02, 0.04, _phase_name(phase_id), transform=ax.transAxes,
↳fontsize=7,
            color='white', bbox=dict(facecolor='black', alpha=0.45,
↳edgecolor='none', pad=1.2))
        ax.set_xticks([]); ax.set_yticks([])

    # Generated (rollout) columns
    for i, t_frame in enumerate(gen_frame_order):
        c = len(frame_order_ctx) + i
        ax = fig.add_subplot(grid[r, c])
        entry = gen_lookup.get((int(t_frame), str(plane_name)))
        if entry is None:
            ax.axis('off')
            continue
        gen_tensor, phase_id = entry
        img = denorm(gen_tensor.unsqueeze(0).unsqueeze(0))[0, 0].cpu().
↳numpy()
        ax.imshow(img, cmap='gray')
        if r == 0:
            ax.set_title(f"t={int(t_frame)}", fontsize=9)
        if c == 0:
            ax.set_ylabel(str(plane_name), fontsize=9)

```

```

        ax.text(0.02, 0.04, _phase_name(phase_id), transform=ax.transAxes,
        ↪ fontsize=7,
                color='white', bbox=dict(facecolor='black', alpha=0.45,
        ↪ edgecolor='none', pad=1.2))
        ax.set_xticks([]); ax.set_yticks([])

    fig.suptitle(
        f"{row['embryo_id']} ({row['plane']}) - context (left) ↪ rollout_
        ↪ (right)",
        fontsize=11,
    )
    save_path = out_dir / f"{rollout_prefix}rollout_grid_k{CONTEXT_K}_h{steps}.
    ↪ png"
    fig.savefig(save_path, dpi=150)
    plt.close(fig)

```

1.12.1 12.1 One-Step Sample Grid Helper

`_save_sample_grid` renders per-epoch qualitative grids in the same layout used by `diffusion_model_run.ipynb`: context frames above, predicted and ground-truth targets below. It is defined here (rather than with the other visualization helpers) so it can use the final classifier defined above.

Checkpoint policy - `BEST_CKPT_PATH`: best weighted validation-selection loss. When `USE_EMA=True`, stores EMA weights as the primary model state. - `BEST_ROLLOUT_CKPT_PATH`: best rollout-only validation loss at `H=VAL_ROLLOUT_STEPS`. Also stores EMA weights when available. - `BEST_ONE_STEP_CKPT_PATH`: best raw one-step `val_loss` (pure diffusion objective, without rollout contribution). Saved every epoch the one-step loss improves, regardless of whether `val_selection_loss` improves. - `LAST_CKPT_PATH`: latest epoch state for resume. Keeps both online and EMA states.

Sample outputs per epoch are written to `diffusion_samples/epoch_XX/{mixed,transitions,rollouts}`.

```

[ ]: def _save_sample_grid(rows: list[dict], out_dir: Path, title_suffix: str,
        ↪ precomputed_gen: list[torch.Tensor] | None = None):
        """Save one-step sample grids using the same layout as diffusion_model_run.
        ↪ ipynb."""
        if not rows:
            return
        out_dir.mkdir(parents=True, exist_ok=True)
        for idx, row in enumerate(rows):
            ctx_tensors_model = [load_image(path).squeeze(0) for path in
            ↪ row['ctx_paths']]
            ctx_paths_vis = row.get('ctx_paths_vis', row['ctx_paths'])
            ctx_tensors_vis = [load_image(path).squeeze(0) for path in
            ↪ ctx_paths_vis]
            tgt_tensor_cpu = load_image(row['tgt_path'])
            target_plane_name = row.get('plane', TARGET_PLANE)

```

```

    if precomputed_gen is not None and idx < len(precomputed_gen):
        x_gen = precomputed_gen[idx].to(device)
    else:
        x_ctx = torch.stack(ctx_tensors_model, dim=0).unsqueeze(0).
        ↪to(device)

        frame_idx = None
        if USE_TIME_MAP:
            frame_idx = torch.tensor(
                [min(int(row['tgt_frame']), int(MAX_FRAME_INDEX))],
                device=device,
                dtype=torch.long,
            )

        x_gen = sample_from_model(
            (1, _target_channels(), IMG_SIZE, IMG_SIZE),
            x_ctx=x_ctx,
            frame_idx=frame_idx,
        )

        ctx_frames = row.get('ctx_frames_vis', row.get('ctx_frames', [None] *
        ↪len(ctx_tensors_vis)))
        ctx_phases = row.get('phase_seq_vis', row.get('phase_seq', [None] *
        ↪len(ctx_tensors_vis)))
        ctx_planes = row.get('ctx_planes_vis', [row.get('plane', TARGET_PLANE)]
        ↪* len(ctx_tensors_vis))
        frame_order = list(dict.fromkeys(ctx_frames))
        plane_order = list(dict.fromkeys(ctx_planes))
        if not frame_order:
            frame_order = list(row.get('ctx_frames', []))
        if not plane_order:
            plane_order = [row.get('plane', TARGET_PLANE)]

        ctx_lookup = {}
        for i, ctx_tensor in enumerate(ctx_tensors_vis):
            frame_val = ctx_frames[i] if i < len(ctx_frames) else None
            plane_name = ctx_planes[i] if i < len(ctx_planes) else None
            phase_id = ctx_phases[i] if i < len(ctx_phases) else None
            if frame_val is None or plane_name is None:
                continue
            ctx_lookup[(frame_val, plane_name)] = (ctx_tensor, phase_id)

        n_rows = max(1, len(plane_order))
        n_cols = max(1, len(frame_order))
        fig_w = min(max(2.1 * n_cols + 3.5, 10.0), 30.0)
        fig_h = min(max(1.7 * n_rows + 3.0, 7.0), 26.0)

```

```

fig = plt.figure(figsize=(fig_w, fig_h), constrained_layout=True)
outer = fig.add_gridspec(2, 1, height_ratios=[n_rows, 1.2], hspace=0.18)

ctx_grid = outer[0].subgridspec(n_rows, n_cols, wspace=0.02, hspace=0.
↪02)

for row_idx, plane_name in enumerate(plane_order):
    for col_idx, frame_val in enumerate(frame_order):
        ax = fig.add_subplot(ctx_grid[row_idx, col_idx])
        entry = ctx_lookup.get((frame_val, plane_name))
        if entry is None:
            ax.axis('off')
            continue
        ctx_img_t, ctx_phase_id = entry
        img = denorm(ctx_img_t.unsqueeze(0).unsqueeze(0))[0, 0].cpu().
↪numpy()

        ax.imshow(img, cmap='gray')
        if row_idx == 0:
            ax.set_title(f't={frame_val}', fontsize=9)
        if col_idx == 0:
            ax.set_ylabel(str(plane_name), fontsize=9)
        ax.text(
            0.02,
            0.04,
            _phase_name(ctx_phase_id),
            transform=ax.transAxes,
            fontsize=7,
            color='white',
            bbox=dict(facecolor='black', alpha=0.45, edgecolor='none',
↪
↪pad=1.2),
        )
        ax.set_xticks([])
        ax.set_yticks([])

pred_grid = outer[1].subgridspec(1, 2, wspace=0.15)
ax_tgt = fig.add_subplot(pred_grid[0, 0])
tgt_img = denorm(tgt_tensor_cpu.unsqueeze(0))[0, 0].cpu().numpy()
ax_tgt.imshow(tgt_img, cmap='gray')
ax_tgt.set_title(
    f"Target {target_plane_name} | {_phase_name(row.
↪get('tgt_phase_id'))} | t={row.get('tgt_frame', '-')}"
)
ax_tgt.set_xticks([])
ax_tgt.set_yticks([])

ax_gen = fig.add_subplot(pred_grid[0, 1])
gen_img = denorm(x_gen)[0, 0].detach().cpu().numpy()
pred_phase = predict_phase_ids_from_generated(x_gen.detach())

```

```

gen_phase = int(pred_phase[0].item()) if pred_phase is not None else_
↳None
ax_gen.imshow(gen_img, cmap='gray')
ax_gen.set_title(
    f"Generated {target_plane_name} | {_phase_name(gen_phase)} | t={row.
↳get('tgt_frame', '-')}"
)
ax_gen.set_xticks([])
ax_gen.set_yticks([])

fig.suptitle(
    f"{row['embryo_id']} ({row['plane']}) - context columns are_
↳timesteps, rows are planes {title_suffix}",
    fontsize=11,
)
save_path = out_dir / f'sample_{idx:02d}.png'
fig.savefig(save_path, dpi=150)
plt.close(fig)

```

```

[ ]: for epoch in range(start_epoch, EPOCHS + 1):
    tl, tmse, tl1, ttrans, tfrac = one_epoch(
        train_loader,
        train=True,
        desc=f"Epoch {epoch:03d} Train",
        epoch_num=epoch,
    )
    with _temporary_diffusion_model(_eval_diffusion_model()):
        with torch.no_grad():
            vl, vmse, vl1, vtrans, vfrac = one_epoch(val_loader, train=False,
↳desc=f"Epoch {epoch:03d} Val")

    val_ssim = None
    val_lpips = None
    val_rollout_loss = None
    val_rollout_mse = None
    val_rollout_l1 = None
    with _temporary_diffusion_model(_eval_diffusion_model()):
        if COMPUTE_VAL_METRICS and VAL_METRIC_SAMPLES > 0:
            diffusion.eval()
            val_ssim, val_lpips = _compute_metrics_from_rows(fixed_val_rows,
↳desc="Val metrics (all)", seed=METRIC_SEED)

    # Rollout validation is not a training loss. It contributes only to the_
↳validation-selection objective.
    if COMPUTE_VAL_ROLLOUT_METRICS and fixed_val_rollout_rows:
        diffusion.eval()

```

```

        val_rollout_loss, val_rollout_mse, val_rollout_l1 =
↪_compute_rollout_metrics_from_rows(
            fixed_val_rollout_rows,
            steps=VAL_ROLLOUT_STEPS,
            desc=f"Val rollout H={VAL_ROLLOUT_STEPS}",
            seed=METRIC_SEED,
        )

    val_selection_loss = float(vl)
    if val_rollout_loss is not None and VAL_ROLLOUT_LOSS_WEIGHT > 0:
        val_selection_loss += float(VAL_ROLLOUT_LOSS_WEIGHT) *
↪float(val_rollout_loss)

    # Adaptive perturbation: nudge up noise_std and prob when the rollout gap
↪grows.
    # When val_rollout_loss > threshold * val_loss, the model is increasingly
↪sensitive
    # to clean context (exposure bias). Intensifying perturbation bridges the
↪gap.
    if PERTURB_CONTEXT and PERTURB_ADAPT_ENABLED and val_rollout_loss is not
↪None and float(vl) > 1e-8:
        # Normalise by rollout horizon so the ratio is per-step, not total.
        # This makes the threshold scale-independent regardless of
↪VAL_ROLLOUT_STEPS.
        rollout_ratio = (float(val_rollout_loss) / VAL_ROLLOUT_STEPS) /
↪float(vl)
        if rollout_ratio > PERTURB_ADAPT_ROLLOUT_RATIO_THRESHOLD:
            old_std = CONTEXT_PERTURB_NOISE_STD
            old_prob = CONTEXT_PERTURB_PROB
            CONTEXT_PERTURB_NOISE_STD = min(
                CONTEXT_PERTURB_NOISE_STD + PERTURB_ADAPT_STEP_SIZE,
                PERTURB_ADAPT_NOISE_STD_MAX,
            )
            CONTEXT_PERTURB_PROB = min(
                CONTEXT_PERTURB_PROB + PERTURB_ADAPT_STEP_SIZE,
                PERTURB_ADAPT_PROB_MAX,
            )
            if CONTEXT_PERTURB_NOISE_STD != old_std or CONTEXT_PERTURB_PROB !=
↪old_prob:
                _log(
                    f" [adapt-perturb] rollout_ratio={rollout_ratio:.3f} >
↪{PERTURB_ADAPT_ROLLOUT_RATIO_THRESHOLD} "
                    f"→ noise_std {old_std:.4f}→{CONTEXT_PERTURB_NOISE_STD:.
↪4f}, "
                    f"prob {old_prob:.4f}→{CONTEXT_PERTURB_PROB:.4f}"
                )

```

```

if lr_scheduler is not None:
    lr_scheduler.step()

current_lr = optimizer.param_groups[0].get('lr', LR)
train_transition_str = f"{ttrans:.4f}" if ttrans is not None else 'n/a'
val_transition_str   = f"{vtrans:.4f}" if vtrans is not None else 'n/a'
cur_ss_prob_log = _get_scheduled_sampling_prob(epoch)

# Show each component in its weighted form so the numbers sum to the total
↪ loss.
# mse is already SNR-weighted; l1_w = L1_XO_WEIGHT * raw_l1.
tl1_w = L1_XO_WEIGHT * tl1
vl1_w = L1_XO_WEIGHT * vl1

log_msg = (
    f"[Epoch {epoch:03d}] "
    f"TRAIN loss {tl:.5f} | mse {tmse:.5f} | l1_w {tl1_w:.5f} (raw {tl1:.5f})"
    ↪ f" | trans {train_transition_str} ({100*tfrac:.1f}%) ||"
    f" VAL loss {vl:.5f} | mse {vmse:.5f} | l1_w {vl1_w:.5f} (raw {vl1:.5f})"
    ↪ f" | trans {val_transition_str} ({100*vfrac:.1f}%) "
    f" | selection {val_selection_loss:.5f} | lr {current_lr:.2e}"
    f" | ss_prob {cur_ss_prob_log:.3f}"
)
if val_rollout_loss is not None:
    log_msg += (
        f" | rollout_h{VAL_ROLLOUT_STEPS}_loss {val_rollout_loss:.5f}"
        f" | rollout_h{VAL_ROLLOUT_STEPS}_mse {val_rollout_mse:.5f}"
        f" | rollout_h{VAL_ROLLOUT_STEPS}_l1 {val_rollout_l1:.5f}"
        f" | rollout_weight {VAL_ROLLOUT_LOSS_WEIGHT:.3f}"
    )
if val_ssim is not None:
    log_msg += f" | SSIM {val_ssim:.4f}"
if val_lpips is not None:
    log_msg += f" | LPIPS {val_lpips:.4f}"
_log(log_msg)

metric_history.append({
    'epoch': epoch,
    'train_loss': tl,
    'train_mse': tmse,
    'train_l1': tl1,
    'train_l1_weighted': tl1_w,
    'train_transition_loss': ttrans,
    'train_transition_frac': tfrac,

```

```

    'val_loss': vl,
    'val_selection_loss': val_selection_loss,
    'val_rollout_loss_weight': VAL_ROLLOUT_LOSS_WEIGHT,
    'val_mse': vmse,
    'val_l1': vl1,
    'val_l1_weighted': vl1_w,
    'val_transition_loss': vtrans,
    'val_transition_frac': vfrac,
    'val_rollout_loss': val_rollout_loss,
    'val_rollout_mse': val_rollout_mse,
    'val_rollout_l1': val_rollout_l1,
    'val_ssim': val_ssim,
    'val_lpips': val_lpips,
    'context_perturb_noise_std': CONTEXT_PERTURB_NOISE_STD,
    'context_perturb_prob': CONTEXT_PERTURB_PROB,
    'scheduled_sampling_prob': cur_ss_prob_log,
})
_save_metric_history(METRIC_HISTORY_PATH, metric_history)

ckpt_cfg = {
    'IMG_SIZE': IMG_SIZE,
    'TIMESTEPS': TIMESTEPS,
    'PHASE_LABELS': PHASE_LABELS,
    'FOCAL_PLANES': FOCAL_PLANES,
    'DDIM_STEPS': DDIM_STEPS,
    'DDIM_ETA': DDIM_ETA,
    'CONTEXT_K': CONTEXT_K,
    'MODEL_CONTEXT_K': MODEL_CONTEXT_K,
    'MODEL_OUT_CHANNELS': MODEL_OUT_CHANNELS,
    'TARGET_PLANE': TARGET_PLANE,
    'USE_EMA': USE_EMA,
    'EMA_DECAY': EMA_DECAY,
    'EMA_USE_FOR_EVAL': EMA_USE_FOR_EVAL,
    'EMA_UPDATE_AFTER_STEP': EMA_UPDATE_AFTER_STEP,
    'USE_MIN_SNR': USE_MIN_SNR,
    'MIN_SNR_GAMMA': MIN_SNR_GAMMA,
    'USE_LR_SCHEDULER': USE_LR_SCHEDULER,
    'LR_MIN': LR_MIN,
    'LR_WARMUP_EPOCHS': LR_WARMUP_EPOCHS,
    'LR_WARMUP_START_FACTOR': LR_WARMUP_START_FACTOR,
    'TRANSITION_LOSS_WEIGHT': TRANSITION_LOSS_WEIGHT,
    'PERTURB_CONTEXT': PERTURB_CONTEXT,
    'CONTEXT_PERTURB_PROB': CONTEXT_PERTURB_PROB,
    'CONTEXT_PERTURB_DECAY': CONTEXT_PERTURB_DECAY,
    'CONTEXT_PERTURB_NOISE_STD': CONTEXT_PERTURB_NOISE_STD,
    'CONTEXT_PERTURB_MIN_SCALE': CONTEXT_PERTURB_MIN_SCALE,
    'PERTURB_ADAPT_ENABLED': PERTURB_ADAPT_ENABLED,

```

```

        'PERTURB_ADAPT_ROLLOUT_RATIO_THRESHOLD': ␣
↪PERTURB_ADAPT_ROLLOUT_RATIO_THRESHOLD,
        'PERTURB_ADAPT_NOISE_STD_MAX': PERTURB_ADAPT_NOISE_STD_MAX,
        'PERTURB_ADAPT_PROB_MAX': PERTURB_ADAPT_PROB_MAX,
        'PERTURB_ADAPT_STEP_SIZE': PERTURB_ADAPT_STEP_SIZE,
        'COMPUTE_VAL_ROLLOUT_METRICS': COMPUTE_VAL_ROLLOUT_METRICS,
        'VAL_ROLLOUT_STEPS': VAL_ROLLOUT_STEPS,
        'VAL_ROLLOUT_SAMPLES': VAL_ROLLOUT_SAMPLES,
        'VAL_ROLLOUT_LOSS_WEIGHT': VAL_ROLLOUT_LOSS_WEIGHT,
        'PHASE_POLICY_MODE': PHASE_POLICY_MODE,
        'PHASE_POLICY_ADVANCE_THRESHOLD': PHASE_POLICY_ADVANCE_THRESHOLD,
        'USE_RESIDUAL_PREDICTION': USE_RESIDUAL_PREDICTION,
        'USE_V_PREDICTION': USE_V_PREDICTION,
    }

    if best_val is None or val_selection_loss < best_val - 1e-8:
        best_val = val_selection_loss
        epochs_no_improve = 0
        torch.save(_build_checkpoint_state(epoch, ckpt_cfg, ␣
↪save_ema_as_primary=EMA_USE_FOR_EVAL), BEST_CKPT_PATH)
        _log(f"Saved best weighted-val checkpoint to {BEST_CKPT_PATH}")
    else:
        epochs_no_improve += 1
        _log(f"No weighted-val improvement for {epochs_no_improve} epoch(s)")

    if val_rollout_loss is not None and (best_rollout is None or ␣
↪val_rollout_loss < best_rollout - 1e-8):
        best_rollout = val_rollout_loss
        torch.save(_build_checkpoint_state(epoch, ckpt_cfg, ␣
↪save_ema_as_primary=EMA_USE_FOR_EVAL), BEST_ROLLOUT_CKPT_PATH)
        _log(f"Saved best rollout checkpoint to {BEST_ROLLOUT_CKPT_PATH}")

    if best_one_step_val is None or float(vl) < best_one_step_val - 1e-8:
        best_one_step_val = float(vl)
        torch.save(_build_checkpoint_state(epoch, ckpt_cfg, ␣
↪save_ema_as_primary=EMA_USE_FOR_EVAL), BEST_ONE_STEP_CKPT_PATH)
        _log(f"Saved best one-step checkpoint to {BEST_ONE_STEP_CKPT_PATH}")

    torch.save(_build_checkpoint_state(epoch, ckpt_cfg), LAST_CKPT_PATH)
    _log(f"Saved last to {LAST_CKPT_PATH}")

    _log(f"\n--- Generating samples for epoch {epoch} ---")
    with _temporary_diffusion_model(_eval_diffusion_model()):
        diffusion.eval()
        with torch.no_grad():
            epoch_sample_root = OUT_SAMPLES_DIR / f'epoch_{epoch:02d}'
            mixed_dir = epoch_sample_root / 'mixed'

```

```

transitions_dir = epoch_sample_root / 'transitions'
rollouts_dir = epoch_sample_root / 'rollouts'
for d in (mixed_dir, transitions_dir, rollouts_dir):
    d.mkdir(parents=True, exist_ok=True)

mixed_rows = []
transition_rows = []
rollout_rows = []

for plane in SAMPLE_PLANES:
    mixed_rows.extend(val_dataset.sample_pairs_for_plane(plane,
↳SAMPLES_PER_PLANE))
    transition_rows.extend(val_dataset.
↳sample_transition_pairs_for_plane(plane, SAMPLES_PER_PLANE))
    rollout_seed = val_dataset.sample_pairs_for_plane(plane, 1)
    if rollout_seed:
        rollout_rows.append(rollout_seed[0])

if mixed_rows:
    _save_sample_grid(mixed_rows, mixed_dir, title_suffix='(mixed)')
if transition_rows:
    _save_sample_grid(transition_rows, transitions_dir,
↳title_suffix='(transition)')

if ENABLE_ROLLOUT_GRIDS:
    for stale_path in rollouts_dir.glob('*gen_t*.png'):
        stale_path.unlink()
    for row_rollout in rollout_rows:
        _save_rollout_grid(row_rollout, rollouts_dir,
↳steps=ROLLOUT_GRID_STEPS)

    # Release rollout-only classifier so the next training epoch starts without
↳extra VRAM pressure.
    if USE_CLASSIFIER_PHASE_POLICY:
        unload_phase_classifier()
        _log(f"Samples saved to {epoch_sample_root}\n")

    if EARLY_STOPPING_PATIENCE and EARLY_STOPPING_PATIENCE > 0 and
↳epochs_no_improve >= EARLY_STOPPING_PATIENCE:
        _log(f"Stopping early after {epochs_no_improve} epochs without
↳weighted-val improvement (patience={EARLY_STOPPING_PATIENCE}).")
        break

_log(f"\nTraining complete! Best weighted-val checkpoint saved to
↳{BEST_CKPT_PATH}")

```

Epoch 001 Train: 0%| 0/7152 [00:00<?, ?it/s]

```

Epoch 001 Val: 0%| 0/928 [00:00<?, ?it/s]
Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[Epoch 001] TRAIN loss 0.04958 | mse 0.02179 | l1_w 0.02779 (raw 1.38945) |
trans 0.0256 (20.0%) || VAL loss 0.02045 | mse 0.00645 | l1_w 0.01401 (raw
0.70027) | trans 0.0029 (2.7%) | selection 0.03235 | lr 1.90e-05 | ss_prob 0.015
| rollout_h3_loss 0.07930 | rollout_h3_mse 0.07554 | rollout_h3_l1 0.18824 |
rollout_weight 0.150
Saved best weighted-val checkpoint to /home/khasion/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best.pt
Saved best rollout checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_
diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-
256_lpb1_best_rollout_h3.pt
Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo
_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 1 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_m
odels/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_01

Epoch 002 Train: 0%| 0/7152 [00:00<?, ?it/s]
Epoch 002 Val: 0%| 0/928 [00:00<?, ?it/s]
Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[Epoch 002] TRAIN loss 0.01388 | mse 0.00487 | l1_w 0.00901 (raw 0.45030) |
trans 0.0070 (20.0%) || VAL loss 0.01067 | mse 0.00412 | l1_w 0.00655 (raw
0.32758) | trans 0.0033 (2.7%) | selection 0.02300 | lr 2.80e-05 | ss_prob 0.030
| rollout_h3_loss 0.08223 | rollout_h3_mse 0.07857 | rollout_h3_l1 0.18289 |
rollout_weight 0.150
Saved best weighted-val checkpoint to /home/khasion/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best.pt
Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo
_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 2 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_m
odels/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_02

```

Epoch 003 Train: 0%| 0/7152 [00:00<?, ?it/s]
Epoch 003 Val: 0%| 0/928 [00:00<?, ?it/s]
Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]
[Epoch 003] TRAIN loss 0.01017 | mse 0.00379 | l1_w 0.00639 (raw 0.31930) |
trans 0.0054 (20.2%) || VAL loss 0.00689 | mse 0.00328 | l1_w 0.00360 (raw
0.18023) | trans 0.0046 (2.7%) | selection 0.01878 | lr 3.70e-05 | ss_prob 0.045
| rollout_h3_loss 0.07928 | rollout_h3_mse 0.07577 | rollout_h3_l1 0.17523 |
rollout_weight 0.150
Saved best weighted-val checkpoint to /home/khasion/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best.pt
Saved best rollout checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_
diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-
256_lpb1_best_rollout_h3.pt
Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_
_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 3 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_m
odels/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_03

Epoch 004 Train: 0%| 0/7152 [00:00<?, ?it/s]
Epoch 004 Val: 0%| 0/928 [00:00<?, ?it/s]
Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]
[adapt-perturb] rollout_ratio=4.388 > 4.0 → noise_std 0.0700→0.0720, prob
0.5000→0.5020
[Epoch 004] TRAIN loss 0.00859 | mse 0.00337 | l1_w 0.00522 (raw 0.26087) |
trans 0.0045 (20.0%) || VAL loss 0.00590 | mse 0.00295 | l1_w 0.00295 (raw
0.14757) | trans 0.0027 (2.7%) | selection 0.01755 | lr 4.60e-05 | ss_prob 0.060
| rollout_h3_loss 0.07766 | rollout_h3_mse 0.07423 | rollout_h3_l1 0.17170 |
rollout_weight 0.150
Saved best weighted-val checkpoint to /home/khasion/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best.pt
Saved best rollout checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_
diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-
256_lpb1_best_rollout_h3.pt
Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_
_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 4 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_04

Epoch 005 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 005 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=4.248 > 4.0 → noise_std 0.0720→0.0740, prob 0.5020→0.5040

[Epoch 005] TRAIN loss 0.00774 | mse 0.00316 | l1_w 0.00458 (raw 0.22907) | trans 0.0042 (20.1%) || VAL loss 0.00610 | mse 0.00276 | l1_w 0.00335 (raw 0.16734) | trans 0.0011 (2.7%) | selection 0.01777 | lr 5.50e-05 | ss_prob 0.075 | rollout_h3_loss 0.07777 | rollout_h3_mse 0.07436 | rollout_h3_l1 0.17058 | rollout_weight 0.150

No weighted-val improvement for 1 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 5 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_05

Epoch 006 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 006 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.283 > 4.0 → noise_std 0.0740→0.0760, prob 0.5040→0.5060

[Epoch 006] TRAIN loss 0.00697 | mse 0.00302 | l1_w 0.00395 (raw 0.19757) | trans 0.0039 (20.0%) || VAL loss 0.00494 | mse 0.00259 | l1_w 0.00235 (raw 0.11742) | trans 0.0011 (2.7%) | selection 0.01669 | lr 6.40e-05 | ss_prob 0.090 | rollout_h3_loss 0.07832 | rollout_h3_mse 0.07488 | rollout_h3_l1 0.17197 | rollout_weight 0.150

Saved best weighted-val checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best.pt

Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 6 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_06

Epoch 007 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 007 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=4.622 > 4.0 → noise_std 0.0760→0.0780, prob 0.5060→0.5080

[Epoch 007] TRAIN loss 0.00745 | mse 0.00294 | l1_w 0.00451 (raw 0.22566) | trans 0.0041 (20.1%) || VAL loss 0.00568 | mse 0.00252 | l1_w 0.00315 (raw 0.15770) | trans 0.0010 (2.7%) | selection 0.01748 | lr 7.30e-05 | ss_prob 0.105 | rollout_h3_loss 0.07870 | rollout_h3_mse 0.07525 | rollout_h3_l1 0.17249 | rollout_weight 0.150

No weighted-val improvement for 1 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 7 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_07

Epoch 008 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 008 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=4.315 > 4.0 → noise_std 0.0780→0.0800, prob 0.5080→0.5100

[Epoch 008] TRAIN loss 0.00728 | mse 0.00287 | l1_w 0.00440 (raw 0.22017) | trans 0.0042 (19.9%) || VAL loss 0.00612 | mse 0.00248 | l1_w 0.00364 (raw 0.18214) | trans 0.0014 (2.7%) | selection 0.01800 | lr 8.20e-05 | ss_prob 0.120 | rollout_h3_loss 0.07922 | rollout_h3_mse 0.07574 | rollout_h3_l1 0.17374 | rollout_weight 0.150

No weighted-val improvement for 2 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 8 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_08

Epoch 009 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 009 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.462 > 4.0 → noise_std 0.0800→0.0820, prob 0.5100→0.5120

[Epoch 009] TRAIN loss 0.00660 | mse 0.00280 | l1_w 0.00380 (raw 0.18998) | trans 0.0036 (19.8%) || VAL loss 0.00487 | mse 0.00238 | l1_w 0.00249 (raw 0.12443) | trans 0.0014 (2.7%) | selection 0.01683 | lr 9.10e-05 | ss_prob 0.135 | rollout_h3_loss 0.07975 | rollout_h3_mse 0.07625 | rollout_h3_l1 0.17469 | rollout_weight 0.150

No weighted-val improvement for 3 epoch(s)

Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 9 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_09

Epoch 010 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 010 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.361 > 4.0 → noise_std 0.0820→0.0840, prob 0.5120→0.5140

[Epoch 010] TRAIN loss 0.00646 | mse 0.00278 | l1_w 0.00368 (raw 0.18408) | trans 0.0033 (20.0%) || VAL loss 0.00502 | mse 0.00236 | l1_w 0.00265 (raw 0.13274) | trans 0.0031 (2.7%) | selection 0.01712 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.08066 | rollout_h3_mse 0.07715 | rollout_h3_l1 0.17580 | rollout_weight 0.150

No weighted-val improvement for 4 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 10 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_10

Epoch 011 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 011 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.181 > 4.0 → noise_std 0.0840→0.0860, prob 0.5140→0.5160

[Epoch 011] TRAIN loss 0.00696 | mse 0.00274 | l1_w 0.00422 (raw 0.21089) | trans 0.0035 (20.1%) || VAL loss 0.00524 | mse 0.00233 | l1_w 0.00291 (raw 0.14539) | trans 0.0010 (2.7%) | selection 0.01746 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.08145 | rollout_h3_mse 0.07791 | rollout_h3_l1 0.17717 | rollout_weight 0.150

No weighted-val improvement for 5 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 11 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_11

Epoch 012 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 012 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=4.788 > 4.0 → noise_std 0.0860→0.0880, prob 0.5160→0.5180

[Epoch 012] TRAIN loss 0.00650 | mse 0.00270 | l1_w 0.00380 (raw 0.18994) | trans 0.0034 (20.0%) || VAL loss 0.00570 | mse 0.00231 | l1_w 0.00339 (raw 0.16969) | trans 0.0029 (2.7%) | selection 0.01799 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.08192 | rollout_h3_mse 0.07836 | rollout_h3_l1 0.17804 | rollout_weight 0.150

No weighted-val improvement for 6 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 12 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_12

Epoch 013 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 013 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.256 > 4.0 → noise_std 0.0880→0.0900, prob
0.5180→0.5200
[Epoch 013] TRAIN loss 0.00618 | mse 0.00266 | l1_w 0.00352 (raw 0.17603) |
trans 0.0027 (19.9%) || VAL loss 0.00523 | mse 0.00226 | l1_w 0.00297 (raw
0.14828) | trans 0.0009 (2.7%) | selection 0.01759 | lr 1.00e-04 | ss_prob 0.150
| rollout_h3_loss 0.08241 | rollout_h3_mse 0.07884 | rollout_h3_l1 0.17877 |
rollout_weight 0.150
No weighted-val improvement for 7 epoch(s)
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 13 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_m
odels/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_13

Epoch 014 Train: 0%| 0/7152 [00:00<?, ?it/s]
Epoch 014 Val: 0%| 0/928 [00:00<?, ?it/s]
Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=6.491 > 4.0 → noise_std 0.0900→0.0920, prob
0.5200→0.5220
[Epoch 014] TRAIN loss 0.00650 | mse 0.00263 | l1_w 0.00387 (raw 0.19337) |
trans 0.0035 (20.1%) || VAL loss 0.00426 | mse 0.00224 | l1_w 0.00202 (raw
0.10091) | trans 0.0013 (2.7%) | selection 0.01669 | lr 1.00e-04 | ss_prob 0.150
| rollout_h3_loss 0.08287 | rollout_h3_mse 0.07928 | rollout_h3_l1 0.17953 |
rollout_weight 0.150
Saved best weighted-val checkpoint to /home/khasion/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best.pt
Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo
_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 14 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_m
odels/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_14

Epoch 015 Train: 0%| 0/7152 [00:00<?, ?it/s]
Epoch 015 Val: 0%| 0/928 [00:00<?, ?it/s]
Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=6.783 > 4.0 → noise_std 0.0920→0.0940, prob
0.5220→0.5240
[Epoch 015] TRAIN loss 0.00630 | mse 0.00261 | l1_w 0.00370 (raw 0.18486) |
trans 0.0030 (20.0%) || VAL loss 0.00409 | mse 0.00223 | l1_w 0.00186 (raw
0.09282) | trans 0.0009 (2.7%) | selection 0.01656 | lr 1.00e-04 | ss_prob 0.150
| rollout_h3_loss 0.08317 | rollout_h3_mse 0.07956 | rollout_h3_l1 0.18013 |
rollout_weight 0.150
Saved best weighted-val checkpoint to /home/khasion/Projects/thesis/diffusion/em
bryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best.pt
Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo
_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 15 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_m
odels/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_15

Epoch 016 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 016 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.487 > 4.0 → noise_std 0.0940→0.0960, prob
0.5240→0.5260
[Epoch 016] TRAIN loss 0.00587 | mse 0.00258 | l1_w 0.00329 (raw 0.16432) |
trans 0.0030 (19.9%) || VAL loss 0.00510 | mse 0.00222 | l1_w 0.00288 (raw
0.14394) | trans 0.0032 (2.7%) | selection 0.01768 | lr 1.00e-04 | ss_prob 0.150
| rollout_h3_loss 0.08389 | rollout_h3_mse 0.08026 | rollout_h3_l1 0.18134 |
rollout_weight 0.150
No weighted-val improvement for 1 epoch(s)
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/ne
xtframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 16 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_m
odels/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_16

Epoch 017 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 017 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.978 > 4.0 → noise_std 0.0960→0.0980, prob 0.5260→0.5280
 [Epoch 017] TRAIN loss 0.00595 | mse 0.00256 | l1_w 0.00339 (raw 0.16927) | trans 0.0032 (20.0%) || VAL loss 0.00471 | mse 0.00220 | l1_w 0.00250 (raw 0.12515) | trans 0.0009 (2.7%) | selection 0.01737 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.08443 | rollout_h3_mse 0.08078 | rollout_h3_l1 0.18251 | rollout_weight 0.150
 No weighted-val improvement for 2 epoch(s)
 Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 17 ---
 Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
 Samples saved to
 /home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_17

Epoch 018 Train: 0%| 0/7152 [00:00<?, ?it/s]
 Epoch 018 Val: 0%| 0/928 [00:00<?, ?it/s]
 Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.224 > 4.0 → noise_std 0.0980→0.1000, prob 0.5280→0.5300
 [Epoch 018] TRAIN loss 0.00590 | mse 0.00256 | l1_w 0.00334 (raw 0.16710) | trans 0.0026 (20.1%) || VAL loss 0.00545 | mse 0.00218 | l1_w 0.00327 (raw 0.16330) | trans 0.0022 (2.7%) | selection 0.01826 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.08538 | rollout_h3_mse 0.08170 | rollout_h3_l1 0.18398 | rollout_weight 0.150
 No weighted-val improvement for 3 epoch(s)
 Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 18 ---
 Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
 Samples saved to
 /home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_18

Epoch 019 Train: 0%| 0/7152 [00:00<?, ?it/s]
 Epoch 019 Val: 0%| 0/928 [00:00<?, ?it/s]
 Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=7.387 > 4.0 → noise_std 0.1000→0.1000, prob 0.5300→0.5320
 [Epoch 019] TRAIN loss 0.00658 | mse 0.00254 | l1_w 0.00404 (raw 0.20187) | trans 0.0036 (20.0%) || VAL loss 0.00393 | mse 0.00217 | l1_w 0.00176 (raw

0.08814) | trans 0.0009 (2.7%) | selection 0.01700 | lr 1.00e-04 | ss_prob 0.150
| rollout_h3_loss 0.08710 | rollout_h3_mse 0.08335 | rollout_h3_l1 0.18741 |
rollout_weight 0.150

No weighted-val improvement for 4 epoch(s)

Saved best one-step checkpoint to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_best_one_step.pt

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 19 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_19

Epoch 020 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 020 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=6.362 > 4.0 → noise_std 0.1000→0.1000, prob 0.5320→0.5340

[Epoch 020] TRAIN loss 0.00585 | mse 0.00252 | l1_w 0.00333 (raw 0.16631) |
trans 0.0031 (20.1%) || VAL loss 0.00458 | mse 0.00216 | l1_w 0.00242 (raw 0.12120) |
trans 0.0009 (2.7%) | selection 0.01771 | lr 1.00e-04 | ss_prob 0.150
| rollout_h3_loss 0.08749 | rollout_h3_mse 0.08373 | rollout_h3_l1 0.18825 |
rollout_weight 0.150

No weighted-val improvement for 5 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 20 ---

Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_20

Epoch 021 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 021 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=6.370 > 4.0 → noise_std 0.1000→0.1000, prob 0.5340→0.5360

[Epoch 021] TRAIN loss 0.00573 | mse 0.00252 | l1_w 0.00321 (raw 0.16058) |
trans 0.0035 (20.1%) || VAL loss 0.00463 | mse 0.00217 | l1_w 0.00246 (raw 0.12299) |
trans 0.0025 (2.7%) | selection 0.01791 | lr 1.00e-04 | ss_prob 0.150
| rollout_h3_loss 0.08851 | rollout_h3_mse 0.08470 | rollout_h3_l1 0.19027 |

rollout_weight 0.150
No weighted-val improvement for 6 epoch(s)
Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 21 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_21

Epoch 022 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 022 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.716 > 4.0 → noise_std 0.1000→0.1000, prob 0.5360→0.5380

[Epoch 022] TRAIN loss 0.00559 | mse 0.00250 | l1_w 0.00310 (raw 0.15490) | trans 0.0029 (20.1%) || VAL loss 0.00521 | mse 0.00215 | l1_w 0.00306 (raw 0.15285) | trans 0.0036 (2.7%) | selection 0.01860 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.08930 | rollout_h3_mse 0.08546 | rollout_h3_l1 0.19190 | rollout_weight 0.150

No weighted-val improvement for 7 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 22 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_22

Epoch 023 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 023 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=5.955 > 4.0 → noise_std 0.1000→0.1000, prob 0.5380→0.5400

[Epoch 023] TRAIN loss 0.00592 | mse 0.00248 | l1_w 0.00344 (raw 0.17202) | trans 0.0033 (19.9%) || VAL loss 0.00506 | mse 0.00215 | l1_w 0.00291 (raw 0.14551) | trans 0.0025 (2.7%) | selection 0.01863 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.09046 | rollout_h3_mse 0.08658 | rollout_h3_l1 0.19410 | rollout_weight 0.150

No weighted-val improvement for 8 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 23 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_23

Epoch 024 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 024 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=6.260 > 4.0 → noise_std 0.1000→0.1000, prob 0.5400→0.5420

[Epoch 024] TRAIN loss 0.00592 | mse 0.00246 | l1_w 0.00346 (raw 0.17296) | trans 0.0033 (19.9%) || VAL loss 0.00491 | mse 0.00215 | l1_w 0.00276 (raw 0.13806) | trans 0.0020 (2.7%) | selection 0.01875 | lr 1.00e-04 | ss_prob 0.150 | rollout_h3_loss 0.09226 | rollout_h3_mse 0.08832 | rollout_h3_l1 0.19733 | rollout_weight 0.150

No weighted-val improvement for 9 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 24 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth
Samples saved to
/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_24

Epoch 025 Train: 0%| 0/7152 [00:00<?, ?it/s]

Epoch 025 Val: 0%| 0/928 [00:00<?, ?it/s]

Val rollout H=3: 0%| 0/4 [00:00<?, ?it/s]

[adapt-perturb] rollout_ratio=6.380 > 4.0 → noise_std 0.1000→0.1000, prob 0.5420→0.5440

[Epoch 025] TRAIN loss 0.00557 | mse 0.00244 | l1_w 0.00312 (raw 0.15615) | trans 0.0029 (20.0%) || VAL loss 0.00486 | mse 0.00215 | l1_w 0.00271 (raw 0.13566) | trans 0.0012 (2.7%) | selection 0.01882 | lr 9.99e-05 | ss_prob 0.150 | rollout_h3_loss 0.09304 | rollout_h3_mse 0.08907 | rollout_h3_l1 0.19859 | rollout_weight 0.150

No weighted-val improvement for 10 epoch(s)

Saved last to /home/khasion/Projects/thesis/diffusion/embryo_diffusion_models/nextframe_ctxk3_256px_u32-64-128-256-256_lpb1_last.pt

--- Generating samples for epoch 25 ---
Loaded phase classifier: /home/khasion/Projects/thesis/classifier/embryo_phase_models/phase_classifier_f0_f1=648.pth

Samples saved to

/home/khasion/Projects/thesis/diffusion/diffusion_samples/epoch_25

Epoch 026 Train: 0%| 0/7152 [00:00<?, ?it/s]

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[120], line 2
      1 for epoch in range(start_epoch, EPOCHS + 1):
----> 2     tl, tmse, tl1, ttrans, tfrc = one_epoch(
      3         train_loader,
      4         train=True,
      5         desc=f"Epoch {epoch:03d} Train",
      6         epoch_num=epoch,
      7     )
      8     with _temporary_diffusion_model(_eval_diffusion_model()):
      9         with torch.no_grad():

Cell In[116], line 135, in one_epoch(loader, train, desc, epoch_num)
    132         new_last[ss_idx] = fake_last_ss.detach().
    ↳to(memory_format=torch.channels_last)
    133         x_ctx = torch.cat([x_ctx[:, :-1], new_last], dim=1)
--> 135     loss, mse_loss, l1_loss, transition_loss, base_loss,
    ↳batch_transition_hits, batch_sample_total = _compute_one_step_objective(
    136         x_ctx,
    137         x_tgt,
    138         frame_idx,
    139         phase_seq,
    140         tgt_phase,
    141         train=train,
    142     )
    144 transition_hits += batch_transition_hits
    145 sample_total += batch_sample_total

Cell In[116], line 19, in _compute_one_step_objective(x_ctx, x_tgt, frame_idx,
    ↳phase_seq, tgt_phase, train)
    10 def _compute_one_step_objective(
    11     x_ctx: torch.Tensor,
    12     x_tgt: torch.Tensor,
    (...)
    16     train: bool,
    17 ):
    18     time_map = diffusion.build_time_map(frame_idx, x_ctx.shape[-2],
    ↳x_ctx.shape[-1]) if USE_TIME_MAP else None
--> 19     mse_loss, l1_loss, mse_per_sample, l1_per_sample =
    ↳diffusion.p_losses(
    20         x_tgt,
```

```

21         x_ctx,
22         time_map=time_map,
23         frame_idx=frame_idx,
24         return_per_sample=True,
25     )
26     base_per_sample = mse_per_sample + L1_X0_WEIGHT * l1_per_sample
27     base_loss = base_per_sample.mean()

```

Cell In[104], line 56, in DiffusionCond.p_losses(self, x_tgt, x_ctx, time_map,
↳ frame_idx, return_x0_pred, return_per_sample)

```

53 sqrt_alpha = sqrt_alphas_cumprod[t][:, None, None, None]
54 sqrt_one_minus = sqrt_one_minus_alphas_cumprod[t][:, None, None, None]
--> 56 pred =   

↳ self.forward(x_noisy, t, x_ctx, time_map=time_map, frame_idx=frame_idx)
58 # ---- Compute prediction target and raw per-sample MSE ----
59 if USE_V_PREDICTION:
60     # v-target = sqrt_alpha * eps - sqrt_one_minus * x0

```

File ~/Projects/thesis/diffusion/models.py:217, in DiffusionCond.forward(self,
↳ x_noisy, t, x_ctx, time_map, frame_idx)

```

215 x_in = self.build_input(x_noisy, x_ctx, time_map=time_map)
216 enc = self.encode_context(x_ctx, frame_idx=frame_idx)
--> 217 noise = self.unet(x_in, t, encoder_hidden_states=enc).sample
218 return noise

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/

```

↳ module.py:1775, in Module._wrapped_call_impl(self, *args, **kwargs)
1773     return self._compiled_call_impl(*args, **kwargs) # type:   

↳ ignore[misc]
1774 else:
-> 1775     return self._call_impl(*args, **kwargs)

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/

```

↳ module.py:1786, in Module._call_impl(self, *args, **kwargs)
1781 # If we don't have any hooks, we want to skip the rest of the logic in
1782 # this function, and just call forward.
1783 if not (self._backward_hooks or self._backward_pre_hooks or self.
↳ _forward_hooks or self._forward_pre_hooks
1784         or _global_backward_pre_hooks or _global_backward_hooks
1785         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1786     return forward_call(*args, **kwargs)
1788 result = None
1789 called_always_called_hooks = set()

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
↳ unets/unet_2d_condition.py:1163, in UNet2DConditionModel.forward(self, sample,
↳ timestep, encoder_hidden_states, class_labels, timestep_cond, attention_mask,
↳ cross_attention_kwargs, added_cond_kwargs, down_block_additional_residuals,
↳ mid_block_additional_residual, down_intrablock_additional_residuals,
↳ encoder_attention_mask, return_dict)
    1153     sample, res_samples = downsample_block(
    1154         hidden_states=sample,
    1155         temb=emb,
    (...)
    1160         **additional_residuals,
    1161     )
    1162 else:
-> 1163     sample, res_samples =
↳ downsample_block(hidden_states=sample, temb=emb)
    1164     if is_adapter and len(down_intrablock_additional_residuals) > 0:
    1165         sample += down_intrablock_additional_residuals.pop(0)

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1775, in Module._wrapped_call_impl(self, *args, **kwargs)
    1773     return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1774 else:
-> 1775     return self._call_impl(*args, **kwargs)

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1786, in Module._call_impl(self, *args, **kwargs)
    1781 # If we don't have any hooks, we want to skip the rest of the logic in
    1782 # this function, and just call forward.
    1783 if not (self._backward_hooks or self._backward_pre_hooks or self.
↳ _forward_hooks or self._forward_pre_hooks
    1784         or _global_backward_pre_hooks or _global_backward_hooks
    1785         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1786     return forward_call(*args, **kwargs)
    1788 result = None
    1789 called_always_called_hooks = set()

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
↳ unets/unet_2d_blocks.py:1357, in DownBlock2D.forward(self, hidden_states,
↳ temb, *args, **kwargs)
    1355 for resnet in self.resnets:
    1356     if torch.is_grad_enabled() and self.gradient_checkpointing:
-> 1357         hidden_states =
↳ self.gradient_checkpointing_func(resnet, hidden_states, temb)
    1358     else:
    1359         hidden_states = resnet(hidden_states, temb)

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
↳ modeling_utils.py:306, in ModelMixin.enable_gradient_checkpointing.<locals>.
↳ _gradient_checkpointing_func(module, *args)
    304 def _gradient_checkpointing_func(module, *args):
    305     ckpt_kwargs = {"use_reentrant": False} if is_torch_version(">=", "1
↳ 11.0") else {}
--> 306     return torch.utils.checkpoint.checkpoint(
    307         module.__call__,
    308         *args,
    309         **ckpt_kwargs,
    310     )

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/_compile.py:53
↳ in _disable_dynamo.<locals>.inner(*args, **kwargs)
    50     disable_fn = torch._dynamo.disable(fn, recursive, wrapping=False)
    51     fn.__dynamo_disable = disable_fn # type: ignore[attr-defined]
----> 53 return disable_fn(*args, **kwargs)

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/_dynamo/
↳ eval_frame.py:1044, in DisableContext.__call__.<locals>._fn(*args, **kwargs)
    1042 _maybe_set_eval_frame(_callback_from_stance(self.callback))
    1043 try:
-> 1044     return fn(*args, **kwargs)
    1045 finally:
    1046     set_eval_frame(None)

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/utils/
↳ checkpoint.py:503, in checkpoint(function, use_reentrant, context_fn,
↳ determinism_check, debug, early_stop, *args, **kwargs)
    501 # Runs pre-forward logic
    502 next(gen)
--> 503 ret = function(*args, **kwargs)
    504 # Runs post-forward logic
    505 try:

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1775, in Module._wrapped_call_impl(self, *args, **kwargs)
    1773 return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1774 else:
-> 1775     return self._call_impl(*args, **kwargs)

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1786, in Module._call_impl(self, *args, **kwargs)
    1781 # If we don't have any hooks, we want to skip the rest of the logic in
    1782 # this function, and just call forward.
    1783 if not (self._backward_hooks or self._backward_pre_hooks or self.
↳ _forward_hooks or self._forward_pre_hooks

```

```

1784         or _global_backward_pre_hooks or _global_backward_hooks
1785         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1786     return forward_call(*args, **kwargs)
1788 result = None
1789 called_always_called_hooks = set()

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/diffusers/models/
↳ resnet.py:327, in ResnetBlock2D.forward(self, input_tensor, temb, *args,
↳ **kwargs)
    323     deprecate("scale", "1.0.0", deprecation_message)
    325 hidden_states = input_tensor
--> 327 hidden_states = self.norm1(hidden_states)
    328 hidden_states = self.nonlinearity(hidden_states)
    330 if self.upsample is not None:
    331     # upsample_nearest_nhwc fails with large batch sizes. see https://
↳ github.com/huggingface/diffusers/issues/984

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1775, in Module._wrapped_call_impl(self, *args, **kwargs)
    1773     return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1774 else:
-> 1775     return self._call_impl(*args, **kwargs)

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ module.py:1786, in Module._call_impl(self, *args, **kwargs)
    1781 # If we don't have any hooks, we want to skip the rest of the logic in
    1782 # this function, and just call forward.
    1783 if not (self._backward_hooks or self._backward_pre_hooks or self.
↳ _forward_hooks or self._forward_pre_hooks
    1784         or _global_backward_pre_hooks or _global_backward_hooks
    1785         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1786     return forward_call(*args, **kwargs)
    1788 result = None
    1789 called_always_called_hooks = set()

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/modules/
↳ normalization.py:325, in GroupNorm.forward(self, input)
    324 def forward(self, input: Tensor) -> Tensor:
--> 325     return
↳ F.group_norm(input, self.num_groups, self.weight, self.bias, self.eps)

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/nn/functional.
↳ py:2956, in group_norm(input, num_groups, weight, bias, eps)
    2949     raise RuntimeError(
    2950         f"Expected at least 2 dimensions for input tensor but received
↳ {input.dim()}"
    2951     )

```

```

2952 _verify_batch_size(
2953     [input.size(0) * input.size(1) // num_groups, num_groups]
2954     + list(input.size()[2:]))
2955 )
-> 2956 return torch_group_norm(
2957     input, num_groups, weight, bias, eps, torch.backends.cudnn.enabled
2958 )

```

```

File ~/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/utils/
checkpoint.py:1129, in _checkpoint_hook.__init__.<locals>.pack_hook(x)
1128 def __init__(self, frame):
-> 1129     def pack_hook(x):
1130         # See Rule 4 above
1131         holder = _Holder()
1132         frame.weak_holders.append(weakref.ref(holder))

```

KeyboardInterrupt:

1.13 13. Training Curves

This cell loads `metric_history.json` when available and compares train loss, one-step validation, weighted validation selection, and rollout-aware validation.

```

[ ]: # Plot one-step, rollout, and weighted validation-selection curves
history_for_plot = globals().get('metric_history', [])
if METRIC_HISTORY_PATH.exists():
    try:
        with open(METRIC_HISTORY_PATH, 'r', encoding='utf-8') as f:
            history_for_plot = json.load(f)
    except Exception as exc:
        print(f"WARNING: failed to load {METRIC_HISTORY_PATH}: {exc}")

if history_for_plot:
    epochs = np.array([m.get('epoch', i + 1) for i, m in
->enumerate(history_for_plot)], dtype=float)
    train_loss = np.array([m.get('train_loss', np.nan) for m in
->history_for_plot], dtype=float)
    val_loss = np.array([m.get('val_loss', np.nan) for m in history_for_plot],
->dtype=float)
    val_selection_loss = np.array([
        m.get('val_selection_loss', m.get('val_loss', np.nan)) for m in
->history_for_plot
    ], dtype=float)
    rollout_loss = np.array([m.get('val_rollout_loss', np.nan) for m in
->history_for_plot], dtype=float)

```

```

rollout_mse = np.array([m.get('val_rollout_mse', np.nan) for m in
↳history_for_plot], dtype=float)
rollout_l1 = np.array([m.get('val_rollout_l1', np.nan) for m in
↳history_for_plot], dtype=float)
val_ssim = np.array([m.get('val_ssim', np.nan) if m.get('val_ssim') is not
↳None else np.nan for m in history_for_plot], dtype=float)
val_lpips = np.array([m.get('val_lpips', np.nan) if m.get('val_lpips') is
↳not None else np.nan for m in history_for_plot], dtype=float)

fig, axes = plt.subplots(2, 2, figsize=(16, 10))

axes[0, 0].plot(epochs, train_loss, label='Train loss')
axes[0, 0].plot(epochs, val_loss, label='One-step val loss')
axes[0, 0].plot(epochs, val_selection_loss, label='Weighted val selection')
axes[0, 0].set_title('Training vs validation objectives')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Loss')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].legend()

axes[0, 1].plot(epochs, val_loss, label='One-step val loss')
axes[0, 1].plot(epochs, val_selection_loss, label='Weighted val selection')
if np.isfinite(rollout_loss).any():
    axes[0, 1].plot(epochs, rollout_loss, label=f'Rollout_
↳H={VAL_ROLLOUT_STEPS} loss')
    best_rollout_idx = int(np.nanargmin(rollout_loss))
    axes[0, 1].scatter(epochs[best_rollout_idx],
↳rollout_loss[best_rollout_idx], s=50, label='Best rollout checkpoint')
    best_selection_idx = int(np.nanargmin(val_selection_loss))
    axes[0, 1].scatter(epochs[best_selection_idx],
↳val_selection_loss[best_selection_idx], s=50, label='Best weighted-val_
↳checkpoint')
    axes[0, 1].set_title('Validation objective comparison')
    axes[0, 1].set_xlabel('Epoch')
    axes[0, 1].set_ylabel('Loss')
    axes[0, 1].grid(True, alpha=0.3)
    axes[0, 1].legend()

if np.isfinite(rollout_mse).any():
    axes[1, 0].plot(epochs, rollout_mse, label=f'Rollout_
↳H={VAL_ROLLOUT_STEPS} MSE')
    if np.isfinite(rollout_l1).any():
        axes[1, 0].plot(epochs, rollout_l1, label=f'Rollout_
↳H={VAL_ROLLOUT_STEPS} L1')
    axes[1, 0].set_title('Rollout error components')
    axes[1, 0].set_xlabel('Epoch')

```

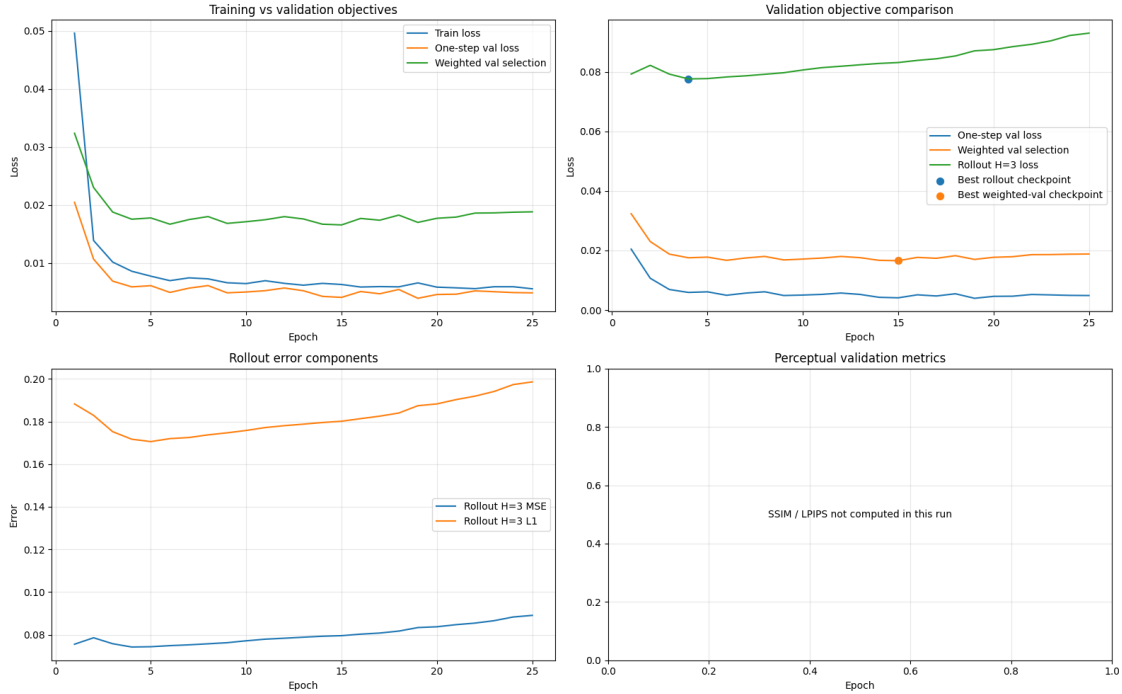
```

axes[1, 0].set_ylabel('Error')
axes[1, 0].grid(True, alpha=0.3)
if axes[1, 0].lines:
    axes[1, 0].legend()
else:
    axes[1, 0].text(0.5, 0.5, 'No rollout metrics logged yet', ha='center',
↪va='center', transform=axes[1, 0].transAxes)

has_perceptual = False
if np.isfinite(val_ssim).any():
    axes[1, 1].plot(epochs, val_ssim, label='Val SSIM')
    has_perceptual = True
if np.isfinite(val_lpips).any():
    axes[1, 1].plot(epochs, val_lpips, label='Val LPIPS')
    has_perceptual = True
axes[1, 1].set_title('Perceptual validation metrics')
axes[1, 1].set_xlabel('Epoch')
axes[1, 1].grid(True, alpha=0.3)
if has_perceptual:
    axes[1, 1].legend()
else:
    axes[1, 1].text(0.5, 0.5, 'SSIM / LPIPS not computed in this run',
↪ha='center', va='center', transform=axes[1, 1].transAxes)

plt.tight_layout()
plt.show()
else:
    print('No metrics logged yet.')

```



1.14 14. Inference Helpers

Utilities for one-step prediction and multi-step rollouts from user-provided context paths. Rollouts follow the plane inferred from the context file paths for naming/visualization only.

```
[ ]: # ---- Low-level path / context helpers ----

def _expected_context_channels() -> int:
    """Return the number of context frames the loaded diffusion model expects.
    ↪ """
    return int(getattr(diffusion, 'context_k', MODEL_CONTEXT_K))

def _stack_context_paths(context_paths: list[str]) -> torch.Tensor:
    """Load grayscale frames from disk and stack into a [1, k, H, W] tensor."""
    imgs = [load_image(p) for p in context_paths] # each [1, H, W]
    return torch.stack([img.squeeze(0) for img in imgs], dim=0).unsqueeze(0)

def _infer_frame_index(path: str) -> int | None:
    """Extract the 1-based frame number from a silver dataset filename (e.g.
    ↪ 00037.jpeg → 37)."""
    m = re.search(r"(\d+)\.jpe?g$", Path(path).name)
    return int(m.group(1)) if m else None
```

```

def _infer_context_plane(context_paths: list[str]) -> str:
    """Infer focal plane from context paths and validate single-plane
    consistency."""
    inferred = [
        part
        for p in context_paths
        for part in Path(p).parts
        if part in FOCAL_PLANE_SET
    ]
    if not inferred:
        raise ValueError(
            'Could not infer focal plane from context paths. '
            'Expected each path to include a focal-plane folder (e.g. F0, F-15,
            ...).'
        )
    plane = inferred[-1]
    if any(pl != plane for pl in inferred):
        raise ValueError('All context paths must be from the same focal plane.')
    return plane

def _initial_phase_seq_from_paths(context_paths: list[str]) -> torch.Tensor |
None:
    """Classify context frames to build an initial phase sequence for ad-hoc
    rollouts."""
    if not USE_CLASSIFIER_PHASE_POLICY:
        return None
    return infer_phase_sequence_from_context_paths(context_paths)

# ---- User-facing inference functions ----

def visualize_samples(num: int = 4, plane: str = 'F0', save_dir=None):
    """Generate and save the same sample grids as during training: mixed,
    transition, and rollout.

    Args:
        num: number of samples per category.
        plane: focal plane to sample from.
        save_dir: output directory (defaults to OUT_SAMPLES_DIR/ad_hoc_samples/
        <plane>).
    """
    if save_dir is None:
        save_dir = OUT_SAMPLES_DIR / 'ad_hoc_samples' / plane
    save_dir = Path(save_dir)

```

```

mixed_dir = save_dir / 'mixed'
transitions_dir = save_dir / 'transitions'
rollouts_dir = save_dir / 'rollouts'
for d in (mixed_dir, transitions_dir, rollouts_dir):
    d.mkdir(parents=True, exist_ok=True)

samples = val_dataset.sample_pairs_for_plane(plane, num)
if samples:
    _save_sample_grid(samples, mixed_dir, title_suffix='(mixed)')

trans_samples = val_dataset.sample_transition_pairs_for_plane(plane, num)
if trans_samples:
    _save_sample_grid(trans_samples, transitions_dir,
↳title_suffix='(transition)')

if ENABLE_ROLLOUT_GRIDS:
    for stale_path in rollouts_dir.glob('*gen_t*.png'):
        stale_path.unlink()
    rollout_seed = val_dataset.sample_pairs_for_plane(plane, 1)
    if rollout_seed:
        _save_rollout_grid(rollout_seed[0], rollouts_dir,
↳steps=ROLLOUT_GRID_STEPS)

if USE_CLASSIFIER_PHASE_POLICY:
    unload_phase_classifier()
print(f"Ad hoc samples saved to {save_dir}")

def predict_next_frame(context_paths: list[str], out_path: str, max_frame_index:
↳ int | None = None) -> dict[str, str]:
    """One-step next-frame prediction from the same focal plane as the provided
↳context.

    Args:
        context_paths: exactly k paths to the most recent context frames
↳(ascending frame order).
        out_path: base output path; the plane name is appended before the
↳extension.
        max_frame_index: override for the time-conditioning cap (default:
↳MAX_FRAME_INDEX).

    Returns:
        Dict mapping plane name → saved image path.
    """
    expected_k = _expected_context_channels()

```

```

    if len(context_paths) != expected_k:
        raise ValueError(f"Need exactly {expected_k} context frames, got {len(context_paths)}")

    plane_name = _infer_context_plane(context_paths)
    x_ctx = _stack_context_paths(context_paths).to(device)

    frame_cap = int(MAX_FRAME_INDEX if max_frame_index is None else max_frame_index)
    if frame_cap <= 0:
        raise ValueError(f"max_frame_index must be positive, got {frame_cap}")

    frame_idx = None
    if USE_TIME_MAP:
        t_end = _infer_frame_index(context_paths[-1]) or 0
        frame_idx = torch.tensor([min(t_end + 1, frame_cap)], device=device, dtype=torch.long)

    out_ch = _target_channels()
    x_gen = sample_from_model((1, out_ch, IMG_SIZE, IMG_SIZE), x_ctx=x_ctx, frame_idx=frame_idx)

    stem = Path(out_path).with_suffix('')
    plane_path = stem.parent / f"{stem.name}_{plane_name}.png"
    save_tensor_image(x_gen[0, 0], plane_path)
    return {plane_name: str(plane_path)}

def rollout_next_frames(
    initial_context_paths: list[str],
    steps: int,
    out_dir: str,
    max_frame_index: int | None = None,
    initial_phase_seq: list[int] | None = None,
) -> list[dict]:
    """Multi-step autoregressive rollout for the focal plane of the provided context.

    Args:
        initial_context_paths: exactly k paths to seed context frames (ascending frame order).
        steps: number of frames to generate autoregressively.
        out_dir: directory for generated frames.
        max_frame_index: time-conditioning cap (default: MAX_FRAME_INDEX).
        initial_phase_seq: list of k phase IDs for the initial context (auto-inferred if None).

```

```

Returns:
    List of per-step dicts with 't_frame', 'path', 'images', and
    ↪ 'phase_ids'.
    """
    expected_k = _expected_context_channels()
    if len(initial_context_paths) != expected_k:
        raise ValueError(f"Need exactly {expected_k} context frames, got
    ↪ {len(initial_context_paths)}")

    plane_name = _infer_context_plane(initial_context_paths)
    ctx_tensors = [load_image(p) for p in initial_context_paths]
    frame_candidates = [f for f in (_infer_frame_index(p) for p in
    ↪ initial_context_paths) if f is not None]
    t_end = max(frame_candidates) if frame_candidates else 0

    phase_seq_tensor = None
    if initial_phase_seq is not None:
        if len(initial_phase_seq) != expected_k:
            raise ValueError(f"initial_phase_seq must have length {expected_k}")
        phase_seq_tensor = torch.tensor([initial_phase_seq], device=device,
    ↪ dtype=torch.long)
    elif USE_CLASSIFIER_PHASE_POLICY:
        inferred = _initial_phase_seq_from_paths(initial_context_paths)
        if inferred is not None:
            phase_seq_tensor = inferred.to(device=device, dtype=torch.long)

    frame_cap = int(MAX_FRAME_INDEX if max_frame_index is None else
    ↪ max_frame_index)
    return _run_rollout(
        ctx_tensors=ctx_tensors,
        t_end=t_end,
        steps=steps,
        out_dir=Path(out_dir),
        phase_seq_tensor=phase_seq_tensor,
        frame_index_cap=frame_cap,
        plane_name=plane_name,
    )

```